



CONFIDENTIAL
Internal Reference

v1.0



CORE RDK-B SPECIFICATION

Broadband Platform – Core Feature Specification

RDK Central | 2026



Executive Cover

Foundation specification for adopting modular, vendor-independent RDK-B components across broadband gateway device profiles, HAL interfaces, certification, security, testing, and platform governance.

24

Core
Feature Areas



7

Device
Profiles



50+

RDK-B Open Source
Repos



RDK Ready
Certification



Prepared for

RDK BROADBAND COMMUNITY

Core RDK-B Specification

RDK-B
Broadband Platform

Document	RDK-B_CoreRDK_Spec_MVP(InternalReference)_v1.0
Prepared by	RDKM
Reviewed by	RDKM
Accepted by	RDKM
Abstract	This specification defines fundamental requirements for adopting RDK Broadband

Version	Date	Author	Description
1.0	2026-06	RDKM	Initial release. 24 features across 7 device profiles.

COPYRIGHT NOTICE, LICENSE AND DISCLAIMER (Draft; Subject to Legal Review)

Copyright © RDK Management, LLC (2025-2026). All rights reserved.

The information within this document is the property of RDK Management, LLC and its use and disclosure are restricted, except as expressly set forth herein.

RDK Management, LLC hereby grants you a fully paid, non-exclusive, nontransferable, worldwide, limited and revocable license (without the right to sublicense), under RDK Management, LLC's applicable copyright rights, to view, download, save, reproduce and use the document solely for your own internal purposes and in accordance with the terms of the license set forth herein.

This license does not authorize you to, and you expressly warrant that you shall not:

- (a) permit others (outside your organization) to use this document.
- (b) post or publish this document.
- (c) modify, adapt, translate, or otherwise change this document in any manner or create any derivative work based on this document.
- (d) remove or modify any notice or label on this document, including this Copyright Notice, License and Disclaimer.

RDK Management, LLC does not grant you any license hereunder other than as expressly stated herein.

Elements of this document may be subject to third party intellectual property rights, including without limitation, patent, copyright or trademark rights, and any such third party may or may not be a member of the RDK community. RDK Management, LLC members grant other RDK community members certain intellectual property rights as set forth in the RDK IPR Policy. RDK Management, LLC members do not grant you any rights under this license. RDK Management, LLC is not responsible for and shall not be held responsible in any manner for, identifying or failing to identify any or all such third-party intellectual property rights.

This document may reference specifications and technologies from third-party organizations including, but not limited to, the Wi-Fi Alliance, Broadband Forum, and Connectivity Standards Alliance. RDK Management, LLC makes no endorsement of any such third-party products, services, or specifications.

Please visit www.rdkcentral.com for more information on how to become a member of the RDK community.

This document and the information contained herein are provided on an "AS IS" basis and RDK Management, LLC DISCLAIMS ALL WARRANTIES EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO (A) ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OF THIRD PARTIES (INCLUDING WITHOUT LIMITATION ANY INTELLECTUAL PROPERTY RIGHTS INCLUDING PATENT, COPYRIGHT OR

TRADEMARK RIGHTS); OR (B) ANY IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE OR NONINFRINGEMENT.

IN NO EVENT WILL RDK MANAGEMENT, LLC BE LIABLE FOR ANY LOSS OF PROFITS, LOSS OF BUSINESS, LOSS OF USE OF DATA, INTERRUPTION OF BUSINESS, OR FOR ANY OTHER DIRECT, INDIRECT, SPECIAL OR EXEMPLARY, INCIDENTAL, PUNITIVE OR CONSEQUENTIAL DAMAGES OF ANY KIND, IN CONTRACT OR IN TORT, IN CONNECTION WITH THIS DOCUMENT OR THE INFORMATION CONTAINED HEREIN, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH LOSS OR DAMAGE.

All company, brand and product names in this document may be trademarks that are the sole property of their respective owners. This notice and disclaimer must be included on all copies of this document.

RDK Management, LLC
www.rdkcentral.com

Acknowledgements

This specification was developed by the RDK Management team with contributions from across the RDK Broadband community. The content reflects the collective engineering work, platform experience, and technical input of operators, silicon vendors, hardware manufacturers, system integrators, and open-source contributors who have built and deployed RDK-B at scale.

Special acknowledgement is due to the RDK Central Technical Advisory Board (RTAB) for their governance oversight and strategic direction, and to the working group members who contributed feature definitions, HAL interface specifications, and testing requirements that form the technical foundation of this document.

We also acknowledge the broader open-source community — the developers who have contributed code, filed issues, reviewed pull requests, and shared their deployment experience through the rdkcentral GitHub organization. This platform exists because of that collective effort.

Specification Basis & Document Conventions

This document follows architecture description practices consistent with ISO/IEC/IEEE 42010, the international standard for architecture description, applying its constructs of stakeholders, concerns, architecture viewpoints, and architecture views as reflected throughout this specification. Normative requirement language in this specification follows the keyword conventions defined in IETF RFC 2119 and RFC 8174; capitalized MUST, MUST NOT, SHOULD, SHOULD NOT, and MAY carry the meanings defined in those documents, while lowercase usage of the same words carries no normative force. Structurally, this document aligns with Broadband Forum Technical Report drafting conventions, consistent with its extensive reliance on Broadband Forum data models (TR-181, TR-369) throughout the specification. The tables below map the principal constructs of each referenced standard to their corresponding treatment in this specification.

Table A — ISO/IEC/IEEE 42010 Construct Mapping

42010 Construct	Treatment in This Specification	Status
System of Interest	The RDK-B platform (Section 2)	Defined
Stakeholder	Section 8 — ISPs, SoC Vendors, OEMs/ODMs, System Integrators, Third-Party App Developers, Cloud/Platform Service Providers, Standards Bodies	Defined
Concern	Section 3 (Goals) and Section 4 (Benefits and Value Proposition)	Addressed
Architecture Viewpoint	Section 2.3 (Five-Tier Model) and Section 2.5 (Layering/Standpoint)	Addressed
Architecture View	RDK-B architecture diagram (component/layer view)	Defined
Architecture Rationale	Section 6 (Roadmap), Section 7.2 (Governance Entry and Review Criteria), and platform-level engineering decisions documented elsewhere	Addressed; consolidation into a single rationale clause planned for a future revision
Correspondence Rules	Section 7.1.1 (Layering & Dependency Direction), Section 7.2.6 (Platform Layer Invocation Rules)	Defined

Table B — RFC 2119 / RFC 8174 Keyword Convention

Location	Current Treatment	Status
Section 10.1 (Feature Status Classification)	Capitalized MUST use correctly	Conformant
Section 7.1.1, 7.1.2, 7.1.3 (Development Standards)	Keywords currently expressed in lowercase form	Capitalization harmonization planned for next revision

Table C — Broadband Forum Technical Report Structural Alignment

BBF TR Convention	Treatment in This Specification	Status
Issue History / Document Control table	Front-matter Document table and Version/Date/Author/Description table	Defined
Purpose and Scope (separate clauses)	Combined under Section 1.1	Structurally merged
Definitions and Abbreviations (separate tables)	Combined under Section 1.3 Terminology	Structurally merged
Numbered, normative References clause	Not yet present as a standalone clause; external specifications currently referenced inline (Section 1.3, Section 7.1.3)	Planned for introduction in a future revision
Numbered requirement body	Sections 7, 9–13 (Architecture Standards, Device Profiles, Feature Matrix, Governance)	Defined

Technical Advisory Board Approved RDK-B Components

- <https://wiki.rdkcentral.com/spaces/RDK/pages/498925914/RDK9+Core+RDK+Broadband+Specification+Approved+by+TAB>

TABLE OF CONTENTS

Executive Summary	15
Key Strategic Objectives	15
1. Introduction	16
1.1 Purpose and Scope	16
1.2 Document Organization	16
1.3 Terminology	16
2. What is RDK-B?	18
2.1 What are RDK-B Components & NOT RDK-B Components	18
2.2 Platform Origins and Business Governance	19
2.3 Platform Architecture — Five-Tier Model	19
2.4 Licensing	19
2.5 RDK-B High level Architecture/Layering/Standpoint	20
End – End	20
RDKB Layering – Production Software Builds	20
	20
	20
RDKB Layering – Vendor Test Software Builds	20
	20
3. RDK-B Component Goals	21
3.1 Goal 1: Easier Component Reuse	21
3.2 Goal 2: Improved Modularity — Build-Time Dependencies	21
3.3 Goal 3: Improved Modularity — Runtime Dependencies	22
3.4 Goal 4: RDK Ready — Vendor Certification Program	23
3.5 Goal 5: Reduced Code Size	24
3.6 Goal 6: Consistent Interface Definitions	24
4. Benefits and Value Proposition	25
5. RDK-B Adoption and Industry Leadership	26
6. Roadmap and Next Steps	27
6.1 Next Release — Quick Wins	27
6.2 Future Releases — Structural Program	27
7. Architecture Standards and Governance	28
7.1 Development Standards	28
7.1.1 Process & Architecture Governance Standards	28
7.1.2 Implementation & Coding Standards	31

7.1.3 Industry Standards Conformance	33
7.2 Technical Governance Process	36
7.2.1. Purpose	36
7.2.2. Governance Scope	36
7.2.3. Classification of Changes	36
7.2.4. Governance Entry Criteria	37
7.2.5. Architecture Review Rules	38
7.2.6. Platform Layer Invocation Rules	39
7.2.7. Dependency Governance	40
7.2.8. Development Flow	41
7.2.9. Testing Governance	42
7.2.10. Testing Expectations by Change Type	43
7.2.11. Platform Test Matrix	44
7.2.12. Branch and Build Governance	45
7.2.13. Review and Approval Model	46
7.2.14. Merge Criteria	47
7.2.15. Test Evidence Requirements	47
7.2.16. Release Governance	48
7.2.17. Flag Governance	49
7.2.18. Reference Platform and Community Contribution Rules	50
7.2.19. Governance Checklist	50
7.2.20. Final Governance Rule	51
7.3 Component Governance Process	52
7.3.1 Purpose	52
7.3.2 Component Registry	52
7.3.3 Component Lifecycle States	53
7.3.4 Component Ownership Model	53
7.3.5 Periodic Health Review	54
7.3.7 Interface Stability Tagging	56
7.3.8 Component Governance Checklist	56
7.4 Escalation Process	57
8. RDK-B Ecosystem and Stakeholders	59
8.1 Internet Service Providers (ISPs / Operators)	59
Value Proposition for ISPs	59
ISP-Specific Feature Requirements	59
8.2 System-on-Chip (SoC) Vendors	60
Value Proposition for SoC Vendors	60
SoC Vendor Responsibilities in RDK-B	60
HAL Integration Model	60
8.3 Original Equipment Manufacturers (OEMs / ODMs)	61
OEM Responsibilities in RDK-B	61
OEM Build System	61
8.4 System Integrators (SIs)	61
What SIs Do in the RDK-B World	61
8.5 Third-Party Application Developers	62
The App Container Ecosystem	62
Categories of Third-Party Applications	62
Developer Access Model	63
8.6 Standards Bodies and Alliances	63

8.7 Summary: Stakeholder Value Matrix	63
9. RDK-B Deployment Models	64
10. Device Profiles	65
10.1 Feature Status Classification	65
11. Master Feature Matrix	66
12. Feature 1: Wi-Fi LAN	68
12.1 Overview	68
12.2 Key Capabilities	68
12.3 Feature Detail and Profile Matrix	68
12.4 Component Dependencies	72
12.5 Implementation Notes	72
12.6 Testing Scope	72
13. Feature 2: Ethernet LAN	74
13.1 Overview	74
13.2 Key Capabilities	74
13.3 Feature Detail and Profile Matrix	74
13.4 Component Dependencies	74
13.6 Testing Scope	75
14. Feature 3: MoCA LAN	76
14.1 Overview	76
14.2 Key Capabilities	76
14.3 Feature Detail and Profile Matrix	76
14.4 Component Dependencies	76
14.6 Testing Scope	76
15. Feature 4: Local WebUI	78
15.1 Overview	78
15.2 Key Capabilities	78
15.3 Feature Detail and Profile Matrix	78
15.4 Component Dependencies	78
15.6 Testing Scope	78
16. Feature 5: Remote Device Management	80
16.1 Overview	80
16.2 Key Capabilities	80
16.3 Feature Detail and Profile Matrix	80

16.4 Component Dependencies	81
16.5 Implementation Notes	81
16.6 Testing Scope	81
17. Feature 6: Telemetry & Data collection	82
17.1 Overview	82
17.2 Key Capabilities	82
17.3 Feature Detail and Profile Matrix	82
17.4 Component Dependencies	83
17.5 Implementation Notes	83
17.6 Testing Scope	83
18. Feature 7: Diagnostics & Testing	84
18.1 Overview	84
18.2 Key Capabilities	84
18.3 Feature Detail and Profile Matrix	84
18.4 Component Dependencies	85
18.5 Implementation Notes	85
18.6 Testing Scope	85
19. Feature 8: System Logging	86
19.1 Overview	86
19.2 Key Capabilities	86
19.3 Feature Detail and Profile Matrix	86
19.4 Component Dependencies	87
19.5 Implementation Notes	87
19.6 Testing Scope	87
20. Feature 9: Inter Process Communication	88
20.1 Overview	88
20.2 Key Capabilities	88
20.3 Feature Detail and Profile Matrix	88
20.4 Component Dependencies	89
20.5 Implementation Notes	89
20.6 Testing Scope	90
21. Feature 10: Matter/IoT	91
21.1 Overview	91
21.2 Key Capabilities	91

21.3 Feature Detail and Profile Matrix	91
21.4 Component Dependencies	92
21.6 Testing Scope	92
22. Feature 11: Downloadable App containers	93
22.1 Overview	93
22.2 Key Capabilities	93
22.3 Feature Detail and Profile Matrix	93
22.4 Component Dependencies	94
22.5 Implementation Notes	94
22.6 Testing Scope	94
23. Feature 12: Voice/VoIP	95
23.1 Overview	95
23.2 Key Capabilities	95
23.3 Feature Detail and Profile Matrix	95
23.4 Component Dependencies	96
23.5 Implementation Notes	96
23.6 Testing Scope	96
24. Feature 13: Firmware Upgrade	97
24.1 Overview	97
24.2 Key Capabilities	97
24.3 Feature Detail and Profile Matrix	97
24.4 Component Dependencies	98
24.5 Implementation Notes	98
24.6 Testing Scope	98
25. Feature 14: Connected device discovery	99
25.1 Overview	99
25.2 Key Capabilities	99
25.3 Feature Detail and Profile Matrix	99
25.4 Component Dependencies	100
25.6 Testing Scope	100
26. Feature 15: System startup & orchestration	101
26.1 Overview	101
26.2 Key Capabilities	101
26.3 Feature Detail and Profile Matrix	101

26.4 Component Dependencies	102
26.5 Implementation Notes	102
26.6 Testing Scope	102
27. Feature 16: Configuration Persistence	103
27.1 Overview	103
27.2 Key Capabilities	103
27.3 Feature Detail and Profile Matrix	103
27.4 Component Dependencies	103
27.5 Implementation Notes	104
27.6 Testing Scope	104
28. Feature 17: Factory reset	105
28.1 Overview	105
28.2 Key Capabilities	105
28.3 Feature Detail and Profile Matrix	105
28.4 Component Dependencies	105
28.5 Implementation Notes	106
28.6 Testing Scope	106
29. Feature 18: IP Routing/NAT/Firewall	107
29.1 Overview	107
29.2 Key Capabilities	107
29.3 Feature Detail and Profile Matrix	107
29.4 Component Dependencies	108
29.5 Implementation Notes	108
29.6 Testing Scope	108
30. Feature 19: Bridge and VLAN configuration	109
30.1 Overview	109
30.2 Key Capabilities	109
30.3 Feature Detail and Profile Matrix	109
30.4 Component Dependencies	109
30.6 Testing Scope	109
31. Feature 20: Traffic redirection	110
31.1 Overview	110
31.2 Key Capabilities	110
31.3 Feature Detail and Profile Matrix	110

31.4 Component Dependencies	110
31.6 Testing Scope	110
32. Feature 21: DHCP Server	111
32.1 Overview	111
32.2 Key Capabilities	111
32.3 Feature Detail and Profile Matrix	111
32.4 Component Dependencies	112
32.6 Testing Scope	112
33. Feature 22: DNS	113
33.1 Overview	113
33.2 Key Capabilities	113
33.3 Feature Detail and Profile Matrix	113
33.4 Component Dependencies	113
33.6 Testing Scope	114
34. Feature 23: WAN Connectivity	115
34.1 Overview	115
34.2 Key Capabilities	115
34.3 Feature Detail and Profile Matrix	116
34.4 Component Dependencies	117
34.5 Implementation Notes	118
34.6 Testing Scope	118
35. Feature 24: Parental control	119
35.1 Overview	119
35.2 Key Capabilities	119
35.3 Feature Detail and Profile Matrix	119
35.4 Component Dependencies	120
35.6 Testing Scope	120
36. Feature 25: WiFi Hotspot	121
36.1 Overview	121
36.2 Key Capabilities	121
36.3 Feature Detail and Profile Matrix	121
36.4 Component Dependencies	121
36.5 Implementation Notes	121
37. Feature 26: Code Security	122

37.1 Overview	122
37.2 Key Capabilities	122
37.3 Feature Detail and Profile Matrix	122
37.4 Component Dependencies	122
37.5 Implementation Notes	122
38. Security Framework	123
38.1 Security Principles	123
38.2 Wi-Fi Security Requirements	123
38.3 Management Channel Security	123
38.4 Firmware Security	124
38.5 Container Security (App Framework)	124
38.6 Security Testing Requirements	124
39. Performance Requirements	125
39.1 System Performance	125
39.2 Wi-Fi Performance Requirements	125
39.3 WAN Performance Requirements	126
39.4 Reliability Requirements	126
40. RDKB Component Testing (draft)	127
40.1 Test Framework Architecture	127
40.2 RDK Platform Test Suite (Mandatory for all Components — L2 + L3)	127
40.2.1 Mandatory Test Categories	127
40.2.2 Timing SLA Gates	129
40.2.3 Test Evidence Required for Release	129
40.3 OneWiFi Test Suite (OneWiFiTestSuite)	129
40.3.1 OneWiFi Test Suite (Non-EasyMesh tests)	129
40.3.2 OneWiFi Test Suite (EasyMesh Controller tests)	129
40.3.3 OneWiFi Test Suite (EasyMesh Agent tests)	129
40.4 Regression Campaign Structure	130
40.5 CI/CD Pipeline	130
40.5.1 Build CI	130
40.5.2 Hardware-in-the-Loop (HIL) CI	130
40.5.3 TIP Wi-Fi Validation	131
40.6 Lab Topology	131
40.7 Acceptance Test Criteria (ATC)	132
41. Legacy Component Migration Plan	133
41.1 Migration Principles	134
42. Component Repository Reference	135
43. Revision History	138

RDK-B BROADBAND PLATFORM

CORE FEATURE SPECIFICATION

Version 1.0 | RDK Central | 2026
Confidential — Internal Reference Document

Executive Summary

What are RDK-B Components?

RDK-B components are the foundational, open-source, vendor-independent building blocks of the RDK Broadband platform. They are modular, actively maintained by the RDK organization and community, and designed to be combined into software builds for a wide variety of broadband device types — from simple broadband modems to low cost Wi-Fi extenders to full-featured gateways with IoT and application container support.

This document is the authoritative specification for the RDK-B Component platform. It formally captures the platform's goals, design principles, ecosystem, all 24 feature areas with comprehensive per-feature analysis, the testing framework, security requirements, performance criteria, and the architectural roadmap. It is intended for the RDK team, engineering contributors, SoC vendors, OEMs, and system integrators.

24 Feature Areas	7 Device Profiles	50+ Open-Source Repos	5 Goals	135M+ Devices Globally	8+ Operator Deployments
----------------------------	-----------------------------	------------------------------------	-------------------	----------------------------------	--------------------------------------

Key Strategic Objectives

Objective	Description
Easier Component Reuse	RDKB components must work across all relevant device profile types without operator-specific modifications.
Improved Modularity	Eliminate unnecessary build-time and runtime dependencies between components.
RDK Ready Certification	Formal vendor test and certification for each HAL interface enabling mix-and-match SoC sourcing.
Reduced Code Size	Support the full range from small extender/modem platforms to high-end gateways.
Consistent Interface Standards	Uniform storage and RBUS IPC across all components — improving testability and talent pool.

1. Introduction

This document is the complete Feature Specification for the RDK-B (Reference Design Kit – Broadband) platform. It covers the platform's goals, design principles, stakeholder ecosystem, device profiles, all 24 feature areas, the testing framework, security framework, performance requirements, architectural roadmap, governance standards, and component repository references.

1.1 Purpose and Scope

- RDK team members defining feature scope, device profiles, and component inclusion decisions
- Engineering contributors integrating and validating RDK-B components on SoC hardware
- QA and test teams building test plans aligned to mandatory and optional features
- SoC vendors understanding HAL integration requirements and RDK Ready certification expectations
- OEMs / ODMs seeking to understand the reference software stack for product development
- System integrators defining customization scope and integration effort
- Third-party developers targeting the RDK-B application container ecosystem

1.2 Document Organization

Executive Summary provides a platform overview. Sections 1–2 introduce and define the platform. Section 3 covers the five Goals in depth. Sections 4–7 cover Benefits, Industry Leadership, Roadmap, and Governance. Sections 8–9 cover the Ecosystem and Deployment Models. Sections 10–11 define Device Profiles and the Master Feature Matrix. Sections 12–37 are the individual feature specifications (one section per feature). Section 38 is the Security Framework. Section 39 covers Performance Requirements. Section 40 covers Testing. Section 41 is the Legacy Component Migration Plan. Section 42 is the Repository Reference.

1.3 Terminology

Term	Definition
CORE	Core components are a small subset of foundational components forming basic infrastructure usable by other software, required across all device profiles, and eligible to be listed as a dependency by other components and features. Marked CORE in the Core Features column. Non-conformant if absent. Test coverage required.
Required	“Required” components designate those that are required in a specific RDKB profile. This designation is profile specific. A component may be required in some RDKB profiles and not required in other RDKB profiles.
OPT	“OPT” components designate those that are optional in a specific RDKB profile. Supported; included at operator/ODM discretion. Optional components frequently provide features that are needed for some operators, markets or devices but not necessarily all. This designation is profile specific. A component may be optional in some RDKB profiles and not optional in other RDKB profiles.
N/A	Not applicable to this device profile. This designation is profile specific. A component may be N/A in some RDKB profiles and not N/A in other RDKB profiles.
HAL	Hardware Abstraction Layer — boundary between RDK-B middleware and vendor SoC firmware. This term includes both industry standard interfaces (example: kernel interfaces) and special purpose APIs used for areas where industry standard interfaces do not exist or are not suitable.
RBUS	Modern RDK-B IPC bus, successor to CCSP/D-Bus. Newer components using RBUS are fully backward compatible and interoperable with older components that used the legacy CCSP/D-Bus approach.
CCSP	Common Component Software Platform — legacy IPC and data model framework. The transport implementation of the CCSP IPC no longer uses DBUS. It has been changed to use the RBUS transport mechanism and protocol so that it is interoperable with newer RBUS implementations.

Term	Definition
TR-181	Broadband Forum data model for broadband device parameters.
TR-369 / USP	User Services Platform — modern device management protocol (commands, events, bulk data).
TR-069 / CWMP	CPE WAN Management Protocol — legacy ACS-based device management.
OneWifi	Unified Wi-Fi management component abstracting all 802.11 radio operations.
EasyMesh	Wi-Fi Alliance multi-AP specification for seamless whole-home mesh networking.
DSM	Device Software Modules — TR-181 SoftwareModules app lifecycle management implementation.
PSM	Persistent Storage Manager — durable key-value config storage across reboots.
P&M	Provisioning and Management — legacy device init scripts (under active refactoring).
OCI	Open Container Initiative — container runtime and image specification standard.
T2	Telemetry 2.0 — profile-driven configurable telemetry collection framework.
xConf	Cross-Configuration — cloud service for RFC feature control and firmware orchestration.
CPE	Customer Premises Equipment — devices deployed at subscriber premises.
SoC	System-on-Chip — the silicon platform on which RDK-B runs.
OEM	Original Equipment Manufacturer — builds and supplies the CPE hardware.
ODM	Original Design Manufacturer — designs CPE hardware for other brands.
SI	System Integrator — customizes and integrates RDK-B for specific operator deployments.
ISP	Internet Service Provider / Operator — deploys CPE to subscribers.
RTAB	Technical Advisory Board — RDK Central governance body that provides technical direction for the RDK community. This is organized as a Broadband RTAB and a separate Video RTAB
NL80211	Industry-standard Linux Wi-Fi kernel interface used as the preferred RDK-B Wi-Fi HAL.
Distro Flag	Yocto DISTRO_FEATURES build-time flag controlling conditional feature inclusion.
DFS	Dynamic Frequency Selection — radar detection and channel avoidance for 5/6 GHz Wi-Fi.
MLO	Multi-Link Operation — Wi-Fi 7 feature enabling simultaneous use of multiple radio bands.
PD	Prefix Delegation — IPv6 mechanism for ISP to allocate a prefix block to the gateway.
SLAAC	Stateless Address Autoconfiguration — IPv6 client self-configures address from RA prefix.

2. What is RDK-B?

Platform Definition

RDK-B (Reference Design Kit – Broadband) is an open-source, modular, operator-grade middleware platform providing a standardized software stack for broadband residential and business devices. RDKB components are open-sourced, vendor-independent, actively maintained, and each represented by one or more GitHub repositories.

RDK-B sits between the hardware SoC/BSP layer and the operator's cloud management systems, providing a rich set of pre-integrated components for Wi-Fi, routing, diagnostics, telemetry, device management, IoT, and application containerization. Its HAL abstraction model means that a single software stack can run on radically different SoC platforms. Typically, each RDKB component contains a logical grouping of related features, and this modular approach allows software developers to select the set of high-level features required for each software build.

2.1 What are RDK-B Components & NOT RDK-B Components

RDKB Components Category	Description
Foundation	RDKB components" are foundational building blocks that can be combined in a modular way to create various software builds. RDKB components are open-sourced and vendor independent.
Component	An RDKB component is represented by one or more GitHub repos containing the code for that building block.
Logical grouping	Typically, each RDKB component contains a logical grouping of related features. This modular approach allows software developers to choose the set of high-level features required for each software build. Many RDKB components are considered optional to provide this flexibility
Actively maintained	Current RDKB components are actively maintained by the RDK organization and community.

Non-RDKB Component Category	Description
Platform-Specific Manifest Lists and Meta Layers	Yocto/BitBake manifests and meta layers specific to a particular hardware board. These are platform integration artefacts, not reusable middleware.
Platform-Specific Build Examples	Example builds and configurations targeting a specific SoC or reference board. Useful for developers but not part of the reusable RDKB Components
Platform-Specific HAL Implementations	HAL implementations for specific platforms (e.g., Banana Pi, specific vendor chipsets). The HAL interface definition used by active RDKB components is part of RDKB open source; the hardware dependent implementation is not.
Deprecated Code Preserved for History	Older components may become deprecated over time and not recommended for new software builds and not actively maintained or supported.

2.2 Platform Origins and Business Governance

RDK was created by Liberty Global and Comcast in 2012 for set-top boxes (RDK-V), then extended to broadband gateways (RDK-B) in 2014. RDK Central (rdkcentral.com) has governed the project since 2016 as a neutral community entity.

- 2012 — RDK inception by Liberty Global and Comcast for video (RDK-V)
- 2014 — RDK-B broadband stream launched; first DOCSIS gateway deployments
- 2016 — RDK Central LLC established as neutral governance entity
- 2018 — RBUS IPC introduced, replacing legacy CCSP/D-Bus
- 2020 — Wan Manager/Interface Managers Introduced
- 2021 — DSM and Dobby container runtime merged
- 2022 — USP/TR-369 published
- 2023 — OneWifi launched;
- 2024 — 100M+ devices deployed globally
- 2025 — Wi-Fi 7 (802.11be) integrated; EasyMesh Introduced
- 2026 — Matter/IoT (BartonCore) introduced

2.3 Platform Architecture — Five-Tier Model

Tier	Layer	Description
5 (Top)	Cloud / ACS	Operator cloud: ACS, USP Controller, xConf, WebConfig, Telemetry backends, Crash collection, Log aggregation.
4	Management Protocols	parodus/WebPA, usp-pa-vendor-rdk, tr069-protocol-agent, xconf-client, WebconfigFramework, T2 telemetry.
3	RDK-B Middleware	RDK-B components: OneWiFi, wan-manager, dhcp-manager, DSM, Dobby, BartonCore, all 24 feature components. All communicate via RBUS.
2	HAL / BSP	rdkb-halif-* interface libraries and vendor BSP. Boundary between RDK-B middleware and SoC firmware.
1 (Base)	Hardware / SoC	Physical silicon: SoC CPU, Wi-Fi radio, modem chipset, Ethernet switch, Flash/RAM.

2.4 Licensing

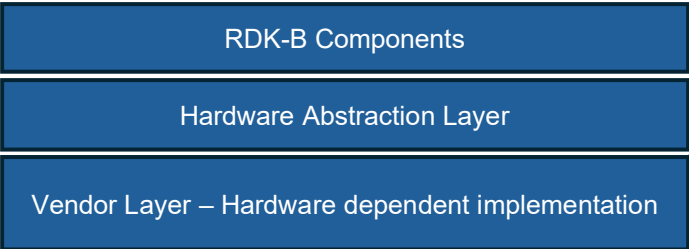
- Apache License 2.0 — primary license for most middleware components
- LGPL-2.1 — used by library components for compatibility with proprietary HAL implementations

2.5 RDK-B High level Architecture/Layering/Standpoint

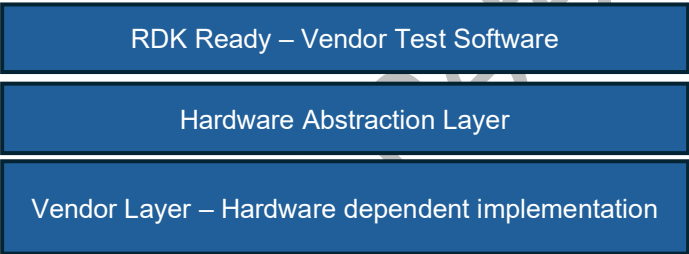
End – End

- To be defined

RDKB Layering – Production Software Builds



RDKB Layering – Vendor Test Software Builds



Test Suite	Definition	Owner
RDK Platform Test Suite	RDK Platform — component behavior, integration, data model, management, security, profile conformance	RDK-B Owner
Vendor Test Suite	Vendor Layer — HAL implementation correctness, SoC-specific behavior	HAL Spec Owner

3. RDK-B Component Goals

Foundation Statement

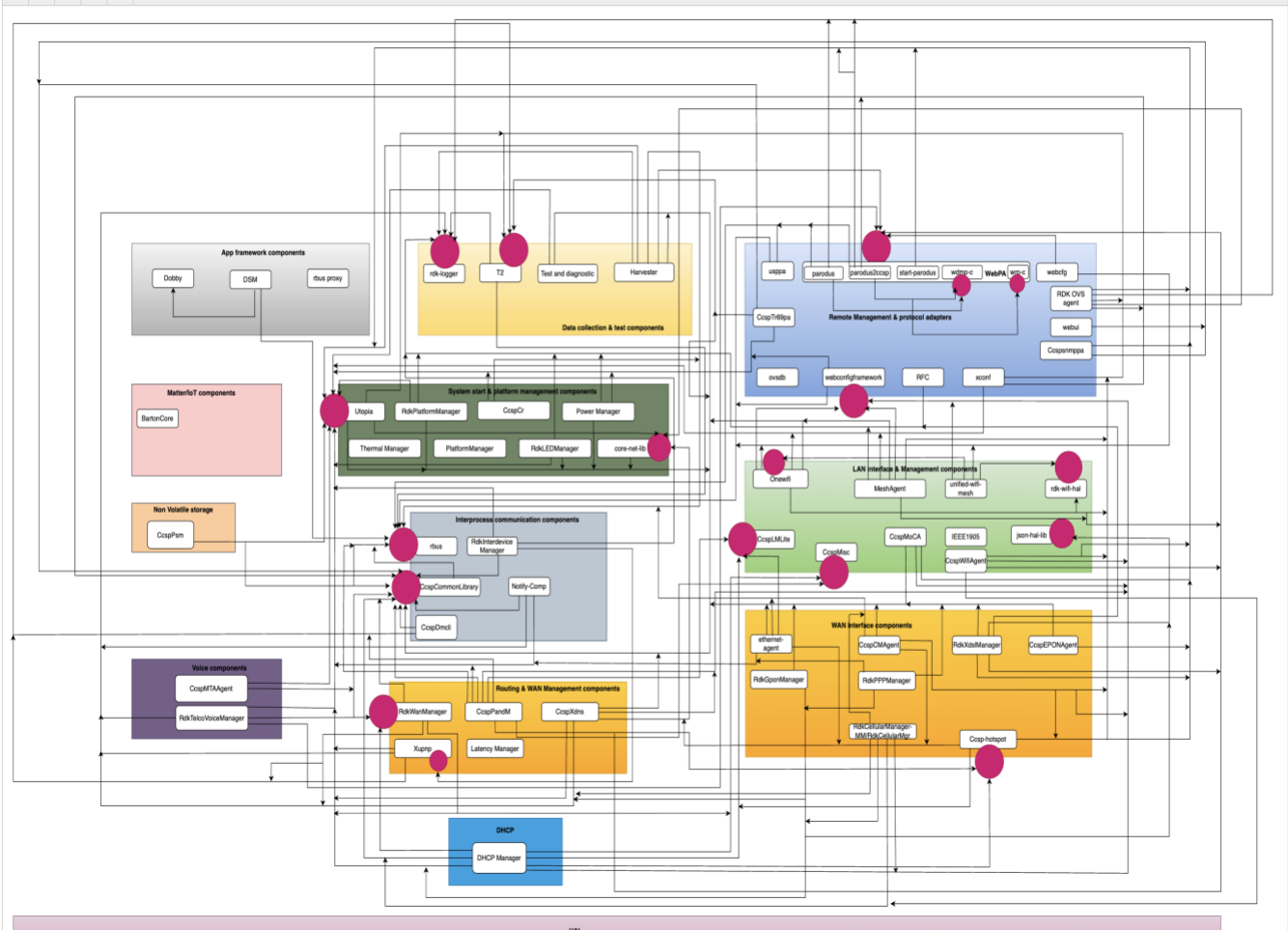
RDK-B Components are the foundation layer providing essential services for all RDK-B devices — from the smallest modem/ONU to the most capable gateway with IoT and app containers.

3.1 Goal 1: Easier Component Reuse

The Goal	The Challenge
Components must be easy to reuse across all device profile types: - Simple Modem/ONU (headless, no Wi-Fi) - Ethernet Router / Wi-Fi AP (standalone) - Gateway Router / Wi-Fi AP (full broadband GW) - Gateway Router + EasyMesh Controller - Easy Mesh Extender (agent-only profile) - Add-on profiles: App lifecycle management, IoT	Reuse was historically difficult because: - RDK Central releases focused only on Ethernet Wi-Fi Router profile - Comcast contributions focused only on Wi-Fi Gateway profile - Code written with profile-specific assumptions making reuse in other profiles difficult - Default values (e.g., Wi-Fi SSID names) were operator-specific — required changes before reusing - Operators had to fork and modify components to use them — increasing cost and time-to-market - High memory requirements prevented deployment on smaller device platforms
Foundation: RDK-B Components are the foundation layer providing essential services for all RDK-B devices.	

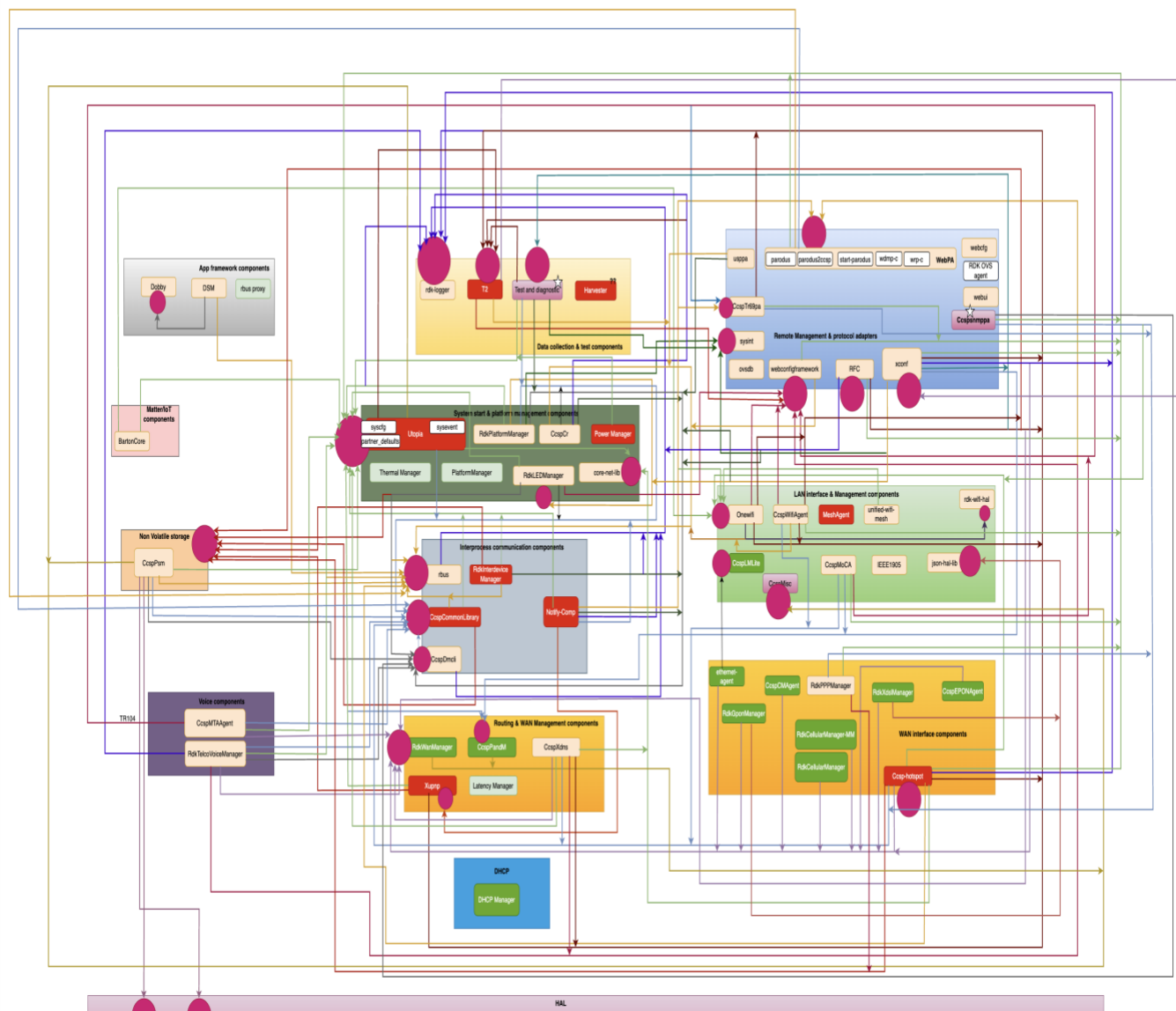
3.2 Goal 2: Improved Modularity — Build-Time Dependencies

The Goal	The Challenge
Avoid tight coupling and unnecessary build-time dependencies: - Developers must not manually edit/remove dependencies when reusing components - Optional components must be excludable without modifying other component build recipes - Build-time Distro Flags must clearly define what is included or excluded	Many unnecessary build-time dependencies exist today. Example — these components incorrectly depend on the Cable Modem HAL: - OneWifi (Wi-Fi should not depend on cable modem) - WAN Manager (should be access-technology agnostic) - DHCP Manager - Provisioning and Management (P&M) - Utopia - Test and Diagnostic - TR-069 Protocol Agent - xConf Client — and many others
Foundation: RDK-B Components are the foundation layer providing essential services for all RDK-B devices.	



3.3 Goal 3: Improved Modularity — Runtime Dependencies

The Goal	The Challenge
Runtime dependencies to optional components must be flexible: - Components must not crash or error when an optional dependency is absent - A component using another component's RBUS API must handle the case where that component is not running - Identify all inflexible runtime dependencies and refactor to dynamic patterns	Runtime dependencies today are not flexible. Example — these components hard-depend on T2 Telemetry (which is optional): - Utopia — crashes if T2 absent - RFC (Remote Feature Control) - CcspCr (CCSP Component Registrar) - Test and Diagnostic - TR-069 Protocol Agent - xConf Client - OneWifi, CcspWiFiAgent, CcspHotspot — and many others
Foundation: RDK-B Components are the foundation layer providing essential services for all RDK-B devices.	



3.4 Goal 4: RDK Ready — Vendor Certification Program

RDK Ready Pillar	Description
Standard and Uniform Testing	A test suite must exist for each HAL interface spec. All vendors run the same tests — published in rdkb-halif-test-* repositories.
Vendor Certification Program	Formal process to certify vendor compliance with specific HAL interface versions.
Modular Certification	SoC vendors only need to certify the HAL interfaces relevant to their silicon. Wi-Fi SoC vendors certify operation with the WiFi HAL; DOCSIS SoC vendors certify operation with the Cable Modem HAL.
Mix-and-Match SoC Support	Operators may combine SoCs from multiple vendors on one platform — Wi-Fi from Vendor A, GPON from Vendor B, DOCSIS from Vendor C.
Industry Standard Interfaces First	Where standards exist (e.g., NL80211 for Wi-Fi), use them as the RDK-B HAL. This reduces certification burden.

RDK Ready Pillar	Description
Supplementary Specs for Gaps	Where standard interfaces have functional gaps, RDK-B defines an extension spec — "NL80211 + RDK extensions" — covering the gaps.
Version Control and Compatibility	Tight version control between each HAL interface spec revision, the RDK-B component that consumes it, and the vendor code certified against it.

3.5 Goal 5: Reduced Code Size

The Goal	The Challenge
Code must scale from small modem/ONUs to high-end gateways: - Each component must support a well-defined, limited set of features - Code must allocate only the minimum required memory - Excluding components from a build must be clean and documented - Feature flags (Distro Flags) must cover sub-feature selection within components	Large, monolithic components make small lean profiles difficult: - Utopia and P&M are so large it is difficult to know what services they provide — must be refactored and split - CCSP Common Library allocates excessive memory — components must be refactored to avoid it - Documentation for including/excluding optional components is insufficient
Foundation: RDK-B Components are the foundation layer providing essential services for all RDK-B devices.	

3.6 Goal 6: Consistent Interface Definitions

The Goal	The Challenge
Uniform IPC improve testability, training, and talent pool: - Use RBUS as the standard IPC for all inter-component communication - Create test code that exercises IPC interfaces as they are actually used in production - Pre-validated RDKB Components reduce integration and testing cycles — faster time-to-market	Multiple inconsistent approaches fragment the developer experience: - Older components still use SysEvent, DMCLI, Common Lib and other legacy IPC — must be refactored to RBUS - Test code today often does not exercise IPC interfaces — components tested without validating integration contracts - The current logging approach needs re-examination for operational adequacy
Foundation: RDK-B Components are the foundation layer providing essential services for all RDK-B devices.	

4. Benefits and Value Proposition

Value Proposition

RDK-B Components enable easier code reuse, faster development, and consistent behavior across diverse vendors and device types.

Benefit Area	Specific Benefits
Modularity, Code Reuse and Scalability	Easier and faster code reuse across device profiles. Minimal build-time dependencies. Flexible runtime dependencies. Code size scales from small extenders to full gateways. Flexible logging and diagnostics.
RDK Ready Certification	Formal certification program for SoC vendors. Updated HAL interface definitions. Test code per HAL interface. Mix-and-match SoC sourcing with certified interoperability.
Developer Experience	Faster onramp for new developers. Easier training with consistent APIs. Larger talent pool. Better testability. Higher quality through consistent patterns.

5. RDK-B Adoption and Industry Leadership

Area	Key Points
Accelerated Market Adoption	Lower Entry Barriers: easier reuse improves schedules, reduces development time and costs. Broader Device Support: scales from low-cost hardware to full-featured gateways with IoT. Faster Time-to-Market: pre-validated components reduce integration cycles. Ecosystem Growth: more vendors participate with lower resource requirements.
Industry Standardization	Reference Implementation: RDK-B Component open source is the industry reference for broadband device software. Compliance and Certification: RDK Ready certifies hardware for rapid adoption. Interoperability: consistent behavior across vendors and platforms. Best Practices: establishes architectural patterns for the connected device industry.

Strategic Value

Positions RDK-B as the standard for next-generation connected broadband devices. Market impact: increase RDK-B deployments across operators and device types globally.

6. Roadmap and Next Steps

6.1 Next Release — Quick Wins

Work Area	Actions
Configuration Options (Distro Flags)	Increase number of Distro Flags to enable/disable features within components. Minimize unnecessary build-time dependencies identified in the dependency audit.
RDK Ready Configuration Management	Expand tested HAL interfaces to support vendor certification. Document how to include/exclude optional components in a build.
Integrated Testing and Documentation	Ramp up Test Working Group. Auto-discover supported component data model elements during testing and document this per component as the API contract. Use component-level versioning for documentation. Define RDKB component and HAL interface tests.

6.2 Future Releases — Structural Program

Work Area	Actions
Legacy Code Migration	Define Architecture Standards for new components. Break up monolithic components (Utopia, P&M) into smaller reusable modules. Migrate all components to RBUS (IPC) and a common storage API
Architecture	Define next-generation component architecture eliminating all legacy IPC and storage. Establish module boundaries, interface contracts, and dependency rules. Define migration path from current to new components.
Dependency Management	Full audit of all build-time and runtime dependencies. Resolve all unnecessary dependencies. Maintain published dependency matrix updated per release.
Testing Infrastructure	Expand test capacity to cover all certified platforms. Automate full regression across all profiles nightly. Integrate TIP Wi-Fi interoperability validation into CI. Expand the test infrastructure to verify that no components have dependencies with optional components so that optional components can be removed safely.

Expected Outcome

RDK-B code becomes cleaner and easier to reuse. Improves time to market and reduces development costs. Increases quality of RDKB open source and vendor code. Number of RDK-B deployments increase. Sets the standard for the industry.

7. Architecture Standards and Governance

7.1 Development Standards

7.1.1 Process & Architecture Governance Standards

Standard	Requirements
Component modularity and organization	New or refactored software components MUST contain only software that implements a grouping of similar features. This design helps to ensure that the component may be added to a software build only when that feature set is needed. Software to implement other unrelated features MUST NOT be included in the component. The software source code, build recipes, unit test mechanisms and documentation for a component SHOULD be contained in a single software repo. If any of this is contained in more than one repo, the documentation in each repo must reference all other software repos containing any of this information for the component. Distro Flags SHOULD be implemented for all sub-features that may not be required for all markets or operators.
Component IPC Guidelines	RBUS APIs are the authoritative interface for RDKB component to component interprocess communications (IPC). All new software contributions to existing and new components MUST exclusively use RBUS APIs for IPC with other RDKB components. Contributions to both existing and new components MUST be scanned using a common RDK static analysis tool to compare with previous versions of the component (if any) to ensure that no new usage of older IPC mechanisms is added, including syseventd, DMCLI or CCSP APIs.
Component Dependency Management Policy	Mandatory architecture review MUST be conducted for all new inter-component dependencies. Core components are those that are required in all RDKB profiles. RDKB components MUST NOT have build-time or run-time dependencies with non-core components. Components MAY have build-time or run-time dependencies with Core Components. Non-core components are those that are considered optional or not applicable in at least one RDKB profile. Core components are those that are required in all RDKB profile.
Component Unit Testing Requirements	All new components MUST ship with isolated unit tests requiring no external dependencies. HAL interfaces MUST have rdkb-halif-test-* suite before being declared stable. Test coverage metrics MUST be published per release. Testing must auto-discover the supported data model elements (including event and method usage) and auto-create a document of the supported data model elements when each feature flag is enabled. This document MUST (at a minimum) list all data model names provided in the component and which APIs are supported for each data model (set, get, add instance, delete instance, subscribe to event, un-subscribe, method invoke).
Data Model Documentation Standards	Data Model documentation MUST be sufficient to enable other developers/testers/system integrators to understand how to use each data model element. When a software contribution is received that updates an existing component or creates a new component, the contribution MUST contain detailed documentation of any new data models and changes to older data models. This MUST include the data model name, the supported RBUS APIs (examples: Set, Get, Add Instance, Subscribe, Unsubscribe, Method Invoke, Method Invoke Async), a detailed description of all properties, events, and input/output parameters. This documentation MUST also indicate if using the data model persists data and if that persistent data is reset to a known value on factory reboot. Before the contribution is accepted, the data models provided by the updated or new component must be auto-discovered using a run-time approach that queries the list of data models registered by the component. The documentation from each contribution MUST be compared against the list of auto-discovered data models to ensure that the documentation is complete and accurate before the contribution can be accepted. Any feature flags required to enable each data model must also be listed in the documentation. Any changes to data models must be documented in the software repo for that component and these changes must be included in the master list of all RDKB data models by component.

Standard	Requirements
Data Model Standards (TR-181)	Data models used in RDKB software contributions SHOULD follow the TR-181 standard when possible. Software contributions that use a data model name previously documented in the TR-181 standard MUST fully comply with the most recent TR-181 documentation for that name. If the most recent TR181 standard does not provide a data model name with the desired new functionality or the software contribution does not fully comply with how TR-181 standard describes how an existing name is used, then a new data model name MAY be created. New data model names MUST use the TR-181 organization and hierarchy when possible. New data model names MUST comply with the applicable extension format for non-standard names. New data model names SHOULD use the format: <existing TR-181 object hierarchy>_X_RDK-<new name>
HAL Documentation Standards	Each HAL interface must have a published interface specification document.
Interface Versioning & Backward Compatibility	All components and HAL interface MUST follow semantic versioning (MAJOR.MINOR.PATCH). Large functional changes and any change that is not backward compatible MUST increment the MAJOR version. Small planned features additions or bug fixes SHOULD increment the MINOR version. Unplanned bug fixes SHOULD increment the PATCH version. Deprecated interfaces SHOULD remain supported a minimum of two release cycles before removal.
Telemetry & Observability Standards	Components MUST emit logs conforming to the RDK-B common logging format. Critical state transitions SHOULD publish RBUS events and/or provide status properties that can be read by other RBUS components.
Build & Packaging Standards (Yocto/OE)	Components MUST build cleanly under the supported Yocto release with no layer dependency cycles. Recipes MUST declare explicit LICENSE and SPDX identifiers. Distro flags MUST gate all profile-specific or optional features.
Error Handling & Resilience Standards	Components MUST use defined RBUS error codes, not custom/undocumented codes. Crash-prone components MUST integrate with systemd watchdog and support clean restart without state corruption. Core dumps MUST be captured via the CrashUpload framework.
Naming & Repository Conventions	Component repos MUST follow the rdkb-<component> convention. New components MUST follow the Rdk* prefix and <Function> HAL suffix conventions already established in the architecture. Branch naming aligns with governance branch policy (7.2.12).
Code Quality & Static Analysis Standards	New C/C++ components MUST pass static analysis (cppcheck/Coverity) with zero high-severity findings before merge. Coding style MUST conform to the published RDK-B style guide. Memory safety tooling (ASan/Valgrind) runs in CI for native builds.
Northbound Protocol Onboarding	Any new remote-management protocol (beyond TR-369/TR-069/WebPA/Webconfig/SNMP/DAC/SSH) MUST be supported with a new Protocol Adapter component designed to abstract that protocol and allow the protocol to transparently send corresponding messages to other components using RBUS APIs. New protocols MUST NOT reach into RDKB components directly.
WAN Technology Onboarding	A new WAN access technology MUST integrate through RdkWanManager plus a technology-specific manager/agent (following the GPON/xDSL/Cellular/Docsis/EPON pattern); bespoke managers outside this pattern require architecture review.
Edge AI / ML Integration Standards*	AI/ML inference components MUST use vendor-neutral model formats (ONNX/RKNN), decoupled from any single NPU vendor SDK. Inference latency and resource budgets documented per model. Versioning/rollback supported for production deployment. <i>Layer placement (dedicated color-coded layer vs folded into other RDKB layers) still needs a decision before this is fully ready.</i>

Standard	Requirements
Integration of differentiating or proprietary software Third-Party Component Integration & HAL Extension Standards	Developers and System Integrators that desire to add differentiation or proprietary software on top of RDKB open source should follow one of two paths: 1) New differentiating or proprietary software that is hardware independent SHOULD use the existing TR-181 data models already supported by RDKB in order to control, optimize or extend RDKB features where possible. If new code needs to be added to the RDKB open source software to provide basic enabling features, then new data models to support this SHOULD be contributed to the RDKB community. This MAY be done while excluding differentiating or proprietary software by packaging that software in components that are NOT open sourced to the community. 2) New differentiating or proprietary software that is hardware dependent may be added in the vendor layer. This software MUST support RDK HAL API definitions. If HAL APIs need to be modified or extended, vendor MUST work with the RDK community to make these changes. This allows vendors to differentiate within the vendor layer while keeping a common interface to the vendor layer.

7.1.2 Implementation & Coding Standards

Standard	Requirements
IPC Mechanism Selection	RBUS is the default and only sanctioned IPC for new RDKB components. D-Bus is permitted only for legacy/vendor OSS that natively requires it (e.g. ModemManager), bridged into RBUS rather than consumed directly. USP/UDS is reserved for the USP Agent↔Controller boundary only. Downloaded Apps may use USP/UDS provided that a USP/UDS API Gateway converts these messages to RBUS. Any other IPC requires architecture review with documented justification.
Eventing Standards	State-change events SHOULD be published via RBUS event/subscribe, not polling Event payloads that contain a value SHOULD carry both updated parameter/value and the previous parameter/value. High-frequency events (>1/sec) SHOULD be rate-limited or coalesced at the publisher unless typical use cases require all high frequency events to be published
Storage & Persistence Standards	Data-model-backed parameters requiring persistence SHOULD use PSM or a new common method defined in the future. Structured local storage (logs, model artifacts) MUST use a defined path with documented size quotas with no writes to rootfs.
Footprint / CPU / RAM Budget	Every new component MUST document typical RAM, peak RAM, and flash footprint targets at design review, scoped to its device profile tier. Regressions beyond an agreed threshold versus prior release require architecture sign-off. Lower-tier profiles MUST never silently inherit other-profile footprint assumptions.
system()/Shell-out Usage	Calling system()/popen() from C/C++ daemons is discouraged by default; native library calls are preferred. Where unavoidable, the call MUST check return codes, sanitize interpolated input against shell injection, and never runs in a signal handler or high-frequency loop.
OSS Usage Standards	OSS dependencies come from the sanctioned Linux OSS layer list; new dependencies require review for license compatibility, CVE history, and maintenance activity. RDK-B wraps OSS via a thin adapter rather than embedding internals directly. Forking OSS instead of upstreaming requires approval plus a reconciliation plan.
@When to Use HAL	Any interaction with vendor/SoC-specific hardware or firmware MUST pass through a published HAL; direct vendor SDK calls MUST NOT be used.. A HAL is required only where behavior differs across silicon vendors — platform-independent behavior belongs in Linux OSS or a native component.
No RDK-B IPC Within vendor layer	Vendor layer HAL implementations MUST NOT contain TR-181/data-model awareness, RBUS usage or sysevent/syscfg calls.
Direct Kernel Interaction	Components talk to the kernel directly (syscalls, sysfs, procfs) only for OS-level facilities with no existing HAL/OSS abstraction. Any kernel interface used by more than one component is wrapped in a shared internal library, not duplicated. New kernel module/driver dependencies require platform-layer governance review (7.2.6).
Netlink vs Other Kernel Interfaces	Netlink is preferred for asynchronous kernel notifications (link state, routing, neighbor changes); polling sysfs/procfs where a netlink equivalent exists is prohibited. ioctl is permitted only for legacy drivers with no netlink support, isolated behind a HAL/adaptor.
C/C++ Coding Anti-Patterns	Blocking I/O MUST NOT occur on a component's main RBUS event-handling thread. strcpy/sprintf and unbounded buffers for variable-length input MUST NOT be used in favor of bounded equivalents; malloc return values MUST always be checked and an error must be logged and returned to higher level software Global mutable state shared across translation units is prohibited except through a documented, lock-protected accessor.
Sysevent Usage Standards	Sysevent usage is expected to be deprecated over time. New software contributions MUST NOT add to sysevent usage or add any new sysevent keys.
Scripting Usage Standards	Shell scripts MAY be used for one-shot boot/init orchestration and MUST NOT be used for long running stateful logic. Interpreted languages are discouraged on memory/flash-constrained profiles unless explicitly justified — native C/C++ or RUST SHOULD be used

Standard	Requirements
	for persistent daemons. Boot-time scripts MUST have a bounded execution time and documented timeout.
Calling C/C++ from Scripts	Scripts MUST call into native functionality via the component's published data model interface using rbuscli or RBUS APIs. Scripts MUST NOT link against internal .so symbols outside a published API. Scripts MUST NOT parse internal binary log/state formats.
RFC vs syscfg vs Build Flag	A features or behavior SHOULD use a data model property or method invocation for enable/disable if the feature or behavior needs remote, post-deployment toggling without a firmware push. An operator specific or device specific "Factory Default" value may be used at build time to control the default behavior of a feature. A feature or behavior that needs to be controlled at build time SHOULD use a Distro/build flag. Large features MAY be controlled at build time by the presence/absence of a software component in the build.
Threading & Concurrency Model	New daemons MUST default to a single-threaded event loop unless a specific workload justifies multi-threading. Where threads are used, lock ordering MUST be documented and mutexes MUST NOT be held across an RBUS call. There MUST NOT be unbounded thread spawning per request/connection.
Logging Level Standards	Components MUST use rdk_logger exclusively, never raw printf/syslog. ERROR is reserved for conditions requiring operator/telemetry attention; DEBUG/TRACE SHOULD be compiled out or runtime-gated in production builds.
Shared Library ABI Versioning	Any .so consumed by more than one component MUST follow symbol versioning

7.1.3 Industry Standards Conformance

Where RDK-B functionality overlaps with an established global standard, the implementation **MUST** conform to that standard's normative requirements rather than introducing a proprietary deviation. Deviations require documented justification and architecture review.

7.1.3.1. Remote management & data modeling

Standard / Body	Applies To	Note
Broadband Forum TR-069 (CWMP)	ACS Server, TR-069 adapter	Legacy remote management protocol
Broadband Forum TR-369 (USP)	USP Controller, TR-369 adapter	Transport-agnostic (WebSocket/MQTT/STOMP/CoAP)
Broadband Forum TR-181 Device:2 Data Model	All TR-181-backed parameters (WebPA, TR-069, USP and internal components that use TR-181 as the IPC contract) and OneWiFi interface to 3 rd party WiFi optimization software using DataElements, downloaded apps that provide or consume services.	Single underlying data model across northbound protocols, internal components, 3 rd party differentiating components and downloaded apps.
OpenAPI 3.x	Any REST API surface (RestAPI-tagged USP/WebPA paths)	Required for documenting REST interfaces

7.1.3.2. Wireless & local networking

Standard / Body	Applies To	Note
IEEE 802.11ax / 802.11be	OneWifi, rdk-wifi-hal, Vendor Wi-Fi HAL	Wi-Fi 6/6E/7 PHY-MAC conformance
Wi-Fi Alliance EasyMesh R6	EasyMesh controller / Agent interface and associated features.	Multi-AP interoperability certification
Wi-Fi Alliance WPA3 / WPS Certification	WPS box, Wi-Fi security stack	
IEEE 802.1Q	RDKVlanBridgingManager, VLAN utils, VLAN HAL	VLAN tagging for ISP/service separation
IEEE 802.3	Ethernet switch HAL	Base Ethernet conformance
Bluetooth SIG Core Specification	Bluetoothctl, Bluetooth HCI	

7.1.3.3. IoT & smart home standards (orchestrated via BartonCore — present in the platform, not yet depicted in the architecture diagram)

Standard / Body	Applies To	Note
Thread Group Thread Specification	BartonCore (Thread network layer)	
Connectivity Standards Alliance (CSA) Matter Specification	BartonCore (Matter controller/bridge)	
CSA Zigbee Specification	BartonCore (Zigbee bridge)	Legacy device compatibility
IEEE 802.15.4	Underlying radio layer for Thread/Zigbee within BartonCore	

Standard / Body	Applies To	Note
Device.IoT.* TR-181 (WT-480 pathway)	barton-rbus-provider	Bridges BartonCore into RBUS/TR-181

7.1.3.4. Network services (DNS/DHCP/PPP)

Standard / Body	Applies To	Note
IETF RFC 1035 (DNS)	DNS proxy (Linux OSS), CcspXdns	CcspXdns is RDK-B's DNS-based filtering/redirect feature, not a separate protocol
IETF RFC 6762/6763 (mDNS/DNS-SD)	Local service discovery, where used by OneWifi/EasyMesh	Confirm actual usage before citing as a hard requirement
IETF RFC 2131 (DHCPv4)	Dhcp Manager, DHCP V4 HAL, Dnsmasq	
IETF RFC 8415 (DHCPv6)	DHCP V6 HAL, Dibbler	
IETF RFC 1661/1662 (PPP) + RFC 2516 (PPPoE)	RDKPPPManger	Relevant for DSL/fiber ISPs using PPP-based WAN authentication
IETF RFC 3376 (IGMPv3)	IGMP Proxy	
UPnP Forum IGD Spec (now under Open Connectivity Foundation)	Upnp box	

7.1.3.5. IPv6 transition mechanisms

Standard / Body	Applies To	Note
IETF RFC 7597/7599 (MAP-T/MAP-E)	MAP-T/E features	May required kernel patches
IETF RFC 6333 (DS-Lite)	Dslite features	

7.1.3.6. Access network / WAN technologies

Standard / Body	Applies To	Note
ITU-T G.984 / G.988 (GPON, OMCI)	RdkGPONManager, GPON Headend (OLT)	
ITU-T G.9807.1 (XGS-PON)	Forward-looking GPON evolution	
IEEE 802.3ah / 802.3av (EPON / 10G-EPON)	CcspEponAgent, EPON Headend	EPON is IEEE, not ITU-T — distinct from GPON's standards body
ITU-T G.993.2 (VDSL2) / G.9701 (G.fast)	RdkXdsIManager, XDSL HAL, DSL Headend (DSLAM)	
CableLabs DOCSIS 3.1/4.0 + eRouter Specification	CcspCMAgent, Docsis Headend (CMTS)	eRouter spec is also the origin of the erouter0 interface naming used in your WAN bridge work
CableLabs PacketCable	CcspMTAAgent, MTA HAL	Voice over cable
3GPP TS 27.007 (AT command set)	RDKCellularManager-MM, Cellular Modem HAL, ModemManager/libqmi	Independent of RAN generation (3G/4G/5G)
USB-IF MBIM Specification	libmbim	Mobile Broadband Interface Model
MoCA Alliance MoCA 2.0/2.5	CcspMoca, MoCA HAL	

7.1.3.7. Boot chain, kernel & firmware integrity

Standard / Body	Applies To	Note
Linux Kernel (GPLv2, kernel.org/Linux Foundation governance)	Linux Kernel & Device Drivers layer	UAPI stability commitment governs what RDK-B can rely on across kernel versions
U-Boot (GPLv2, Denx community)	Bootloader layer	
Arm SystemReady (IR/SR profiles) + EBBR (Embedded Base Boot Requirements)	Bootloader layer, boot chain standardization	Standardizes the boot contract (U-Boot+EBBR for embedded profiles) for interoperability across Arm SoCs; complements PSA Certified, which covers security rather than boot interface
NIST SP 800-193 (Platform Firmware Resiliency Guidelines)	Bootloader → kernel verification chain	Detection/protection/recovery model for firmware integrity
Arm PSA Certified / TrustZone	Secure boot chain on ARM-based reference platforms (e.g. MT7988/BPI R4)	Security/attestation, distinct from SystemReady's boot-interface scope
GlobalPlatform TEE Client/Internal API Specifications	Where a TEE holds boot/signing keys	Only applies if the SoC's secure boot path uses a TEE rather than ROM-only verification

7.1.3.8. Cryptographic signing for secure boot

Standard / Body	Applies To	Note
NIST FIPS 186-5 (Digital Signature Standard — covers ECDSA)	Bootloader/kernel/firmware image signature verification	
SEC 1 / SEC 2 (Certicom Standards for Efficient Cryptography)	Curve parameter selection (e.g. NIST P-256/secp256r1)	Defines the actual EC curves referenced by FIPS 186-5
IETF RFC 8032 (EdDSA/Ed25519)	Alternative signing scheme	Increasingly used over ECDSA for boot-time verification due to faster, simpler constant-time implementation

7.1.3.9. Containers & application platform

Standard / Body	Applies To	Note
OCI Image Spec & Runtime Spec	crun, DAC, Dobby/BundleGen	Ties to your libbundlegen/Dobby OCI work

7.1.3.10. Security & coding standards

Standard / Body	Applies To	Note
MISRA-C:2012 / CERT C Secure Coding Standard	C/C++ coding anti-pattern enforcement (7.1.2)	External normative anchor for the internal coding rule
IETF RFC 8446 (TLS 1.3) / RFC 5280 (X.509 PKI)	WebPA/USP/ACS transport security, secure boot certs	
NIST SP 800-57 (key management)	PSM-backed credential storage	
OWASP IoT Top 10	General embedded device security review checklist	

7.2 Technical Governance Process

7.2.1. Purpose

The purpose of RDKB Broadband governance is to ensure that every new component, enhancement, bug fix, interface change, and platform integration follows a common technical standard before it is accepted into the RDK-B codebase.

The governance process must ensure:

- RDKB architecture consistency
- Platform portability
- Generic implementation suitable for community adoption
- Clear ownership and review responsibility
- Mandatory validation before merge
- Test evidence availability for every change
- Controlled release inclusion and traceability

No code change should be merged into a RDKB branch unless architecture, interface, dependency, test, and release expectations are satisfied.

7.2.2. Governance Scope

This process applies to:

- New component development
- Existing component enhancement
- Bug fixes
- HAL, Linux OSS, or platform-layer interaction
- Interface changes
- Dependency changes
- Build system changes
- Reference platform contributions
- Community contributions
- Release-bound changes

7.2.3. Classification of Changes

Every change must be classified before development starts.

7.2.3.1 New Component

A new component is any new software contribution that creates a new RDKB open source repo or collection of repos that provides new features, services or other software to be included in RDKB open source. New components SHOULD be organized so that operators can include this software only when they need a particular set of new features or services. New components MUST be organized as a collection of logically similar features or services that may be included or excluded from a finished software build as a group. New components MUST NOT be an unrelated collection of features or services.

Examples:

- New RDK-B open source software that provides a logical group of features or services that may be compiled and used as part of a monolithic software image
- New RDKB open source software that provides a logical group of features or services that may be compiled and used as a downloaded app.

- New RDKB open source software that provides a logical group of features or services that may contain multiple build recipes so that can optionally be compiled and built as part of a monolithic image or built as a downloaded app

7.2.3.2 Enhancement

An enhancement modifies or extends the features or services provided by an existing component without changing the basic ownership of the component.

Examples:

- New features or services inside existing component repositories
- New data model parameter(s) provided by the component used for configuration, telemetry, eventing or method invocation.
- Support for a modified HAL API
- [7.2.3.3 Bug Fix](#)

A bug fix corrects an existing defect without introducing new feature behavior.

Examples:

- Crash fix
- Memory leak fix
- Functional defect fix
- Race condition fix
- Regression fix
- Build failure fix
- Test failure fix
- Platform compatibility fix

Bug fixes must be generic and community-aligned. Local hacks, platform-specific shortcuts, or hidden behavior under flags should not be accepted unless formally justified.

7.2.4. Governance Entry Criteria

Before development starts, the contributor must create a design/review ticket with the following information.

7.2.4.1 Mandatory Inputs for New Component

A new component proposal must include:

- Component name
- Problem statement
- Functional scope including a list of the features/services provided
- Non-functional scope
- Ownership
- Details on usage in all RDKB profiles. The proposal should indicate if this new component is required, optional or not applicable for each RDKB profile.
- A list of all planned data model names to be provided by this component. This must include as much detail as possible regarding the feature and purpose of each data model and detailed syntax and usage information.
- A list of all planned data model names to be consumed by this component (data model names that are provided by other components). Include how this component's behavior is impacted if any of the components that provide these data models are not included in the software build.
- A refactoring statement that lists any software or features that are currently or were previously in other components (if any).
- Runtime process model
- Build-time dependency list of other RDKB components
- Build-time dependency list of other open source software Runtime dependency list

- List of schedule dependencies with other proposed changes (if any) Example if this new component is created due to refactoring of an existing component, this should reference the changes required in the other component(s) that have to be done at the same time. Telemetry impact
- Logging impact
- Security impact
- Memory and CPU expectations
- Boot-time impact
- Restart/recovery behavior
- HAL APIs used (including open source implementations like NL80211)
- Platform portability plan
- Test plan
- Release target
- Rollback plan

7.2.4.2 Mandatory Inputs for Enhancement

An enhancement proposal must include:

- Existing component impacted
- Feature description
- Interface impact
- Changes to data models provided by this component
- Changes to data models consumed by this component and how the behavior of this component is impacted if any of the components that provide these data models are not included in the software build.
- Dependency impact with other RDKB components
- Dependency impact with non-RDK open-source dependencies
- A list of HAL APIs that are updated or any new HAL APIs used by this component
- Backward compatibility assessment
- Platform impact
- Test plan
- Release target
- Regression risk

7.2.4.3 Mandatory Inputs for Bug Fix

A bug fix proposal must include:

- Defect description
- Reproduction steps
- Logs or evidence
- Root cause analysis
- Fix approach
- Regression impact
- Platforms impacted
- Test evidence
- Whether the fix is generic or platform-specific
- Whether the issue exists in stable branch, rdk-next, or both

7.2.5. Architecture Review Rules

All new component contributions and major enhancements require architecture review before implementation or merge.

Architecture review must validate:

- Whether the component is required, optional or not applicable to each RDKB profile
- Whether the component or change duplicates existing functionality
- Whether the new component name is a good summary or label for its functionality. Don't use names without meaning that make it hard to understand.
- Whether the component or change is documented sufficiently for review
- Whether the component or changes is hardware independent and properly abstracted from any hardware or vendor dependencies.
- Whether the component or change meets the IPC guidelines
- Whether the group of new features or services make sense to be in the proposed new or existing component from an operator perspective. (Is this a logical grouping of features or services that may be included/excluded from a software build)
- Whether the component follows RDKB design principles
- Whether the build time dependency model is acceptable
- Whether the run-time dependency model is acceptable
- Whether platform-specific logic is isolated correctly
- Whether HAL/Linux OSS access is done through approved layers
- Whether interfaces are stable and backward compatible
- Whether the component can run across reference platforms
- Whether the component is testable in CI and on hardware
- Whether this software is covered by an acceptable software license

approval is mandatory for:

- New RDBK component
- Breaking interface change
- New mandatory dependency
- New HAL contract
- Data model provider change
- Release-blocking feature
- Change impacting multiple components
- Change impacting community platform portability

7.2.6. Platform Layer Invocation Rules

Components must not directly embed board-specific logic inside RDKB implementations to be aligned with HAL spec

7.2.6.1 Preferred Access Order

An RDKB component should access platform functionality in the following order:

1. Standard Linux OSS interface
2. Existing RDK-B abstraction or service
3. Approved HAL interface
4. Vendor/platform plugin
5. Platform-specific fallback only when formally approved

7.2.6.2 Linux OSS Access

Use Linux OSS interfaces when the functionality is standard and portable.

Examples:

- netlink
- sysfs

- procfs
- iproute2-compatible behavior
- standard kernel networking APIs
- standard systemd interaction where applicable

Linux OSS usage must be validated across supported kernel versions and reference platforms.

7.2.6.3 HAL Access

Use HAL when:

- A common vendor independent interface to vendor specific drivers or low-level software is required.
- Hardware-specific behavior is required
- Vendor implementation may differ
- Acceleration path is SoC-specific
- Radio, switch, PHY, WAN, LAN, or hardware diagnostics behavior is not portable through standard Linux APIs

HAL usage must follow these rules:

- HAL API must be generic
- HAL must not expose vendor-private implementation details
- HAL header must be reviewed before implementation
- HAL behavior must be documented
- HAL failure handling must be defined
- Stub/reference implementation must be available where possible
- Component must degrade gracefully if capability is not supported

7.2.6.4 Direct Vendor Calls

Direct vendor SDK/API calls make RDKB open source software vendor dependent and violate the spirit and intent of RDKB. Direct vendor SDK/API calls inside RDKB component code are not allowed unless formally approved.

7.2.7. Dependency Governance

Every component must declare and justify:

- Build-time dependencies
- Runtime dependencies
- Optional dependencies
- Platform-specific dependencies
- Service startup dependencies
- Library dependencies
- Package dependencies

New dependencies must be reviewed for:

- Portability
- License compatibility
- Security risk
- Maintenance ownership
- Image size impact
- Boot-time impact
- Runtime memory impact
- Impact on minimal profile builds

No new mandatory dependency should be added to RDKB components unless it is required for generic broadband functionality.

7.2.8. Development Flow

7.2.8.1 New Component Flow

The required flow is:

1. Create architecture proposal ticket
2. Complete architecture review
3. Identify component owner and reviewers
4. Define interface and dependency contract
5. Define HAL/Linux OSS interaction model
6. Define all data model provided
7. List all data models consumed
8. Define test plan
9. Develop implementation
10. Run local build and unit tests
11. Run platform validation
12. Submit PR
13. Run CI checks
14. Complete peer review
15. Complete RTAB review if required
16. Attach test evidence
17. Merge only after sanity pass (including a comparison with run-time data model auto-discovery)
18. Track release inclusion

7.2.8.2 Enhancement Flow

The required flow is:

1. Create enhancement ticket
2. Document design and impact
3. Review interface and dependency changes
4. Confirm backward compatibility
5. Develop change
6. Run unit, functional, and regression tests
7. Submit PR
8. Attach test evidence
9. Complete peer/component owner review
10. Complete RTAB approval if component behavior changes
11. Merge after sanity pass (including a comparison with run-time data model auto-discovery)
12. Include in release notes if user-visible or interface-impacting

7.2.8.3 Bug Fix Flow

The required flow is:

1. Create defect ticket
2. Attach reproduction evidence
3. Complete root cause analysis
4. Add details of how this was fixed to the defect ticket
5. Confirm affected platforms and branches
6. Implement generic fix
7. Validate original failure scenario
8. Run regression tests

9. Submit PR
10. Attach before/after evidence
11. Merge only after sanity pass (including a comparison with run-time data model auto-discovery)
12. Decide whether fix is needed in stable branch, rdk-next, or both

7.2.9. Testing Governance

Testing is mandatory for every new component, enhancement, and bug fix.

No code change should be merged if required sanity testing fails.

7.2.9.1 Test Types Required

Each change must be evaluated against the following test categories.

Build Validation

- Clean build
- Incremental build
- Component build
- Image build
- Package dependency validation
- Yocto recipe validation
- Static build dependency check

Unit Testing

- Function-level test
- Error path test
- Boundary condition test
- Data Model run-time auto-discovery
- Sysevent scan to verify no new sysevent usage
- Invalid input test
- API contract test
- Mock HAL test if HAL is involved

Functional Testing

- Feature enable/disable
- Basic workflow
- Runtime behavior
- Service start/stop/restart
- Configuration persistence
- Data model get/set validation
- RBUSvalidation where applicable
- CLI/API validation where applicable

Integration Testing

- Interaction with dependent services
- Boot order validation
- Systemd dependency validation
- Sysevent/RBUS/event validation
- Telemetry/logging validation
- HAL/Linux OSS integration validation
- Upgrade/reboot persistence

Regression Testing

- Existing feature sanity
- Component regression
- Impacted component regression
- Data model regression
- Boot sanity
- Connectivity sanity
- Memory/process sanity

Negative Testing

- Dependency missing
- HAL failure
- Invalid configuration
- Permission failure
- Interface unavailable
- Process crash/restart
- Unsupported platform capability

Performance Testing

- CPU usage
- Memory usage
- Boot-time impact
- Runtime latency
- Event storm behavior
- Log volume
- Telemetry volume

Security Testing

- Input validation
- Command injection risk
- File permission validation
- Secret handling
- Privilege usage
- Network exposure
- API authentication/authorization where applicable

Stability Testing

- Reboot validation
- Process restart validation
- Long-run validation
- Memory leak check
- Crash check
- Log flooding check

7.2.10. Testing Expectations by Change Type

7.2.10.1 New Component

Minimum required evidence:

- Successful image build
- Successful component start/stop/restart
- Functional test results

- Unit test results
- Integration test results
- Reboot test results
- Memory and CPU observation
- Negative test results
- Logs showing normal behavior
-
- Data Model run-time auto-discovery to document usage
- Sysevent scan to verify no new sysevent usage
- Platform validation report
- CI result link
- Known limitations

7.2.10.2 Enhancement

Minimum required evidence:

- Build pass
- Existing functionality regression pass
- New functionality test pass
- Interface backward compatibility result
- Impacted component sanity result
- Logs before and after change
- Data Model run-time auto-discovery to document usage
- Sysevent scan to verify no new sysevent usage
- Platform validation result
- CI result link

7.2.10.3 Bug Fix

Minimum required evidence:

- Original issue reproduction evidence
- Root cause analysis
- Fix validation evidence
- Before/after logs
- Regression test result
- Data Model run-time auto-discovery to document usage
- Sysevent scan to verify no new sysevent usage
- Impacted platform validation
- Confirmation that fix is generic
- CI result link

7.2.11. Platform Test Matrix

Each change must define which platforms were tested and why.

7.2.11.1 Minimum Platform Validation

At minimum, RDK-B component changes should be validated on:

- Primary reference platform
- One active hardware reference platform
- Emulated or CI platform where available

Example platform categories:

- qemux86broadband or equivalent CI reference
- Raspberry Pi or generic reference platform
- Banana Pi R4 / Filologic / active broadband reference hardware
- Vendor platform if the change is hardware-specific

7.2.11.2 Platform Selection Rules

Test on hardware platform when the change impacts:

- WAN
- LAN
- Wi-Fi
- Switch
- PHY
- HAL
- Kernel interface
- Boot sequence
- Telemetry
- TR-181 runtime behavior
- USP/TR-369 runtime behavior
- WebPA runtime behavior
- TR-069 runtime behavior
- System service behavior

Test on CI/emulated platform when the change impacts:

- Generic component logic
- Build system
- Data model provider
- Library behavior
- Static dependency
- Configuration parsing
- Non-hardware-specific functionality

7.2.11.3 Hardware-Specific Changes

For hardware-specific changes:

- Validate on the target hardware
- Validate that generic platforms are not broken
- Isolate platform logic outside the RDKB components when possible
- Provide fallback behavior for unsupported capability

7.2.12. Branch and Build Governance

Every change must state which branch/build was validated.

7.2.12.1 rdk-next/develop

Use rdk-next/develop for:

- New component development
- New feature development
- Major enhancement
- Interface change
- New dependency
- HAL contract update

- Experimental RDKB Component work
- Community-first development

7.2.12.2 Stable Working Branches (one branch for each supported LTS release)

Use stable branches for:

- Critical bug fixes
- Release-bound fixes
- Regression fixes
- Security fixes
- Operator-required fixes
- Backport candidates

Replace the change on other supported LTS branches where applicable

7.2.12.3 Testing Rule

For new RDKB development:

- Validate first on rdk-next
- Validate release need on target stable branches if required
- Backport only after fix/feature is stable and approved

For bug fixes:

- Validate on the branch where issue was found
- Validate on rdk-next if the code exists there
- Validate backport branches if release requires it

7.2.13. Review and Approval Model

7.2.13.1 Where to Raise Reviews

All reviews must be tracked using:

- GitHub issue for feature/defect tracking
- GitHub PR for code review
- Project board for release tracking
- Architecture review record for new RDKB component or major change
- Review record for applicable changes or breaking changes

7.2.13.2 Required Reviewers

Reviewers should include:

- Component owner
- Peer reviewer
- Platform/reference platform owner
- Architecture owner for new component or major enhancement
- HAL owner if HAL is impacted
- Test owner or QA representative
- Release engineer for release-bound changes
- RTAB for major RDKB component or breaking interface changes

7.2.13.3 Review Checklist

Reviewers must confirm:

- Design is generic
- Code follows RDKB standards
- Dependency model is acceptable
- No unnecessary platform-specific logic exists

- HAL/Linux OSS usage is correct
- Backward compatibility is preserved
- Error handling is complete
- Logs are useful but not excessive
- Data Models used for telemetry are meaningful
- Security risks are addressed
- Tests are sufficient
- CI passed
- Test evidence is attached
- Release impact is documented

7.2.14. Merge Criteria

A change can be merged only when all applicable conditions are met.

7.2.14.1 Mandatory Merge Criteria

- PR is linked to issue/ticket
- Architecture review completed if required
- RTAB approval completed if required
- CI checks passed
- Build validation passed
- Required sanity tests passed including data model run-time auto-discovery
- Required platform tests passed
- Test evidence attached
- No unresolved critical review comments
- No unexplained functional test failures
- Release impact documented
- Rollback plan documented for high-risk changes

7.2.14.2 Blocking Conditions

Code must not be merged if:

- Sanity test failed
- CI failed without approved exception
- Functional test failure has no root cause
- Test evidence is missing
- Dependency impact is not reviewed
- HAL/interface change is not approved
- Change is hidden under a flag without justification
- Fix is platform-specific but placed in RDKB Component generic code
- Release engineer cannot confirm primary expectation is met

7.2.15. Test Evidence Requirements

All test results must be attached to the ticket or PR.

Evidence must include:

- Test summary
- Test case list
- Pass/fail status for each test case

- Logs for important test cases
- Failure logs where applicable
- Component level data model auto-discovery list
- Platform details
- Build details
- Branch details
- Image version
- Commit ID
- Date of validation
- Tester name or automation job link
- Known issues
- Deviations or exceptions

For every failed functional test:

- A development ticket must be created
- Root cause analysis must be provided
- Fix must be generic enough for community alignment
- Release impact must be assessed

No release-bound change should be accepted with unexplained failures.

7.2.16. Release Governance

7.2.16.1 Release Inclusion Criteria

A change can be included in release only when:

- Feature/bug ticket is closed or release-approved
- PR is merged into correct branch
- CI passed
- Required platform validation passed
- Test evidence is available
- Release engineer confirms primary expectation is met
- Known issues are documented
- Release notes are updated if needed

7.2.16.2 Release Notes

Release notes must include:

- New component details
- Enhancement summary
- Bug fix summary
- Interface changes
- Data model changes from run-time auto-discovery
- HAL changes
- Dependency changes
- Migration steps if any
- Known limitations
- Test platforms
- Validation summary

7.2.16.3 Release Timing

Changes should be released based on classification:

New Component

Release in planned major/minor release after full validation.

Enhancement

Release in planned feature release unless it is required for a committed operator/customer milestone.

Bug Fix

Release in next maintenance release or patch release depending on severity.

Critical/Security Fix

Release through expedited patch process after mandatory validation and approval.

7.2.17. Flag Governance

No code change should be accepted under a flag unless truly required.

Flags are allowed only when at least one of the following is true:

- Feature is optional by design
- Feature is not required for all operators or markets
- Feature consumes large amounts of memory and may not be suitable for all devices
- Feature consumes large amounts of CPU and may not be suitable for all devices
- Feature is not applicable for some RDKB profiles when other features in the same component are applicable
- Platform capability is not universally available
- Migration requires staged rollout
- Release risk needs controlled enablement
- Backward compatibility requires coexistence

Flags are not allowed for:

- Hiding incomplete code
- Avoiding proper architecture review
- Avoiding test failures
- Carrying platform-specific hacks
- Bypassing community alignment

Every flag must have:

- Clear owner
 - Default behavior
 - Documentation
 - Test coverage for enabled and disabled modes
 - Removal plan if temporary
-

7.2.18. Reference Platform and Community Contribution Rules

All community contributions and Reference Platform team changes must be validated before merge.

Required expectations:

- Changes must have required documentation
- Code must build on reference platform
- RDKB Component sanity must pass
- Functional validation must be attached
- No reference-only assumption should enter RDKB Components
- Platform-specific changes must remain in platform layer
- Generic code must remain portable
- Test evidence must be available in ticket/PR

7.2.19. Governance Checklist

7.2.19.1 New Component Checklist

- New component ticket created
- Architecture proposal created
- Component ownership defined
- RDKB Component classification confirmed
- Dependency list reviewed
- Interface contract reviewed
- HAL/Linux OSS usage reviewed
- Data model impact documented
- Security impact reviewed
- Telemetry/logging plan reviewed
- Test plan approved
- CI passed
- Reference platform tested
- Hardware platform tested if applicable
- Reboot/restart tested
- Negative testing completed
- Test evidence attached
- RTAB approval completed if required
- Release plan documented

7.2.19.2 Enhancement Checklist

- Enhancement ticket created
- Impacted component identified
- Backward compatibility checked
- Interface/data model impact documented
- Dependency impact reviewed
- Unit/functional tests passed
- Regression tests passed
- Platform validation completed
- Test evidence attached
- Release notes updated if required

7.2.19.3 Bug Fix Checklist

- Defect ticket created
 - Reproduction steps attached
 - Root cause analysis completed
 - Fix approach reviewed
 - Before/after evidence attached
 - Regression testing completed
 - Branch impact checked
 - Backport need assessed
 - Test evidence attached
 - Release inclusion decision made
-

7.2.20. Final Governance Rule

RDK Broadband changes must be generic, portable, testable, and community aligned.

No code change should be merged or released unless:

- The primary expectation is met
- Sanity testing is passed
- Test evidence is attached
- Failures are understood and tracked
- Review ownership is clear
- Release impact is documented
- The implementation aligns with RDK-B architecture principles

7.3 Component Governance Process

7.3.1 Purpose

The purpose of Component Governance is to ensure that every RDK-B component has a defined owner, a known lifecycle state, a health baseline, and a clear exit path. While Section 7.2 governs how changes are introduced and validated, Section 7.3 governs what happens to a component across its entire life — from registration through active maintenance to deprecation and removal.

Component governance must ensure:

- Every RDK-B component has a registered owner at all times
- Component lifecycle state is known and current
- Orphaned or unhealthy components are identified and resolved
- Deprecated components have a defined migration path
- HAL contracts and TR-181 parameters are transferred cleanly when components change state
- Community contributors and operators have visibility into component health and roadmap

7.3.2 Component Registry

Every component accepted into RDK-B must be registered in the Component Registry at merge time. The registry is the authoritative record of all active, deprecated, and removed components.

7.3.2.1 Mandatory Registry Fields

Each component entry must include:

- Component name
- Primary GitHub repository
- Supporting repositories (if any)
- Classification for each RDKB profile (CORE / Required / Optional / Not Applicable)
- Current lifecycle state (Active / Stable / Deprecated / Removed)
- Primary owner (name and organization)
- Secondary owner (name and organization)
- Primary interfaces (RBUS / HAL)
- HAL contract reference (if applicable)
- Provided TR-181 data model provided by the component with list of names, syntax and usage details for each name
- List of dependent RDKB components
- List of dependent non-RDKB open source components
- Last validated release
- Known open critical issues
- Distro flag(s) controlling inclusion

7.3.2.2 Registry Maintenance Rules

- Registry must be updated at every release

- Any change to owner, state, or interfaces must trigger a registry update
- RTAB is responsible for registry accuracy
- Registry must be publicly accessible to the RDK community

7.3.3 Component Lifecycle States

Every RDK-B component must be in exactly one lifecycle state at all times.

State	Meaning
Active	Component is under active development and maintenance. New features, enhancements, and bug fixes are accepted. Full governance process applies.
Stable	Component interface is frozen. Only critical bug fixes and security fixes are accepted. No new features. HAL contract is locked.
Deprecated	Component is scheduled for removal. No new features or enhancements accepted. Critical security fixes only. Migration path is defined and published.
Removed	Component is no longer part of RDK-B. Repository archived. No changes accepted.

7.3.3.1 State Transition Rules

- **Active** → **Stable**: Requires architecture owner approval. Interface freeze declared. HAL contract version locked.
- **Stable** → **Deprecated**: Requires RTAB approval. Migration path must be defined before transition.
- **Deprecated** → **Removed**: Requires RTAB approval. Migration path must be confirmed complete. Operator notification period must have elapsed.
- **Any State** → **Active**: Requires RTAB approval if previously Stable or Deprecated.

7.3.4 Component Ownership Model

Every RDK-B component must have a defined primary owner and a secondary owner at all times.

7.3.4.1 Owner Responsibilities

The primary owner is responsible for:

- Architecture decisions for the component
- Review and approval of all PRs touching the component
- Maintaining the component's test suite
- Representing the component in RTAB and architecture reviews
- Keeping the registry entry current
- Participating in periodic health reviews
- Defining the migration plan if deprecation is required

7.3.4.2 Ownership Transfer Process

When a primary owner is no longer available:

- Secondary owner automatically assumes primary responsibility
- A new primary/secondary owner must be nominated within one release cycle
- If no secondary owner exists, the component is escalated to RTAB for ownership resolution
- RTAB may assign ownership to another organization, open it to community volunteers, or initiate deprecation if no owner is found

7.3.4.3 Orphan Resolution

A component is considered orphaned when:

- Primary and secondary owner are both unavailable
- No PR reviews have occurred in two consecutive release cycles
- Health review has not been completed for one year

Orphaned components are escalated to RTAB. RTAB must resolve ownership or initiate deprecation within one release cycle.

7.3.5 Periodic Health Review

Every RDK-B component classified as CORE, Required or Optional and is included in reference builds for any RDKB profile that is being released must undergo a health review once per release cycle.

7.3.5.1 Health Review Criteria

The health review must confirm:

- Ownership is current and active
- Component builds cleanly on the current reference platform
- CI is passing
- Test suite is current and passing
- No unresolved critical or security defects older than one release cycle
- HAL contract is current and matches active vendor implementations
- TR-181 data model scope is accurate and agrees with run-time auto-discovery
- Registry entry is up to date
- No known community-reported regressions are unaddressed

7.3.5.2 Health Review Outcome

Outcome	Action
Healthy	No action required. Registry updated with review date.
At Risk	Owner must submit a remediation plan within 30 days.
Unhealthy	Escalated to RTAB. RTAB decides: remediation, ownership transfer, or deprecation.

7.3.5.3 Health Review Responsibility

- Component owner conducts the review
- Architecture owner validates the result
- RTAB reviews all At Risk and Unhealthy outcomes

7.3.6 Deprecation and Migration Policy

7.3.6.1 Deprecation Declaration

A component may be deprecated when:

- Functionality is superseded by a newer component
- Component cannot meet architecture standards and remediation is not feasible
- No active owner can be identified
- RTAB determines the component no longer aligns with RDK-B goals

Deprecation requires:

- RTAB approval
- Migration path defined before deprecation is declared
- Operator notification published in release notes
- Minimum one full release cycle between Deprecated state and Removed state

7.3.6.2 Migration Path Requirements

A migration plan must define:

- Replacement component or approach
- Step-by-step migration guide for operators and integrators
- Timeline for migration
- Support period during coexistence (deprecated component + replacement)
- Test coverage for the migration path

7.3.6.3 HAL Contract Handoff

When a deprecated component owns a HAL contract:

- HAL contract must be transferred to the replacement component (if any) before removal
- HAL contract version must be incremented
- Vendor re-validation must be required against the new contract owner
- Transition period must be defined allowing vendors to update implementations

7.3.6.4 TR-181 Parameter Transfer

When a deprecated component owns TR-181 parameters:

- Parameters must be transferred to the replacement component (if any)
- Data model backward compatibility must be maintained during the transition period
- Breaking data model changes require RTAB approval and operator notification

7.3.6.5 Removal Criteria

A component can move to Removed state only when:

- RTAB approval is obtained
- Migration path is confirmed complete by at least one reference platform
- Operator notification period has elapsed
- HAL contract handoff is complete
- TR-181 parameter transfer is complete
- Repository is archived and marked as removed in registry

7.3.7 Interface Stability Tagging

Every HAL interface and RBUS API must carry an explicit stability tag.

7.3.7.1 Stability Levels

Tag	Meaning
Experimental	Interface is under active design. Breaking changes may occur without notice. Not for production use.
Stable	Interface is production ready. Breaking changes require RTAB approval and versioning.
Frozen	Interface will not change. Only bug fixes to documentation or behavior clarifications permitted.
Deprecated	Interface is scheduled for removal. Migration path is defined. No new consumers should adopt it.

7.3.7.2 Tagging Requirements

- All HAL headers must carry a stability tag in the file header comment
- All RBUS APIs must carry a stability tag in the API documentation
- Tag changes must be reviewed by the architecture owner
- Downgrade from Stable to Experimental is not permitted without RTAB approval

7.3.7.3 Freeze and Unfreeze Process

- Interface freeze requires architecture owner declaration and registry update
- Unfreeze (Frozen → Stable) requires RTAB approval and a versioning increment
- Frozen interfaces that need behavioral change must be versioned as a new interface, not modified in place

7.3.8 Component Governance Checklist

For every release cycle, the component owner must confirm:

- Health review completed
- Registry entry current
- Ownership confirmed
- CI passing
- Test suite current
- No unresolved critical defects
- HAL contract current
- TR-181 provider list and details are accurate
- Interface stability tags current

- Deprecation or migration actions on track (if applicable)

7.4 Escalation Process

Step	Escalation Stage	When to Use	Escalation Owner	Required Participants	Required Evidence / Inputs	Expected Decision / Output	Exit Criteria
1	Identify Blocker	Change is blocked due to architecture, review, test, platform, merge, or release issue	Contributor / PR Owner	Contributor	Problem summary, impacted component, PR/ticket link	Confirm blocker type	Blocker is clearly identified
2	Record Issue in Ticket / PR	Any blocker needs formal tracking	Contributor / PR Owner	Contributor, Component Owner if needed	Logs, test results, branch, build, platform, failure details, expected decision	Ticket/PR updated with escalation details	Ticket/PR has enough information for review
3	Level 1: Component Owner Review	Implementation issue, component design question, unclear fix approach	Component Owner	Contributor, Peer Reviewer	Code change, design note, unit/functional test result	Approve, request rework, request more tests, or escalate	Component-level decision recorded
4	Level 2: Platform / Test Review	Platform behavior issue, HAL/Linux OSS concern, test failure, missing validation	Platform Owner or Test Owner	Component Owner, Platform Owner, Test Owner, Contributor	Platform logs, test report, HAL/Linux OSS behavior, failed test details	Resolve, request platform validation, create defect, or escalate	Platform/test decision recorded
5	Level 3: Component Architecture Review	Core design disagreement, Component vs platform boundary issue, dependency/interface dispute	Component Architecture Owner	Component Owner, Platform Owner, HAL Owner if applicable, Test Owner, Contributor	Architecture proposal, dependency impact, interface contract, portability assessment	Approve design, reject design, request modification, or escalate to RTAB	Architecture decision recorded
6	Level 4: RTAB Review	Component-classification change, breaking interface change, major dependency, unresolved architecture conflict	RTAB Chair / RTAB Delegate	Component Architecture Owner, Component Owner, Release Engineer if impacted, Platform Owner, Contributor	Architecture decision, risk summary, test evidence, interface/dependency impact	Approve, reject, approve with conditions, defer, or escalate	RTAB decision recorded

Step	Escalation Stage	When to Use	Escalation Owner	Required Participants	Required Evidence / Inputs	Expected Decision / Output	Exit Criteria
7	Level 5: Release Governance	Release inclusion, release blocker, backport, waiver, release cutoff issue	Release Engineer / Release Manager	RTAB Delegate, Component Owner, Test Owner, Platform Owner, Contributor	Release ticket, validation summary, known issues, test evidence, waiver request if any	Include, defer, backport, reject, approve waiver, or escalate	Release decision recorded
8	Document Final Decision	Any escalation reaches a decision	Current Escalation Owner	All required stakeholders	Decision summary, owner, risk, evidence reviewed, missing validation, follow-up actions	Final decision documented in PR/ticket	Decision is traceable
9	Execute Decision	Code/test/release action is required after decision	Contributor + Component Owner + Test Owner	Contributor, Component Owner, Test Owner, Release Engineer if needed	Updated code, new test result, updated release notes, follow-up tickets	Complete required action	Required actions completed
10	Close Escalation	Escalation decision and actions are complete	Current Escalation Owner	Contributor, Component Owner, Test Owner, Release Engineer if needed	Closure summary, final evidence, release impact, follow-up tickets	Close escalation	Escalation closed with evidence
11	Organization Leadership Escalation	Issue remains unresolved after Step 10, or has business, ownership, resource, operator/customer, or cross-organization impact	Organization Leadership Owner	RTAB Chair/Delegate, Release Manager, Component Architecture Owner, Component Owner, Platform Owner, Test Owner, Program Manager, Contributor if needed	Full escalation history, unresolved risks, business/release impact, ownership gap, resource need	Assign final owner, priority, release decision, waiver decision, target closure date	Leadership decision recorded and issue closed or converted to leadership-tracked action

Expected Outcome

RDK-B code becomes cleaner and easier to reuse. Improves time to market. Increases quality of RDKB Components and vendor code. New developers are onboarded faster with consistent, documented patterns.

8. RDK-B Ecosystem and Stakeholders

RDK-B's strength comes not from any single vendor but from a multi-party ecosystem of operators, silicon vendors, hardware manufacturers, system integrators, and application developers. This section describes each stakeholder category, their relationship to the platform, their specific needs and value drivers, and representative examples.

Ecosystem Overview

The RDK-B ecosystem operates as an inverted pyramid: ISPs at the top set requirements and fund the platform; SoC vendors provide the hardware foundation; OEMs build the physical devices; SIs integrate and customize; third-party developers extend with applications. Each layer depends on the layers below and enables the layers above.

8.1 Internet Service Providers (ISPs / Operators)

Primary Role

ISPs are the primary consumers and primary funders of the RDK-B platform. They deploy RDK-B devices to millions of subscribers and use the platform's management, telemetry, and configuration capabilities to operate their broadband networks.

Value Proposition for ISPs

ISPs adopt RDK-B because it eliminates the need to build and maintain a proprietary CPE software stack. A single RDK-B codebase can be deployed across multiple hardware vendors and SoC platforms, reducing fragmentation and allowing operators to consolidate their engineering investment. The platform provides:

- Vendor independence: the same management APIs and telemetry schemas regardless of underlying hardware supplier
- Rapid feature deployment: RFC-based remote feature control enables fleet-wide feature toggles without firmware upgrades
- Standardized data collection: T2 telemetry with configurable profiles provides consistent KPIs across all device types
- Reduced time-to-market: pre-integrated components accelerate device certification and deployment
- Lower total cost of ownership: shared open-source development amortizes engineering cost across the community

ISP-Specific Feature Requirements

ISP Requirement	RDK-B Capability
Remote Management	USP/TR-369, TR-069 and WebPA for centralized configuration, monitoring, and status, xConf for RFC-based runtime feature control, WebConfig for configuration push without full firmware upgrades.
Telemetry & KPIs	T2 profile-driven telemetry for Wi-Fi performance metrics, WAN health statistics, error rates, and device-level KPIs published to Backoffice analytics platforms.
Firmware Lifecycle	OTA firmware orchestration via xConf and platform-manager enabling phased rollouts, A/B firmware updates, and emergency rollback.
Subscriber Self-Service	Local WebUI for residential subscribers to configure Wi-Fi, set parental controls, view connected devices, and run diagnostics without calling customer support.

ISP Requirement	RDK-B Capability
Network Quality	DFS, band steering, 802.11kv neighbor reporting, and EasyMesh R6 for whole-home Wi-Fi quality management.
Security Compliance	WPA3/WPA3 Enterprise, stateful firewall, secure management channels (TLS), and signed firmware to meet regulatory and compliance obligations.
Public Wi-Fi	Passpoint/Hotspot 2.0 and public SSID support for operator-managed public Wi-Fi hotspot overlays at subscriber premises.
Parental Controls	DNS-based filtering, device pause, and managed site lists as value-added subscriber services.

8.2 System-on-Chip (SoC) Vendors

Primary Role

SoC vendors provide the silicon foundation on which RDK-B runs. Their primary responsibility is implementing the HAL layer — the set of APIs that RDK-B middleware calls to control Wi-Fi radios, DOCSIS modems, DSL chipsets, Ethernet switches, and other hardware blocks.

Value Proposition for SoC Vendors

SoC vendors who support RDK-B gain access to the entire operator community that has standardized on the platform. A certified RDK-B HAL implementation means their SoC can be qualified for operator CPE program without the operator needing a custom software integration effort for that silicon. This dramatically reduces time-to-design-win and lowers the barrier for OEMs to adopt the vendor's chipset.

SoC Vendor Responsibilities in RDK-B

- Implement the HAL interfaces supported by their chipset
- Pass the HAL test suites published alongside each HAL interface definition (rdkb-halif-test-*)
- Provide a Linux BSP (kernel, drivers, firmware blobs) compatible with the Yocto/OE build system used by RDK-B
- Contribute NL80211-compatible or extension-based Wi-Fi driver interfaces for OneWifi integration (if applicable)
- Maintain HAL compatibility across RDK-B releases and platform versions
- Participate in RTAB feature discussions to ensure new features are achievable on their silicon roadmap

HAL Integration Model

SoC vendors ship their HAL implementations as pre-compiled binary libraries or as source code under NDA. The rdkb-halif-* header packages define the API contract. The rdkb-halif-test-* packages provide automated test scripts that SoC vendors must pass before claiming RDK-B compatibility. This test-first HAL model has dramatically improved cross-vendor portability and reduced integration defects.

8.3 Original Equipment Manufacturers (OEMs / ODMs)

Primary Role

OEMs and ODMs design and manufacture the physical CPE hardware that operators deploy to subscribers. They assemble the SoC, memory, Wi-Fi radios, WAN interfaces, and enclosure into a finished product, then integrate RDK-B (sometimes with SI support) to produce a certifiable firmware image.

OEM Responsibilities in RDK-B

- Select the SoC and confirm HAL availability for the target RDK-B features
- Integrate the SoC vendor's BSP and HAL libraries into the Yocto/BitBake build environment
- Configure RDK-B build recipes (meta-rdk, meta-rdk-broadband layers) for the specific board
- Develop board-specific platform init, LED management, button handling, and thermal management
- Conduct EMC/RF certification and operator device qualification testing
- Maintain firmware branches for the operational lifetime of the deployed device (often 5–10 years)
- Provide factory imaging tools and production test fixtures

OEM Build System

RDK-B is built using Yocto Project / Open Embedded (OE). OEMs maintain a machine configuration layer (meta-<platform>) that defines the target board, SoC BSP path, and hardware-specific overrides. This sits alongside the RDK Central community layers (meta-rdk, meta-rdk-broadband, meta-rdk-bsp-*) that provide the RDK-B component recipes.

8.4 System Integrators (SIs)

Primary Role

System Integrators bridge the gap between the generic RDK-B platform and operator-specific deployment requirements. They take the community baseline, apply operator customizations, integrate proprietary cloud backends, and deliver a production-ready firmware binary that meets the operator's certification requirements.

What SIs Do in the RDK-B World

- Operator customization: branding, feature flags, default configuration, SKU-specific builds
- Cloud backend integration: connecting T2 telemetry, WebPA/USP, Xconf, and crash upload to operator-specific cloud infrastructure
- Security hardening: applying operator security policies, disabling unused services, adding secure boot keys
- Certification support: conducting operator device qualification (ODQ) test campaigns and resolving defects
- Legacy protocol bridging: adapting RDK-B management interfaces to existing operator NMS/OSS systems
- Field upgrade tooling: building OTA distribution infrastructure and fleet management tooling
- Performance optimization: tuning Wi-Fi parameters, memory footprint, boot time, and throughput for specific hardware
- Long-term maintenance: applying security patches and CVE fixes for the operational fleet lifetime

8.5 Third-Party Application Developers

Primary Role

The RDK-B container runtime (Dobby + DSM) enables third-party software companies to deploy their applications directly onto gateway devices as downloadable OCI containers. This transforms the broadband gateway from a fixed-function appliance into an open application platform.

The App Container Ecosystem

With DSM (Downloadable Software Modules) and Dobby, operators can install, start, stop, and remove third-party applications on live deployed gateways without requiring a full firmware update. Applications run in isolated OCI containers with defined resource limits, security policies, and network access controls.

Categories of Third-Party Applications

Application Category	Description and Representative Companies
Home Security & Safety	Smart home security applications: motion detection, door/window sensor hubs, smoke/CO alert integration, and real-time alert notification. Companies: Alarm.com, Ring (Amazon), SimpliSafe.
Parental Control & Digital Wellbeing	Advanced parental control platforms with content categorization, screen time management, per-device scheduling, and threat protection beyond basic DNS filtering. Companies: Circle (Disney), Bark, Qustodio, Gryphon.
Network Security & Threat Intelligence	Inline threat detection, botnet prevention, malware blocking, and anomaly detection for all LAN devices. Companies: F-Secure (With Secure), Trend Micro Home Network Security, McAfee Secure Home Platform, Cujo AI.
Smart Home & IoT Hubs	Matter/Thread/Zigbee gateway applications that bridge IoT devices to cloud platforms. Companies: Amazon Alexa hub, Google Home hub integrations, SmartThings (Samsung), Home Assistant.
Wi-Fi Analytics & Optimization	Crowd-sourced and AI-driven Wi-Fi channel selection, interference mitigation, and quality optimization. Companies: Plume (Adapt), Cognitive Systems (Wi-Fi motion), Minim.
Voice Assistants & Smart Speakers	Embedded voice assistant runtimes that use the gateway as an always-on hub with multi-room audio routing. Companies: Amazon Alexa, Google Assistant (select ISP deployments).
Operator Value-Added Services	Operator-specific app bundles: advanced VPN, premium DNS (security/privacy), cloud backup for home, digital legacy, and enhanced parental dashboards offered as subscription upsells.
Network Performance & Gaming	Low-latency optimization, QoS gaming profiles, and bandwidth allocation applications. Companies: Netgear Armor (Bitdefender), TP-Link HomeCare.
Business / SMB Applications	VPN gateway, remote worker support, SD-WAN edge, and business continuity applications for the small business gateway profile. Companies: Cisco Meraki (select), OpenVPN.
Energy Management	Smart meter integration, energy disaggregation, and home energy management system (HEMS) bridges for utilities. Companies: Itron, Landis+Gyr, Sense.

Developer Access Model

Third-party developers access the RDK-B app platform through operator-managed app stores. Operators curate and approve applications, define resource quotas, and control deployment schedules via DSM. Developers package their software as OCI-compliant containers conforming to the RDK-B application container specification, which defines networking namespaces, RBUS IPC access permissions, and resource limit declarations.

8.6 Standards Bodies and Alliances

RDK-B is deeply embedded in the broader broadband standards ecosystem:

- Broadband Forum (BBF) — TR-181 data model, TR-369 (USP), TR-069 (CWMP), TR-104 (VoIP). RDK-B implements all of these.
- Wi-Fi Alliance — WPA3, EasyMesh, Wi-Fi 6/6E/7, Passpoint/Hotspot 2.0, WPS certifications. RDK-B's OneWiFi implements the full WFA certification feature set.
- IEEE 802.11 — the underlying wireless standard. RDK-B supports 802.11n/ac/ax/be.
- CableLabs — DOCSIS 3.1/3.0 specification, Packet Cable MTA. RDK-B cable-modem-agent and MTA agent implement these.
- Open Connectivity Foundation (OCF) / CSA — Matter protocol. RDK-B's BartonCore implements Matter specification.
- Open Container Initiative (OCI) — container runtime and image spec. Dobby and crun implement OCI.
- RDK Central TAB — RDK-B's own governance and specification body.

8.7 Summary: Stakeholder Value Matrix

Stakeholder	Role in Ecosystem	Primary Value from RDK-B
ISPs / Operators	Platform Consumer, Funder, Roadmap Driver	Vendor independence, operational consistency, rapid feature deployment, telemetry, fleet management, reduced TCO
SoC Vendors	HAL Implementer, Silicon Supplier	Design-win acceleration, operator access, HAL certification, faster customer qualification cycles
OEMs / ODMs	Hardware Builder, Firmware Integrator	Reduced software development cost, access to operator program, shorter certification timelines, reuse across SKUs
System Integrators	Customization & Integration Services	Revenue from platform expertise, long-term managed services, operator relationships, differentiated CI/CD toolchains
Third-Party App Developers	Application Publisher	Access to 100M+ deployed gateway devices as an application platform, operator billing integration, secure container deployment model
Cloud Service Providers	Backend Connectivity	Integration with management, telemetry, and orchestration endpoints; extension of cloud service reach to the home edge
Standards Bodies	Specification Authors	Reference implementation of their standards, broad industry adoption, feedback from real-world deployments

9. RDK-B Deployment Models

RDK-B supports a wide range of physical deployment models. Each model maps to a combination of device profiles, features, and cloud backends.

Deployment Model	Description
DOCSIS Cable Gateway	The original and most widely deployed RDK-B configuration. A single device combining a DOCSIS 3.0/3.1 cable modem with a Wi-Fi router and 4-port Ethernet LAN switch. Managed via TR-069 or USP by an operator ACS. Profiles: GW Key features: cable-modem-agent, MTA (for voice), OneWiFi, wan-manager, full management stack.
Fiber / DSL Broadband Gateway	A multi-WAN capable gateway with xDSL, GPON, or Ethernet WAN uplink. Wi-Fi, routing, NAT, and full management stack. Common in European deployments. Profiles: GW Key features: xdsl-manager / gpon-manager, EasyMesh, OneWiFi Wi-Fi 6/7, USP management.
Wi-Fi Mesh System (Gateway + Extenders)	A primary gateway (GW EasyMesh or GW OpenSync profile) paired with one or more wireless extenders (EXT EasyMesh or EXT OpenSync profile). Provides whole-home Wi-Fi coverage. The gateway is the EasyMesh Controller; extenders are Agents. Profiles: GW EasyMesh + EXT EasyMesh (or OpenSync variants) Key features: unified-wifi-mesh, ieee1905-rs, EasyMesh R6, 802.11kv, band steering.
Standalone Wi-Fi Router	A device with Ethernet WAN and multi-band Wi-Fi for subscribers with a separate modem (e.g., fiber ONT). No DOCSIS/DSL modem. Profile: EthWAN WiFi Router Key features: OneWiFi Wi-Fi 6/7, NAT/routing, DHCP server, WebUI, parental controls.
Business / SMB Gateway	A gateway optimized for small business use with business-grade security, VLAN segmentation, multi-WAN failover, VPN support, and business management portal. Profile: GW with business-grade features Key features: webui-bwg, multi-WAN, cellular backup, VPN, advanced firewall.
Hybrid / Multi-WAN Gateway	A gateway with multiple WAN interfaces including a primary broadband WAN and one or more backup WAN interfaces. Supports automatic WAN failover/restore. Profiles: GW or EthWAN WiFi Router Key features: wan-manager failover, cellular-manager (QMI/MBIM), GPS, USB RNDIS or other WAN interfaces
Edge Compute / App Platform	A gateway acting as an edge compute node hosting operator and third-party containerized applications. Requires the downloadable app container stack. Optional IoT support. Profile: GW with DSM/Dobby Key features: DSM, Dobby/crun, OCI bundle management, BartonCore IoT hub.

10. Device Profiles

RDK-B defines seven device profile archetypes. A profile is a high-level description of a device type — each built from required and optional components. "EthWAN WiFi Router" is a combined profile covering Ethernet-WAN-attached devices with integrated Wi-Fi.

Profile	Description
Modem / ONU	Headless cable modem or ONT. WAN termination and basic platform services only. No Wi-Fi, no LAN switching. Typical: DOCSIS cable modem, fiber ONT.
EthWAN WiFi Router	Ethernet WAN with full Wi-Fi, routing, NAT, and LAN capabilities. Covers Ethernet-WAN routers and Wi-Fi routers. IPv4/IPv6 router with simple Ethernet WAN port and Wi-Fi AP features.
GW (Gateway)	Full broadband gateway: multi-band Wi-Fi, routing, NAT, firewall, DHCP, DNS, device management. One or more WAN access technologies. Primary reference deployment target.
GW OpenSync	GW profile with OpenSync cloud management plane. Cloud-managed Wi-Fi optimization and remote configuration.
GW EasyMesh	GW with EasyMesh R6 Controller+Agent. Supports multi-AP mesh networks; gateway as Controller. Wi-Fi AP fronthaul and Wi-Fi/Ethernet backhaul.
EXT OpenSync	EasyMesh extender under OpenSync cloud management. Mesh agent/extender only. Wi-Fi AP fronthaul and Wi-Fi/Ethernet backhaul.
EXT EasyMesh	Pure EasyMesh extender without OpenSync. 802.11 relay and EasyMesh agent. Minimal system stack.

10.1 Feature Status Classification

Status	Meaning
CORE	Applies to the Core Features column only. Identifies a small subset of foundational components that: (1) represent a minimal code footprint, (2) form basic infrastructure reusable by other software, (3) are required across all device profiles, and (4) may be listed as a dependency by other components and features. CORE components must be implemented and functional for any profile to be conformant. Test coverage required.
Required	Applies to profile columns. The feature or sub-feature MUST be implemented and functional for the device to be conformant with this profile. Test coverage required.
OPT	Optional. Supported by the platform; included at operator/ODM discretion. Must comply with the specification when included.
N/A	Not applicable. The feature has no architectural relevance to this device profile.
—	Not defined for this profile/feature combination in the current specification.

11. Master Feature Matrix

Consolidated top-level summary of all 26 RDK-B feature areas mapped against all seven device profiles. Status reflects the highest-priority classification found across all sub-features within each area. Refer to Sections 12–36 for complete per-feature specifications.

#	Feature	Sub-System	Core Features	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
1	Wi-Fi LAN	LAN	—	—	Required	Required	Required	Required	Required	Required
2	Ethernet LAN	LAN	—	—	OPT	—	—	—	—	—
3	MoCA LAN	LAN	—	—	—	OPT	OPT	OPT	—	—
4	Local WebUI	Device Mgmt	—	—	OPT	OPT	OPT	OPT	—	—
5	Remote Device Management	Remote Mgmt	—	—	OPT	OPT	OPT	OPT	OPT	OPT
6	Telemetry & Data collection	Diagnostics	—	—	OPT	OPT	OPT	OPT	OPT	OPT
7	Diagnostics & Testing	Diagnostics	CORE	—	Required	Required	Required	Required	OPT	OPT
8	System Logging	Diagnostics	CORE	Required	Required	Required	Required	Required	Required	Required
9	Inter Process Communication	IPC	CORE	Required	Required	Required	Required	Required	Required	Required
10	Matter/IoT	IOT	—	—	OPT	OPT	OPT	OPT	OPT	OPT
11	Downloadable App containers	App Framework	—	—	OPT	OPT	OPT	OPT	OPT	OPT
12	Voice/VoIP	Voice	—	OPT	OPT	OPT	OPT	OPT	—	—
13	Firmware Upgrade	Device Mgmt	—	OPT	OPT	OPT	OPT	OPT	OPT	OPT
14	Connected device discovery	LAN	—	—	Required	Required	Required	Required	OPT	OPT
15	System startup & orchestration	System	CORE	Required	Required	Required	Required	Required	Required	Required
16	Configuration Persistence	System	—	Required	Required	Required	Required	Required	—	—
17	Factory reset	System	—	Required	Required	Required	Required	Required	—	—
18	IP Routing/NAT/Firewall	Routing & Bridging	—	Required	Required	Required	Required	Required	Required	OPT
19	Bridge and VLAN configuration	Routing & Bridging	—	—	Required	OPT	OPT	OPT	— OPT	— OPT
20	Traffic redirection	Routing & Bridging	—	—	OPT	OPT	OPT	OPT	—	—
21	DHCP Server	LAN	—	—	Required	Required	Required	Required	—	—

2 2	DNS	DNS	—	—	OPT	Required	Required	Required	—	—
2 3	WAN Connectivity	WAN	—	Required	Required	Required	Required	Required	—	—
2 4	Parental control	Parental Control	—	—	OPT	OPT	OPT	OPT	—	—
2 5	WiFi Hotspot	LAN	—	—	—	OPT	OPT	OPT	OPT	OPT
2 6	Code Security	Security	CORE	Required	Required	Required	Required	Required	Required	Required

12. Feature 1: Wi-Fi LAN

Sub-System: LAN **Primary Repository:** <https://github.com/rdkcentral/OneWifi>

12.1 Overview

Wi-Fi LAN is the single largest and most complex feature area in the RDK-B platform. It is delivered entirely through the OneWiFi component, which provides a unified, vendor-independent management layer over all 802.11 radio hardware on the device. OneWiFi abstracts vendor-specific driver behavior through the industry-standard NL80211 kernel interface, with a defined extension mechanism for features not yet covered by NL80211.

The feature spans the full IEEE 802.11 generational stack — from 802.11n (Wi-Fi 4) through 802.11ac (Wi-Fi 5), 802.11ax (Wi-Fi 6/6E), and the latest 802.11be (Wi-Fi 7). Security coverage is equally comprehensive: WPA2, WPA2-Enterprise (802.1X/RADIUS), WPA3-Personal (SAE), WPA3-Enterprise, WPA3-Transition mode, WPA3 Enhanced Open (OWE), and WPA3 PCM (Post-Connection Management). Each security mode is independently selectable per SSID.

Multi-band operation covers 2.4 GHz, 5 GHz (lower/upper sub-bands), and 6 GHz. The platform supports up to 8 Virtual Access Points (VAPs) per physical radio, enabling SSID segmentation for residential, business, public hotspot, and IoT traffic. EasyMesh R6 Controller and Agent functionality is implemented through the unified-wifi-mesh component, using the ieee1905-rs IEEE 1905 transport layer for backhaul communication between mesh nodes.

12.2 Key Capabilities

- 802.11n/ac/ax/be multi-generation multi-band radio management through a single unified component
- Full WPA3 security suite including SAE, Enterprise, Transition, Enhanced Open (OWE), and PCM modes
- EasyMesh R6 Controller+Agent with IEEE 1905.1 backhaul transport via ieee1905-rs
- Wi-Fi 7 Single-Link and Multi-Link Operation (SLO/MLO) with 320 MHz channel bandwidth support
- NL80211-based HAL interface with vendor extension support for new features
- Dynamic Frequency Selection (DFS) with radar detection and Channel Switch Announcement (CSA)
- Band steering, client steering, 802.11kv neighbor reporting and BSS transition
- Up to 8 Virtual APs per radio with independent SSID, security, and QoS settings
- Public Wi-Fi SSID with encrypted WAN, Passpoint/Hotspot 2.0 GAS/ANQP support
- Off-channel scanning, MAC filtering per network, connection admission control
- Comprehensive event reporting: assoc/disassoc events, management frames, RSSI thresholds
- Spectrum analyzer, CSI data collection for motion sensing, advanced telemetry data collection
- Seamless dynamic reconfiguration without subscriber service interruption

12.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Mode m /ONU	EthWA N WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
Standards								
802.11n	OneWiFi	—	Required	Require d	Required	Required	Required	Required
802.11ac	OneWiFi	—	Required	Require d	Required	Required	Required	Required

Feature Detail	Repository	Mode m /ONU	EthWA N WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
802.11ax (Wi-Fi 6/6E)	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
802.11be (Wi-Fi 7)	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Security								
WPA2	OneWiFi	—	Required	Require d	Required	Required	Required	Required
WPA2 Enterprise	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
WPA3	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
WPA3 Enterprise	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
WPA3 Transition	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
WPA3 Enhanced Open	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
WPA3 PCM	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Easy Mesh								
EasyMesh R6 – Controller+Agent	unified-wifi-mesh	—	OPT	—	—	Required	—	
EasyMesh R6 – IEEE 1905 transport ▶ Dependency for EasyMesh	ieee1905-rs	—	OPT	—	—	Required	—	Required
Wi-Fi 7								
802.11 be	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
320 MHz	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
SLO (Single Link Operation) and related WFA Data Model	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
MLO (Multi-Link Operation) and related WFA Data Model	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
HAL interface to vendor software								
Support industry standard NL80211 vendor interface	OneWiFi	—	Required	Require d	Required	Required	Required	Required

Feature Detail	Repository	Mode m /ONU	EthWA N WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
Support extensions to NL80211 interface where needed for new features	OneWiFi	—	Required	—	—	—	—	—
Multi Band								
2.4 GHz	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
5 GHz / 5U GHz / 5L GHz	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
6 GHz	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Supported Operational Modes								
AP mode supporting up to 8 Virtual APs per radio ▶ AP mode	OneWiFi	—	OPT	Require d	Required	Required	—	—
Extender mode ▶ Extender profile	OneWiFi	—	N/A	—	—	—	Required	Required
Event reporting								
802.11 event reporting	OneWiFi	—	Required	Require d	Required	Required	Required	Required
Reporting of all Mgmt Frames	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Data Collection for Wi-Fi Assisted device typing	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Connection admission control	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
CSI data collection for Motion Sensing and other applications	—	—	OPT	OPT	OPT	OPT	—	—
Band Steering	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Client steering to another AP	—	—	OPT	OPT	OPT	OPT	OPT	OPT
Data collection for Lev1 Fingerprinting	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Mac Filters Per Network	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT

Feature Detail	Repository	Mode m /ONU	EthWA N WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
Data Collection for Harvester Telemetry ▶ WiFi telemetry	OneWiFi	—	OPT	OPT	OPT	OPT	—	—
802.11 kv ▶ Neighbor report & BSS transition	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Off channel scan	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
DFS ▶ Dynamic Frequency Selection	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Management frame Power control	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Data Collection for Advanced Telemetry	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Radius Failover & IPV6 Support for Public Access ▶ Encrypted WAN is needed for security with Public Hotspot	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Cloud control WiFi optimization	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Public Wi-Fi SSID								
public SSID ▶ Encrypted WAN is needed for security with Public Hotspot	hotspot	—	—	OPT	OPT	OPT	OPT	OPT
Passpoint / Hotspot 2.0								
Passpoint ▶ Encrypted WAN is needed for security with Public Hotspot	OneWiFi	—	—	OPT	OPT	OPT	OPT	OPT
WPS								
WiFi Protected Setup	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Connected client reporting								

Feature Detail	Repository	Mode m /ONU	EthWAN WiFi Router	GW	GW OpenSyn c	GW EasyMesh	EXT OpenSyn c	EXT EasyMesh
Connected client list & assoc. events	OneWiFi	—	Required	Required	Required	Required	Required	Required
Spectrum analyzer								
Spectrum analyzer	OneWiFi	—	OPT	OPT	OPT	OPT	OPT	OPT
Test framework support								
Onewifi test suite & Vendor test suite	OneWifitestsuite	—	N/A	—	—	—	—	—
Reconfiguration								
seamless WiFi dynamic reconfiguration support	OneWiFi	—	Required	Required	Required	Required	Required	Required

12.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- OneWiFi
- unified-wifi-mesh
- ieee1905-rs
- OneWiFiTestSuite
- hotspot

12.5 Implementation Notes

EasyMesh R6 – IEEE 1905 transport: Dependency for EasyMesh

AP mode supporting up to 8 Virtual APs per radio: AP mode

Extender mode: Extender profile

Data Collection for Harvester Telemetry: WiFi telemetry

802.11 kv: Neighbor report & BSS transition

DFS: Dynamic Frequency Selection

Radius Failover & IPV6 Support for Public Access: Encrypted WAN is needed for security with Public Hotspot

public SSID: Encrypted WAN is needed for security with Public Hotspot

Passpoint: Encrypted WAN is needed for security with Public Hotspot

12.6 Testing Scope

- Radio bring-up and configuration validation for all supported bands and channel widths
- WPA2/WPA3 security handshake validation (SAE, OWE, 802.1X/RADIUS) per mode
- EasyMesh R6 Controller/Agent pairing, topology convergence, and STA steering validation
- Wi-Fi 7 MLO link setup/teardown and per-link statistics data model validation

- DFS radar detection compliance and CSA propagation timing
- 802.11kv neighbor report generation and BSS transition request/response
- Band steering trigger conditions, request generation, and client response handling
- VAP isolation — validate that traffic between SSIDs is correctly firewalled
- WPS PBC and PIN mode: session, credential exchange, and AP config update
- Passpoint GAS/ANQP query/response and Hotspot 2.0 indication element
- Connected client list accuracy, assoc event timing, and RSSI reporting precision

DRAFT (Internal Reference)

13. Feature 2: Ethernet LAN

Sub-System: LAN **Primary Repository:** —

13.1 Overview

Ethernet LAN provides wired connectivity management for devices connected to the gateway's LAN-side Ethernet ports. The Ethernet Agent monitors port link state, manages link speed negotiation, and provides operators with the ability to force IP reconfiguration on connected devices.

The forced IP reconfiguration capability is particularly valuable for operators managing dynamic WAN failover/restore scenarios — by momentarily dropping and re-establishing a LAN Ethernet link, the gateway triggers the connected device to release and reacquire its DHCP lease, ensuring it receives up-to-date network configuration when the WAN changes, without requiring the subscriber to manually power-cycle their client device.

Link speed detection and management enable the gateway to report and control the negotiated speed on each Ethernet port, providing visibility into potential bottlenecks caused by legacy cabling or client hardware limitations.

13.2 Key Capabilities

- Per-port link up/down detection with event generation for connected device tracking
- Forced IP reconfiguration on reconnect — momentarily drops link to trigger DHCP renewal on connected client
- Link speed detection and reporting for all LAN Ethernet ports
- Integration with Device.Hosts table for connected device inventory updates

13.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Connected device detection								
Link up/down detection	Ethernet Agent	—	OPT	OPT	OPT	OPT		—
Enable/disable link including forced IP reconfig on reconnect								
Includes ability to force a connected device to reconfigure IP by momentarily dropping the link	Ethernet Agent	—	OPT	OPT	OPT	OPT	—	—
Link Speed management								
Link speed detection and management	Ethernet Agent	—	OPT	OPT	OPT	OPT	—	—

13.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- Ethernet Agent
- lan-manager-lite
- dhcp-manager

13.6 Testing Scope

- Link state event generation on cable connect/disconnect
- Forced IP reconfiguration — This feature is often used to force client DHCP renewal after WAN failover / restore
- Link speed negotiation reporting accuracy across 10/100/1000 Mbps rates

14. Feature 3: MoCA LAN

Sub-System: LAN **Primary Repository:** <https://github.com/rdkcentral/moca-agent>

14.1 Overview

MoCA (Multimedia over Coax Alliance) LAN enables high-performance home networking over existing coaxial cable infrastructure, providing an alternative to Wi-Fi backhaul for EasyMesh extenders or wired connectivity for LAN devices without rewiring.

RDK-B implements MoCA support through the moca-agent component, which monitors node status, link quality, PHY rates, and network topology. A dedicated HAL interface (rdkb-halif-moca) provides the abstraction layer between the moca-agent middleware and the silicon vendor's MoCA firmware.

The rdkb-halif-test-moca test suite validates all mandatory HAL functions before a MoCA SoC implementation can be considered RDK-B compatible, covering node initialization, link establishment, frequency selection, and statistics reporting.

14.2 Key Capabilities

- MoCA LAN-over-coax connectivity for devices without Wi-Fi capability
- MoCA network node monitoring including PHY rate, SNR, and topology
- HAL abstraction enabling multiple MoCA silicon implementations
- HAL test suite for vendor certification of MoCA implementations

14.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
MoCA LAN Features								
MoCA LAN-over-cable	moca-agent	—	N/A	OPT	OPT	OPT	—	—
MoCA HAL interface	rdkb-halif-moca	—	N/A	OPT	OPT	OPT	—	—
MoCA HAL test	rdkb-halif-test-moca	—	N/A	OPT	OPT	OPT	—	—

14.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- moca-agent
- rdkb-halif-moca
- rdkb-halif-test-moca

14.6 Testing Scope

- MoCA node initialization and network join/leave event reporting
- PHY rate, SNR, and frequency reporting accuracy via HAL

- HAL test suite: all mandatory rdkb-halif-moca functions validated

DRAFT (Internal Reference)

15. Feature 4: Local WebUI

Sub-System: Device Mgmt **Primary Repository:** <https://github.com/rdkcentral/webui>

15.1 Overview

The Local WebUI provides a browser-accessible management interface served directly from the gateway, enabling subscribers and network administrators to configure the device without requiring a cloud connection or operator support call.

Two portal variants are provided: a residential portal (webui) delivering a consumer-friendly interface covering Wi-Fi settings, connected device management, parental controls, and basic diagnostics; and a business gateway portal (webui-bwg) providing additional features appropriate for SMB deployments such as VLAN configuration, advanced firewall rules, and multi-WAN status.

Both portals are served over HTTPS and require authentication, ensuring that local management access is protected even in shared or semi-public network environments.

15.2 Key Capabilities

- HTTPS-secured local management portal accessible from any browser on the LAN
- Residential portal: Wi-Fi SSID/password configuration, connected device list, parental controls, speed test, diagnostics
- Business gateway portal: VLAN configuration, advanced firewall, multi-WAN status, business-specific reporting
- Authentication-protected access with session management

15.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Residential Portal								
HTML/HTTPS residential mgmt portal	webui	—	OPT	OPT	OPT	OPT	—	—
Business Gateway Portal								
Business gateway HTML portal	webui-bwg	—	OPT	OPT	OPT	OPT	—	—

15.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- webui
- webui-bwg

15.6 Testing Scope

- HTTPS certificate validation and redirect from HTTP

- Authentication enforcement — verify unauthenticated access is blocked
- Wi-Fi configuration changes via UI propagate to OneWiFi RBUS data model
- Connected device list accuracy matches Device.Hosts TR-181 table
- Business portal: VLAN and firewall rule application from UI

DRAFT (Internal Reference)

16. Feature 5: Remote Device Management

Sub-System: Remote Mgmt **Primary Repository:** <https://github.com/xmidt-org/parodus2ccsp>
<https://github.com/xmidt-org/parodus> <https://github.com/rdkcentral/start-parodus>

16.1 Overview

Remote Device Management is the set of protocols and frameworks through which operators remotely configure, monitor, and update deployed gateway devices at scale. RDK-B supports the complete spectrum of management protocols, including USP/TR-369, TR-069, WebPA and other protocols providing operators with flexibility to choose the protocol that best fits their management infrastructure.

WebPA (parodus) provides a cloud-based parameter access mechanism for WebPA-based management systems such as Comcast's XPC platform. USP (TR-369, via usp-pa-vendor-rdk) is another management protocol providing full device management including commands, events, and bulk data collection over MQTT or WebSocket. TR-069/CWMP (tr069-protocol-agent) provides ACS-based management for operators using traditional CWMP infrastructure.

RFC (Remote Feature Control) provides another alternative for operators to toggle platform features at runtime at scale—a capability critical for operators managing fleets of 10M+ devices. WebConfig enables configuration push of structured JSON payloads to specific component namespaces.

16.2 Key Capabilities

- WebPA/parodus: cloud-based parameter get/set over WebSocket; Comcast XPC and compatible systems
- USP (TR-369): device management with commands, events, bulk data; MQTT/WebSocket transport
- TR-069/CWMP: ACS-based configuration, firmware management, diagnostics; legacy operators
- RFC (Remote Feature Control): runtime feature enable/disable via xConf
- WebConfig: structured configuration push to configuration atomically

16.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
WebPA								
WebPA device management ▶ No commands or event support	parodus start-parodus xmidt-org	—	OPT	OPT	OPT	OPT	OPT	OPT
USP								
USP (TR-369) device management	usp-pa-vendor-rdk	—	OPT	OPT	OPT	OPT	OPT	OPT
TR069								
CWMP / ACS device management	tr069-protocol-agent	—	OPT	OPT	OPT	OPT	OPT	OPT
RFC (remote feature control)								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Runtime enable/disable via xConf	xconf-client	OPT	OPT	OPT	OPT	OPT	OPT —	—
WebConfig								
WebConfig based device configuration ▶ No TR-181 support	WebConfig	OPT	OPT	OPT	OPT	OPT	OPT	—

16.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- parodus
- start-parodus
- usp-pa-vendor-rdk
- tr069-protocol-agent
- snmp-protocol-agent
- xconf-client
- WebconfigFramework

16.5 Implementation Notes

WebConfig based device configuration: No TR-181 support

16.6 Testing Scope

- WebPA: parameter get/set/notify round-trip through parodus to RBUS
- USP: device management session establishment, command execution, event subscription
- TR-069: ACS bootstrap, provisioning, inform cycle, firmware download trigger
- RFC: feature flag toggle via xConf
- WebConfig: configuration payload delivery and namespace-specific application
- All management protocols: verify TLS channel security and certificate validation

17. Feature 6: Telemetry & Data collection

Sub-System: Diagnostics **Primary Repository:** <https://github.com/rdkcentral/telemetry>

17.1 Overview

Telemetry and Data Collection provide the mechanisms through which operators gather operational metrics from deployed devices to monitor fleet health, detect issues proactively, and measure QoS KPIs across millions of simultaneous deployments.

The primary telemetry mechanism is T2 (Telemetry 2.0), a profile-driven framework that allows operators to define telemetry collection profiles specifying which metrics to collect, how frequently, how to aggregate them, and where to upload them. T2 profiles can be delivered and updated at runtime without a firmware upgrade, giving operators dynamic control over what data is collected from their fleet.

Crash data upload (crashupload) provides automatic capture and delivery of crash dumps, core files, and stack traces to the operator Backoffice. This component is intentionally kept dependency-free to ensure it can operate even when other components have failed. The Harvester component provides legacy hard-coded telemetry for backward compatibility with WebPA-based collection systems.

17.2 Key Capabilities

- T2 Telemetry 2.0: profile-driven telemetry collection with configurable markers, reports, and upload destinations
- Runtime profile delivery via xConf — no firmware upgrade required to change what is collected
- Crash data upload: zero-dependency crash dump capture, core file collection, and Backoffice delivery
- Harvester: legacy hard-coded WebPA telemetry pipeline for backward-compatible collection
- Configurable upload frequency, aggregation windows, and backoff policies
- Support for structured (JSON) and unstructured (log-based) telemetry formats

17.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Configurable Telemetry								
Profile-driven telemetry collection	T2 telemetry	—	OPT	OPT	OPT	OPT	OPT	OPT
Crash Reporting								
Crash data upload ▶ Zero deps	crashupload	—	OPT	OPT	OPT	OPT	OPT	OPT
Harvester Telemetry								
Harvester – hard-coded WebPA telemetry ▶ Legacy – not for new dev	harvester	—	OPT	OPT	OPT	OPT	OPT	OPT

17.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- telemetry
- crashupload
- harvester
- xconf-client
- rdk_logger

17.5 Implementation Notes

Crash data upload: Zero deps

Harvester – hard-coded WebPA telemetry: Legacy – not for new dev

17.6 Testing Scope

- T2: profile delivery via xConf and marker activation within specified latency
- T2: report generation accuracy against reference dataset
- T2: upload success rate and retry behavior on connectivity loss
- crashupload: crash dump captured and uploaded on controlled fault injection
- Validate T2 operates without errors when Harvester is not present (OPT dependency)

18. Feature 7: Diagnostics & Testing

Sub-System: Diagnostics **Primary Repository:** <https://github.com/rdkcentral/test-and-diagnostic>

18.1 Overview

Diagnostics and Testing provide the tools through which operators, field engineers, and automated systems validate connectivity, measure performance, investigate issues, and ensure continuous device health. These tools are designed to be remotely triggerable via RBUS/USP, enabling fleet-wide diagnostic campaigns without physical device access.

The self-heal module is a critical component that continuously monitors the device's operational state and triggers recovery actions when components become unresponsive or when connectivity is lost. Self-heal implements a tiered recovery strategy: first attempting component restart, then partial reboot sequences, and finally a full device reboot if softer recovery actions are insufficient.

TCP latency measurement provides a standardized mechanism for operators to measure round-trip latency from the gateway to a specified target, enabling detection of WAN congestion, ISP network issues, and subscriber-reported slowness at scale.

18.2 Key Capabilities

- Layer 4 connectivity tests: ping, traceroute, DNS resolution, HTTP reachability
- Self-heal monitoring: continuous health checks with tiered recovery (restart → partial reboot → full reboot)
- TCP latency measurement to configurable targets; results published via RBUS/T2
- Gateway speed test (IMP): measures WAN bandwidth from the gateway perspective
- Live field debug via remote_debugger: SSH-based access to running device for L2/L3 issue investigation
- CPU and process analyzer (cpuprocanalyzer): on-demand resource profiling for memory and CPU investigations
- WAN health check (IPoE): validates IPoE WAN connectivity and DHCP lease health
- Legacy notification component for hard-coded DM value change notifications

18.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Connectivity Tests								
Layer 4 connectivity tests	test-and-diagnostic	—	OPT	OPT	OPT	OPT	OPT	OPT
WAN Health								
IPOE WAN health check	ipoe-health-check	—	OPT	OPT	OPT	OPT	—	—
Field Debug								
Live field issue debugging	remote_debugger	Required	Required	Required	Required	Required	Required	Required
cpuproc analyzer								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Process Analyzer	cpuprocanalyzer	—	OPT	OPT	OPT	OPT	OPT	OPT
Legacy Notifications								
Hard-coded DM value change notify ▶ Legacy	notify-component	—	OPT	OPT	OPT	OPT	—	—
Gateway speed test								
IMP speed test	test-and-diagnostic	—	OPT	OPT	OPT	OPT	—	—
TCP Latency measurement								
Latency	Test & Diagnostics	—	OPT	OPT	OPT	OPT	—	—
Self heal								
monitoring and recovery module	Test & Diagnostics	OPT	OPT	OPT	OPT	OPT	OPT	OPT

18.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- test-and-diagnostic
- remote_debugger
- cpuprocanalyzer
- notify-component
- rdk_logger

18.5 Implementation Notes

IPOE WAN health check:

Hard-coded DM value change notify: Legacy

18.6 Testing Scope

- Self-heal: inject component failure and validate correct recovery action and timing
- TCP latency: validate measurement accuracy against reference latency source
- Layer 4 tests: validate each test type (ping, traceroute, DNS, HTTP) against known good/bad targets
- remote_debugger: validate secure session establishment and command execution
- cpuprocanalyzer: validate process list accuracy and resource metrics against OS ground truth

19. Feature 8: System Logging

Sub-System: Diagnostics **Primary Repository:** https://github.com/rdkcentral/rdk_logger

19.1 Overview

System Logging is a foundational platform service required across all device profiles and all component types. The `rdk_logger` framework provides a unified, consistent logging API that any RDK-B component can use without introducing additional dependencies — the library is intentionally kept zero-dependency to ensure it can be included in the most resource-constrained Modem/ONU or Extender profiles.

The logging framework enforces consistent log formatting across all components, including mandatory fields such as component name, log level, timestamp, and process ID. This consistency is essential for automated log parsing, log aggregation in Backoffice systems, and for support engineers who need to correlate logs from multiple components across different device types.

Log rotation prevents flash storage exhaustion on resource-constrained devices by managing log file size limits and rotation policies. The DCM agent (Device Configuration Manager) handles secure upload of collected logs to the operator Backoffice infrastructure, supporting both on-demand upload triggered by operator request and scheduled automatic upload.

19.2 Key Capabilities

- Zero-dependency unified logging API — included in all device profiles from Modem/ONU to Gateway
- Consistent log format: component name, timestamp, log level, process ID, thread ID
- Configurable log levels (FATAL, ERROR, WARN, INFO, DEBUG, TRACE) per component
- Log rotation with configurable file size limits and retention policies for flash-constrained devices
- DCM agent: secure log upload to operator Backoffice on schedule or on-demand trigger
- Log buffering to handle temporary upload failures with automatic retry

19.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Logger								
Unified log framework — zero deps ▶ Required in all profiles	<code>rdk_logger</code>	Required	Required	Required	Required	Required	Required	Required
Log rotation								
Log file rotation management	<code>rdk_logger</code>	Required	Required	Required	Required	Required	Required	Required
Log upload								
Log upload to backoffice	<code>dcm</code>	OPT	OPT	OPT	OPT	OPT	OPT	OPT

19.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- rdk_logger
- dcm-agent

19.5 Implementation Notes

Unified log framework – zero deps: Required in all profiles

19.6 Testing Scope

- Validate rdk_logger compiles and functions with zero external library dependencies
- Log format conformance: all mandatory fields present and correctly formatted
- Log rotation: validate file size limits are enforced and rotated files are managed correctly
- DCM upload: validate upload success, retry on failure, and upload confirmation
- Log level filtering: validate that setting DEBUG level does not emit TRACE output

20. Feature 9: Inter Process Communication

Sub-System: IPC **Primary Repository:** <https://github.com/rdkcentral/rbus>

20.1 Overview

Inter-Process Communication (IPC) is the fundamental communication backbone of the RDK-B platform. All data model access, configuration changes, event notifications, and method invocations between components flow through the RBUS framework. RBUS is the modern, actively developed IPC system for RDK-B, providing a clean, typed, publish-subscribe plus RPC communication model based on the TR-181 data model hierarchy.

RBUS provides five primitives: get (read a TR-181 parameter value), set (write a TR-181 parameter value), multi-instance object management (create/delete/iterate TR-181 table rows), method invocation (RPC call to a component's registered handler, both Sync and Async methods are supported), and event pub/sub (subscribe to notifications when an event occurs, a parameter value changes or notifications can be sent on a periodic basis for property values that are continuously changing and don't make sense to send "on change"). All five primitives are available over the RBUS socket, accessible from any component in the system.

For backward compatibility with the large installed base of components written against the legacy CCSP/DMCLI IPC model, the CCSP shim layer has been modified to use RBUS transport protocols and provide interoperability with newer RBUS components. This allows legacy components to continue functioning without modification and interoperate with newer RBUS components, while the platform migrates to pure RBUS.

The inter-device-manager component extends RBUS connectivity across LAN-attached devices, enabling a gateway to access data model parameters, subscriptions and methods that are implemented on other compatible devices on the same LAN..

20.2 Key Capabilities

- RBUS get/set: read/write TR-181 data model parameters with type validation and access control
- RBUS multi-instance: create, delete, and iterate TR-181 table rows (e.g., Device.WiFi.SSID.{i})
- RBUS method invocation: typed RPC with defined input/output parameter schemas
- RBUS method invocation ASYNC: typed RPC with defined input/output parameter schemas implemented with non-blocking calls and asynchronous notification when the RPC call completes.
- RBUS event pub/sub: event notification, value-change notification with configurable filter and debounce and periodic value notification on interval.
- RBUS CLI: Newer command-line interface for operator and developer data model access. This can be used both to access data models in newer RBUS compatible components and in legacy CCSP components.
- CCSP shim (common-library): backward-compatible CCSP API over RBUS for legacy components
- DMCLI (data-model-cli): legacy command-line CCSP data model access for legacy scripting. Unlike the newer RBUS CLI, the older DMCLI does not support newer RBUS features like methods and Eventing. This is not recommended for new use cases or development.
- inter-device-manager: RBUS bridge extending IPC connectivity across LAN-connected nodes

20.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
RBUS – Modern IPC								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
RBUS CLI Support	rbus	Required	Required	Required	Required	Required	Required	Required
Support for get and set	rbus	Required	Required	Required	Required	Required	Required	Required
Support for multi-instance objects	rbus	Required	Required	Required	Required	Required	Required	Required
Method / command invocation	rbus	Required	Required	Required	Required	Required	Required	Required
Event subscription & notification	rbus	Required	Required	Required	Required	Required	Required	Required
Legacy CCSP Shim								
Legacy DMCLI support	DMCLI	OPT—	Required	Required —	Required —	Required —	OPT—	OPT—
CCSP backward-compat API over RBUS ▸ Legacy – needed by existing components	common-library	Required	Required	Required	Required	Required	Required	Required
Inter-Device IPC								
RBUS bridge between LAN devices	inter-device-manager	OPT —	OPT	OPT	OPT	OPT	OPT —	OPT —

20.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- rbus
- common-library
- data-model-cli
- inter-device-manager

20.5 Implementation Notes

CCSP backward-compatible API over RBUS: Legacy – needed by some older components

20.6 Testing Scope

- RBUS get/set round-trip latency measurement against TR-181 reference parameters
- RBUS event subscription and notification delivery accuracy and timing
- RBUS method invocation with valid and invalid argument sets
- Multi-instance CRUD: create/read/update/delete TR-181 table row lifecycle
- CCSP shim: DMCLI get/set maps correctly to RBUS outcome
- inter-device-manager: validate parameter access across device bridge
- Concurrent access: validate correct serialization under multi-threaded access patterns

21. Feature 10: Matter/IoT

Sub-System: IOT **Primary Repository:** <https://github.com/rdkcentral/BartonCore>

21.1 Overview

Matter/IoT provides native smart home protocol support, enabling the RDK-B gateway to act as a Matter border router and IoT hub for the subscriber's home network. This positions the broadband gateway as the central home intelligence node, rather than requiring a separate smart home hub device.

The BartonCore framework implements Matter (formerly CHIP — Connected Home over IP), Zigbee, Thread, and Bluetooth Low Energy (BLE) commissioning. Matter support enables the gateway to discover, commission, and control Matter-certified IoT devices from any ecosystem — Apple Home, Google Home, Amazon Alexa, and Samsung SmartThings all support Matter interoperability.

Thread is the IPv6-based mesh networking protocol used as the transport layer for many Matter devices. BartonCore implements a Thread Border Router, connecting Thread network devices to the broader IP network. Zigbee bridging allows legacy Zigbee IoT devices to be integrated into the Matter ecosystem through the gateway.

21.2 Key Capabilities

- Matter border router: discover, commission, and control Matter-certified smart home devices
- Thread Border Router: connect Thread mesh network to IPv6 home network
- Zigbee bridging integrate legacy Zigbee devices into Matter/IP ecosystem
- BLE for Matter commissioning: Bluetooth LE as the out-of-band commissioning transport
- Multi-ecosystem interoperability: Apple, Google, Amazon, Samsung via Matter standard

21.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Matter								
Matter IoT device support	BartonCore	—	OPT	OPT	OPT	OPT	OPT	OPT
Zigbee								
Zigbee IoT device support	BartonCore	—	OPT	OPT	OPT	OPT	OPT	OPT
Thread								
Thread IoT device support	BartonCore	—	OPT	OPT	OPT	OPT	OPT	OPT
Bluetooth								
Bluetooth support for Matter comissioning	BartonCore	—	OPT	OPT	OPT	OPT	OPT	OPT

21.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- BartonCore

21.6 Testing Scope

- Matter device commissioning flow: QR code scan, BLE discovery, onboarding, and control
- Thread Border Router: Thread device join, IP connectivity, and mesh formation
- Zigbee bridge: device pairing and command relay to Matter ecosystem
- Multi-device concurrency: validate stable operation with 10+ simultaneous IoT devices

22. Feature 11: Downloadable App containers

Sub-System: App Framework **Primary Repository:** <https://github.com/rdkcentral/DSM>

22.1 Overview

Downloadable App Containers enable operators to deploy, update, and remove third-party and operator-developed applications on live deployed gateways without requiring a full firmware upgrade. This transforms the broadband gateway from a fixed-function appliance into an open, programmable edge compute platform.

The Downloadable Software Modules (DSM) component implements the TR-181 SoftwareModules data model, which defines the data model objects and RPCs for application lifecycle management: install, update, start, stop, and remove. DSM communicates with operator app stores and orchestration systems via USP or WebPA and manages the application inventory and state on the device.

Application execution is handled by Dobby, an OCI-compatible container runtime purpose-built for embedded Linux environments. Dobby uses crun as the low-level container runtime, implementing the OCI Runtime Specification. Containers run in isolated Linux namespaces with defined CPU, memory, and network resource limits declared in OCI bundle configuration. Dobby enforces security policies preventing container escape and unauthorized access to host resources.

22.2 Key Capabilities

- TR-181 SoftwareModules data model: application inventory, version, and lifecycle state management
- App install/update/start/stop/remove via USP RPC or WebConfig delivery
- OCI-compliant container runtime (Dobby + crun) with Linux namespace isolation
- Resource limits: CPU, memory, network bandwidth per container via cgroup constraints
- Security isolation: container escape prevention, restricted host filesystem access
- Multiple container runtimes supported: crun (primary), youki (optional alternative)
- App store integration: operator-curated app delivery and versioning

22.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Mode m /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
App LCM								
TR-181 SoftwareModules data model ▶ Active dev – TR-181 compliance	DSM	OPT	OPT	OPT	OPT	OPT	OPT	OPT
App install/start/stop/remove	DSM	OPT	OPT	OPT	OPT	OPT	OPT	OPT
Container Runtime								
OCI container runtime (crun)	Dobby	OPT	OPT	OPT	OPT	OPT	OPT	OPT
Container lifecycle management	Dobby	OPT	OPT	OPT	OPT	OPT	OPT	OPT

22.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- DSM
- Dobby
- crun

22.5 Implementation Notes

TR-181 SoftwareModules data model: Active dev – TR-181 compliance

22.6 Testing Scope

- App install via USP: OCI bundle download, verification, and container instantiation
- App lifecycle: start/stop/restart transitions and state machine validation
- Resource limits: validate CPU and memory constraints are enforced by cgroup
- Isolation: validate container cannot access host filesystem paths outside its mount
- App update: validate running app can be updated in-place with minimal service interruption
- Remove: validate full cleanup of container rootfs and data volumes on removal

23. Feature 12: Voice/VoIP

Sub-System: Voice **Primary Repository:** <https://github.com/rdkcentral/media-terminal-adapter-agent>

23.1 Overview

Voice/VoIP support in RDK-B covers two distinct deployment scenarios: Packet Cable 2.0 MTA (Multimedia Terminal Adapter) (also known as eDVA) for traditional cable operator voice services, and TR-104-based telco voice stack management for DSL and fiber operator VoIP deployments.

The Packet Cable 2.0 MTA agent (media-terminal-adapter-agent) monitors the status of the embedded Digital Voice Adapter (eDVA) within a DOCSIS gateway, reporting registration state, call state, line status, and diagnostic information via the TR-181 data model. This enables operators to remotely diagnose voice service issues, trigger diagnostics, and monitor battery backup status for E911 compliance. Battery backup management for MTA scenarios ensures continuous E911 service availability during power outages.

The telco-voice-manager implements TR-104 data model management for SIP-based VoIP stacks, supporting operators who deploy RDK-B on DSL or fiber gateways where the voice stack is provided by a third-party SIP client library.

23.2 Key Capabilities

- Packet Cable MTA status monitoring: registration state, call state, line status, endpoint status
- MTA diagnostics: loopback, tone generation, hook state reporting
- MTA HAL interface (rdkb-halif-mta): vendor-agnostic MTA firmware abstraction
- MTA HAL test suite for RDK Ready certification of MTA silicon implementations
- TR-104 telco voice stack management: SIP registration, call state, line configuration
- Battery backup management: power mode transitions, battery health, runtime estimation
- Power manager HAL and test suite for battery SoC integration

23.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
PacketCable 2.0 MTA								
MTA status & diagnostics ▶ PacketCable 2.0 eDVA	media-terminal-adapter-agent	OPT	N/A	OPT	OPT	OPT	—	—
MTA HAL interface	rdkb-halif-mta	OPT	N/A	OPT	OPT	OPT	—	—
MTA HAL test	rdkb-halif-test-mta	OPT	N/A	OPT	OPT	OPT	—	—
Telco Voice								
TR104 telco voice stack mgmt	telco-voice-manager	OPT	OPT	OPT	OPT	OPT	—	—
Battery Backup								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Battery-backed power modes ▶ For MTA battery scenarios	power-manager	OPT —	N/A	OPT	OPT	OPT	—	—
Power manager HAL	rdk-halif-power_manager	OPT —	N/A	OPT	OPT	OPT	—	—
Power manager HAL test	rdk-halif-test-power_manager	OPT —	N/A	OPT	OPT	OPT	—	—

23.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- media-terminal-adapter-agent
- rdkb-halif-mta
- rdkb-halif-test-mta
- telco-voice-manager
- power-manager
- rdk-halif-power_manager

23.5 Implementation Notes

MTA status & diagnostics: Packet Cable 2.0 eDVA

Battery-backed power modes: For MTA battery scenarios

23.6 Testing Scope

- MTA HAL: all mandatory rdkb-halif-mta functions validated by test suite
- MTA registration state monitoring: validate state transitions are reported accurately
- Battery backup: validate power mode transitions and runtime estimation
- TR-104: SIP registration state and line status reporting accuracy
- MTA diagnostics: validate loopback and tone test trigger/result via RBUS

24. Feature 13: Firmware Upgrade

Sub-System: Device Mgmt **Primary Repository:** <https://github.com/rdkcentral/platform-manager>

24.1 Overview

Firmware Upgrade is a critical operational capability enabling operators to remotely update device software across their entire deployed fleet. Firmware updates deliver security patches, bug fixes, and new feature activations — maintaining device security and service quality throughout the device's operational lifetime, which commonly spans 5–10 years.

The platform-manager component handles the complete firmware upgrade lifecycle: receiving the upgrade request from the cloud, downloading the firmware image from the CDN, validating the image signature, applying the image to the inactive flash partition, and triggering the reboot to the new firmware. On failure, the device automatically falls back to the previous firmware version, ensuring the device remains operational.

The xConf client /USP/WebPA manages the cloud-side firmware orchestration: checking for available updates, receiving update schedules, managing maintenance windows, and reporting upgrade status back to the xConf service. This enables operators to perform phased rollouts to subsets of their fleet, reducing the risk of fleet-wide issues from a single bad firmware release.

24.2 Key Capabilities

- OTA firmware downloads with CDN redirect support and resume-on-failure
- Image signature verification before applying any firmware update
- Dual-partition (A/B) firmware update with automatic rollback on boot failure
- xConf-based cloud orchestration: update scheduling, maintenance windows, phased rollout
- Firmware upgrade HAL (rdkb-halif-fwupgrade): vendor-agnostic flash write abstraction
- HAL test suite for RDK Ready certification of firmware upgrade implementations
- Update status reporting via RBUS and T2 telemetry for fleet-wide upgrade tracking

24.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
OTA Upgrade								
Firmware download & apply ► Platforms without CM Agent	platform-manager	OPT —	OPT	OPT	OPT	OPT	OPT	OPT
FW upgrade HAL interface	rdkb-halif-fwupgrade	OPT —	OPT	OPT	OPT	OPT	OPT	OPT
FW upgrade HAL test	rdkb-halif-test-fwupgrade	OPT —	OPT	OPT	OPT	OPT	OPT	OPT
Cloud Connection								
xConf-based FW orchestration	xconf-client	OPT —	OPT	OPT	OPT	OPT	OPT	OPT

24.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- platform-manager
- xconf-client
- rdkb-halif-fwupgrade
- rdkb-halif-test-fwupgrade

24.5 Implementation Notes

Firmware download & apply: Platforms without CM Agent

24.6 Testing Scope

- FW upgrade HAL: all mandatory functions validated by rdkb-halif-test-fwupgrade
- End-to-end OTA: trigger via xConf, download, signature verify, apply, reboot, verify new version
- Rollback: inject controlled boot failure on new firmware, verify automatic rollback to previous
- Phased rollout: validate device checks eligibility against xConf criteria before applying
- Download resume: interrupt download mid-transfer, validate resume from checkpoint
- Signature failure: present deliberately corrupted image, validate rejection without flash write

25. Feature 14: Connected device discovery

Sub-System: LAN **Primary Repository:** <https://github.com/rdkcentral/lan-manager-lite>

25.1 Overview

Connected Device Discovery maintains a real-time inventory of all devices connected to the gateway's LAN — both wired and wireless. This inventory is exposed through the TR-181 Device.Hosts data model table, which records each device's MAC address, IP address, hostname, connection type, and associated interface.

The lan-manager-lite component drives device discovery through multiple detection mechanisms: ARP monitoring for wired clients, DHCP lease tracking for IP assignment events, and Wi-Fi association event consumption from OneWiFi for wireless clients. Join and leave events are generated in near-real-time as devices connect and disconnect, enabling operators and applications to respond to fleet-level connectivity events.

Secure UPnP (secure-upnp) extends device visibility by providing UPnP service discovery while enforcing security policies that prevent malicious UPnP abuse. Applications and the local WebUI consume the Device.Hosts table to display connected device lists and enable per-device management features.

Note: The lan-manager-lite component only supports discovery of devices that are connected directly to the gateway via wired or wireless connections. It does not support discovery of devices that are connected to the LAN via EasyMesh extenders. This is supported in the EasyMesh components.

25.2 Key Capabilities

- Device.Hosts TR-181 table: real-time inventory of all LAN-connected devices
- Multi-method detection: ARP, DHCP lease events, Wi-Fi association events
- Device join/leave events with near-real-time latency from connection to discovery
- Per-device metadata: MAC address, IP address, hostname, interface type, active/inactive state
- Secure UPnP service discovery with security policy enforcement
- Integration with parental controls and local WebUI for per-device management actions

25.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Device.Hosts Table								
Device.Hosts TR-181 data model	lan-manager-lite	—	Required	Required	Required	Required	—	—
Device Discovery								
Detect devices joining/leaving LAN	lan-manager-lite	—	Required	Required	Required	Required	—	—
Secure device discovery								
Secure UPnP services	secure-upnp	—	OPT	OPT	OPT	OPT	—	—

25.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- lan-manager-lite
- secure-upnp
- dhcp-manager
- OneWiFi

25.6 Testing Scope

- Device join detection: connect wired client and validate Device.Hosts entry within SLA
- Device leave detection: disconnect client and validate entry marked inactive within SLA
- Wi-Fi client: associate Wi-Fi device, validate Device.Hosts entry with correct SSID/BSSID
- DHCP integration: validate IP address and hostname from DHCP lease match Device.Hosts
- Secure UPnP: validate device discovery while testing for port mapping abuse prevention

26. Feature 15: System startup & orchestration

Sub-System: System **Primary Repository:** <https://github.com/rdkcentral/provisioning-and-management>

26.1 Overview

System Startup and Orchestration manage the complete device boot sequence, from kernel initialization through to all RDK-B middleware components reaching their operational state. The startup framework must correctly sequence service dependencies, handle partial boot failures, and emit system-ready signals at appropriate milestones.

The provisioning-and-management (P&M) scripts package is the incumbent startup implementation, containing the shell scripts that initialize interfaces, apply persistent configuration, start service daemons, and configure networking at boot time. P&M is acknowledged as a legacy component — it is monolithic, difficult to extend, and scheduled for refactoring into smaller, modular components.

The Utopia framework provides the sysEvent IPC mechanism used by P&M scripts for inter-script communication during the boot sequence, and the sysConfig key-value store for persistent configuration. Like P&M, Utopia is a legacy component with planned deprecation. The component-registry (system-ready signaling) and miscellaneous-broadband (platform init) components provide additional portions parts of the startup infrastructure.

26.2 Key Capabilities

- Boot sequence orchestration: ordered service startup with dependency resolution
- sysEvent IPC: event-based inter-script communication for boot stage coordination
- sysConfig: persistent key-value configuration store for network and system settings
- component-registry: system-ready event signaling for components waiting on dependencies
- miscellaneous-broadband: platform-specific initialization tasks (MAC setup, RNG seed, etc.)
- LED management: device status indication via LED state management component

26.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Startup Scripts								
Device init & provisioning scripts ▶ Legacy – will be split/deprecated	provisioning-and-management	Required	Required	Required	Required	Required	OPT	OPT
sysEvent IPC + sysConfig storage ▶ Legacy – deprecation planned	utopia	Required	Required	Required	Required	Required	OPT	OPT
Service Registry								
System-ready event signaling	component-registry	—	Required	Required	Required	Required	—	—

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
▸ Legacy (dbus era) – not for new dev								
Platform Misc								
Miscellaneous platform init ▸ Refactor candidate	miscellaneous-broadband	Required	Required	Required	Required	Required	Required	Required
LED Management								
LED state management	led-manager	OPT	OPT	OPT	OPT	OPT	OPT	OPT

26.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- provisioning-and-management
- utopia
- component-registry
- miscellaneous-broadband
- led-manager

26.5 Implementation Notes

Device init & provisioning scripts: Legacy – will be split/deprecated

sysEvent IPC + sysConfig storage: Legacy – deprecation planned

System-ready event signaling: Legacy (dbus era) – not for new dev

Miscellaneous platform init: Refactor candidate

26.6 Testing Scope

- Cold boot: validate all services reach operational state within boot time ATC
- sysEvent: validate events are delivered to registered listeners during boot sequence
- component-registry: validate system-ready signal is emitted only when all components are up
- LED management: validate correct LED state transitions at each boot stage
- Power cycle soak: validate clean boot/shutdown cycle across 100 iterations without hang

27. Feature 16: Configuration Persistence

Sub-System: System **Primary Repository:** <https://github.com/rdkcentral/persistent-storage-manager>

27.1 Overview

Configuration Persistence ensures that all device settings configured by operators, subscribers, or management systems survive device reboots, power cycles, and controlled resets. Without reliable persistence, every reboot would return the device to factory defaults, requiring complete reconfiguration.

PSM (Persistent Storage Manager) is a frequently used persistence mechanism for RDK-B. It provides a typed key-value store with RBUS-accessible get/set/notify APIs, factory-default management, and atomic write semantics to prevent configuration corruption on unexpected power loss.

sysConfig (via Utopia) is the legacy flat-file key-value persistence mechanism still used by P&M scripts and older components. It is being progressively replaced but remains required for backward compatibility with existing components during the migration period.

27.2 Key Capabilities

- PSM: typed key-value persistence with RBUS APIs, factory-default management, atomic writes
- PSM factory reset: restore all PSM-managed parameters to their factory default values
- sysConfig: legacy flat-file key-value store for P&M script compatibility
- Power-loss safe writes: atomic operations prevent configuration corruption on unexpected shutdown
- Component-specific namespace partitioning prevents configuration collisions

27.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Config Storage								
Persistent config + factory reset ▶ Not all components use PSM	persistent-storage-manager	Required	Required	Required	Required	Required	OPT—	OPT—
sysConfig								
Persistent config ▶ Not all components use sysConfig	utopia	Required	Required	Required	Required	Required	OPT—	OPT—

27.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- persistent-storage-manager
- utopia

27.5 Implementation Notes

Persistent config + factory reset: Not all components use PSM

Persistent config: Not all components use sysConfig

27.6 Testing Scope

- PSM write/reboot/read cycle: validate value survives power cycle
- Factory reset: validate PSM returns all values to factory defaults
- Power-loss simulation: interrupt write mid-operation, validate config integrity on next boot
- Concurrent access: multiple components writing to PSM simultaneously — validate no corruption
- sysConfig legacy compatibility: validate existing P&M scripts read/write sysConfig correctly

28. Feature 17: Factory reset

Sub-System: System **Primary Repository:** <https://github.com/rdkcentral/persistent-storage-manager>

28.1 Overview

Factory Reset restores the device to its original factory-default software configuration state, clearing all subscriber-specific settings, operator-applied configurations, and persisted application data. Factory reset is a critical operational tool used for device reuse (subscriber moves), defect resolution, and end-of-life data erasure. RDK-B implements factory reset through a coordinated multi-component process: PSM restores all its managed parameters to their recorded factory defaults; Utopia/sysConfig clears its persistent configuration files; P&M scripts execute platform-specific reset actions such as clearing wireless credentials and resetting network interface configuration; and the device reboots to apply the clean state. Factory reset can be triggered through multiple paths: local physical button press, local WebUI action, remote management protocol trigger (USP, WebPA, TR-069), or RFC/xConf command. All trigger paths must produce an identical end state.

28.2 Key Capabilities

- PSM factory default restoration: all PSM parameters restored to their factory-default values
- sysConfig/Utopia erasure: persistent configuration files cleared
- P&M reset scripts: platform-specific reset actions (Wi-Fi credentials, network config)
- Multiple trigger paths: physical button, WebUI, USP/TR-069 command, RFC/xConf
- Controlled reboot: device restarts cleanly after reset completion

28.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Config Reset								
Restore factory default config ▶ PSM manages factory defaults	persistent-storage-manager/utopia/PSM	Required	Required	Required	Required	Required	OPT—	OPT—
Factory reset scripts ▶ Via Utopia/P&M scripts	persistent-storage-manager/utopia/PSM	Required	Required	Required	Required	Required	OPT—	OPT—

28.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- persistent-storage-manager

- utopia
- provisioning-and-management

28.5 Implementation Notes

Restore factory default config: PSM manages factory defaults

Factory reset scripts: Via Utopia/P&M scripts

28.6 Testing Scope

- Factory reset from each trigger path (button, WebUI, USP, WebpA, TR-069) — validate identical end state
- Post-reset state: validate all Wi-Fi credentials, IP settings, and parental controls are cleared
- Post-reset boot: device boots cleanly and reaches operational state after reset
- Factory reset timing: reset completes within 180s ATC
- Multi-cycle: 10 consecutive factory reset cycles without degradation

29. Feature 18: IP Routing/NAT/Firewall

Sub-System: Routing & Bridging **Primary Repository:** <https://github.com/rdkcentral/core-net-library>

29.1 Overview

IP Routing, NAT, and Firewall form the data-plane of the RDK-B routers and gateways, responsible for forwarding traffic between WAN and LAN interfaces, translating private LAN addresses to the public WAN address, and protecting LAN devices from unsolicited inbound traffic.

IPv4 and IPv6 routing rules are managed by provisioning-and-management scripts working in concert with the WAN Manager. When the WAN interface type changes (e.g., failover from primary to secondary WAN), the routing table is updated accordingly. The core-net-library provides a zero-dependency utility library for interface, bridge, VLAN, and route management operations used by multiple RDK-B components.

The stateful packet filter (firewall) is implemented using iptables (and transitionally nftables) to enforce security policies protecting LAN devices. NAT/PAT (Network Address Translation/Port Address Translation) maps LAN device connections through the single WAN IP. Port forwarding and port triggering rules enable LAN-hosted services to be accessible from the WAN.

29.2 Key Capabilities

- IPv4 + IPv6 dual-stack routing rules with WAN interface change tracking
- Stateful packet filtering (iptables/nftables): SYN flood protection, connection tracking, zone-based policies
- NAT/PAT: LAN-to-WAN address translation for all LAN devices through single WAN IP
- Port forwarding: static mapping of WAN port to LAN device/port for hosted services
- Port triggering: dynamic port mapping activated by outbound trigger traffic
- core-net-library: zero-dependency interface, bridge, VLAN, and route utility functions
- DMZ host configuration: single LAN device exposed to all WAN ports

29.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Mode m /ONU	EthW AN WiFi Router	GW	GW OpenSy nc	GW EasyMe sh	EXT OpenSy nc	EXT EasyMe sh
Network Utilities								
Interface/bridge/VLAN /route utils ▶ Zero dependencies	core-net-library	—	Require d	Requir ed	Required	Required	Required	OPT
IPv4/IPv6 Routing								
IPv4 + IPv6 routing rules ▶ Legacy – still required today	provisioning-and-management	---	Require d	Requir ed	Required	Required	—	—
Route management utilities	core-net-library/Utopia	—	Require d	Requir ed	Required	Required	—	—
Firewall								
Stateful packet filtering	provisioning-and-management,Utopia	---	Require d	Requir ed	Required	Required	—	—

Feature Detail	Repository	Modem /ONU	Ethernet/WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
NAT / PAT	provisioning-and-management/Utopia	---	Required	Required	Required	Required	—	—
Port forwarding								
Port forwarding rules ▸ Via P&M iptables rules	provisioning-and-management/Firewall/Utopia	—	Required	Required	Required	Required	—	—
Port triggering rules ▸ Via P&M iptables rules	provisioning-and-management/Firewall/Utopia	—	Required	Required	Required	Required	—	—

29.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- core-net-library
- provisioning-and-management
- utopia
- wan-manager

29.5 Implementation Notes

Interface/bridge/VLAN/route utils: Zero dependencies

IPv4 + IPv6 routing rules: Legacy – still required today

Port forwarding rules: Via P&M iptables rules

Port triggering rules: Via P&M iptables rules

29.6 Testing Scope

- IPv4 routing: validate LAN client can reach internet via WAN after gateway boot
- IPv6 routing: validate LAN client receives IPv6 prefix and can reach IPv6 internet
- NAT: validate multiple LAN clients can simultaneously access the internet through one WAN IP
- Firewall: validate unsolicited inbound SYN packets are dropped at WAN interface
- Port forwarding: configure rule, validate external access to LAN-hosted service on specified port
- Port triggering: trigger outbound traffic, validate dynamic inbound port mapping activation
- WAN failover: validate routing table updates correctly when primary WAN fails

30. Feature 19: Bridge and VLAN configuration

Sub-System: Routing & Bridging **Primary Repository:** <https://github.com/rdkcentral/wan-manager>
<https://github.com/rdkcentral/miscellaneous-broadband/>

30.1 Overview

Bridge and VLAN Configuration enables the gateway to operate in network topologies beyond the standard NAT router mode. WAN bridge mode disables the NAT and routing functions, passing frames transparently between WAN and LAN interfaces — used by operators for scenarios where a separate router handles IP routing.

VLAN configuration enables traffic segmentation across the gateway's LAN and WAN interfaces, supporting operator use cases such as separating residential traffic from business traffic on a shared access link, or isolating IoT device traffic from primary LAN devices.

30.2 Key Capabilities

- WAN bridge mode: transparent MAC-level bridging between WAN and LAN without NAT
- VLAN tagging and untagging on LAN and WAN interfaces
- Bridge interface creation and management for multi-interface bridging scenarios
- VLAN-to-interface mapping for traffic segmentation

30.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Bridge mode support (no router/NAT)								
WAN bridge mode configuration	wan-manager/bridge utils	—	OPT	OPT	OPT	OPT	—	—
Bridge Utilities								
VLAN/bridge interface management	core-net-library	—	Required	Required	Required	Required	Required	Required

30.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- wan-manager
- core-net-library
- miscellaneous-broadband

30.6 Testing Scope

- WAN bridge mode: validate LAN client receives IP address from upstream DHCP server (not gateway)
- VLAN: validate tagged traffic on VLAN-configured interface is isolated from untagged traffic
- Bridge: validate frames bridge correctly between two specified interfaces

31. Feature 20: Traffic redirection

Sub-System: Routing & Bridging **Primary Repository:** —

31.1 Overview

Traffic Redirection enables the gateway to intercept subscriber HTTP traffic and redirect it to a captive portal for operator notification or authentication purposes. This is used in "no service" scenarios — when the WAN connection is technically functional, but the subscriber's service has been suspended, restricted, or not yet activated.

Two redirection modes are supported: network-based captive portal redirection (forwarding to an operator-hosted portal on the WAN), and local captive portal redirection (serving a locally hosted portal when there is no DOCSIS RF WAN signal at all, i.e., in full offline scenarios).

31.2 Key Capabilities

- Network-based captive portal: redirect HTTP traffic to operator-hosted notification portal
- Local captive portal: serve locally hosted portal page when no WAN connectivity is present
- DNS-based redirect for portal URL resolution before WAN routing is available

31.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Forward user traffic to network based captive portal								
For "no service" and other similar use cases	provisioning-and-management	—	OPT	OPT	OPT	OPT	—	—
Forward user traffic to local Captive portal when no DOCSIS RF WAN								
No RF Captive portal	provisioning-and-management	—	N/A	OPT	OPT	OPT	—	—

31.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- provisioning-and-management

31.6 Testing Scope

- Network captive portal: simulate no-service WAN state, validate HTTP redirect to portal URL
- Local portal: simulate no-RF state, validate locally hosted page served on redirect
- HTTPS handling: validate redirect behavior for HTTPS requests (browser certificate warning expected)

32. Feature 21: DHCP Server

Sub-System: LAN **Primary Repository:** <https://github.com/rdkcentral/dhcp-manager>

32.1 Overview

DHCP Server provides automatic IP address assignment and network configuration delivery to all LAN-connected devices. The dhcp-manager component implements both DHCPv4 and DHCPv6 server functionality, handling the complete address lifecycle: discover/offer/request/acknowledge for IPv4, and solicit/advertise/request/reply for IPv6.

IPv6 prefix delegation (PD) enables the gateway to request an IPv6 prefix from the upstream ISP network and sub-delegate portions of that prefix to LAN devices for SLAAC or DHCPv6 address formation. When the WAN connection fails over to a secondary WAN, the delegated prefix may change — dhcp-manager handles this transition gracefully, renewing LAN device prefixes with minimal service interruption.

IPv6 SLAAC (Stateless Address Autoconfiguration) support enables LAN devices to self-configure IPv6 addresses from the delegated prefix using their interface MAC address, without requiring explicit DHCP exchange. This is complementary to DHCPv6 and both may be active simultaneously.

32.2 Key Capabilities

- DHCPv4 server: address pool management, lease allocation, renewal, and release
- DHCPv6 server: IA_NA (non-temporary address) and IA_PD (prefix delegation) support
- DHCPv4 options: DNS servers, domain name, NTP server, vendor-specific options (43, 60, 82)
- IPv6 prefix delegation (PD): upstream prefix request and LAN sub-delegation
- PD change handling on WAN failover/restore; graceful prefix transition to LAN clients
- IPv6 SLAAC: Router Advertisement with delegated prefix for stateless address configuration
- Static DHCP lease assignment based on MAC address for known LAN devices

32.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
DHCP Server for LAN Clients								
DHCPv4 server	dhcp-manager	—	Required	Required	Required	Required	—	—
DHCPv6 servr	dhcp-manager	—	OPT	OPT	OPT	OPT	—	—
IPv6 prefix delegation	dhcp-manager	—	Required	Required	Required	Required	—	—
IPv6 prefix delegation change on WAN failover/restore	dhcp-manager	—	OPT	OPT	OPT	OPT	—	—
IPv6 SLAAC support (stateless address autoconfig)	dhcp-manager	—	Required	Required	Required	Required	—	—

32.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- dhcp-manager
- wan-manager

32.6 Testing Scope

- DHCPv4: client discovers, receives offer, requests, and receives ACK with valid IP in pool
- DHCPv4 lease renewal: client renews lease before expiry, validate same IP retained
- DHCPv6 IA_NA: client receives unique IPv6 address in delegated prefix
- Prefix delegation: gateway requests /56 from upstream, delegates /64 to LAN
- PD change on WAN failover: trigger failover, validate LAN clients receive updated prefix
- SLAAC: validate Router Advertisement is sent with correct prefix and prefix length
- DHCP options: validate DNS server option matches gateway DNS relay address

33. Feature 22: DNS

Sub-System: DNS **Primary Repository:** <https://github.com/rdkcentral/dhcp-manager>

33.1 Overview

DNS services in RDK-B provide local name resolution for LAN clients, enabling them to use the gateway as their DNS resolver rather than communicating directly with upstream DNS servers. The gateway acts as a DNS relay, forwarding queries it cannot resolve locally to configured upstream DNS servers.

DNS-based parental controls (xdns) extend the local DNS relay with content filtering capability, enabling subscribers and operators to block access to categories of content or specific domains by returning negative responses to DNS queries for blocked domains. xdns integrates with configurable upstream DNS filtering services that categorize domains by content type.

Dynamic DNS (DDNS) enables subscribers with dynamic WAN IP addresses to maintain a stable DNS hostname that resolves to their gateway's current WAN IP, enabling remote access to home-hosted services without a fixed IP address.

33.2 Key Capabilities

- DNS relay: forward LAN client queries to configured upstream DNS servers
- Local DNS override: respond to specific hostname queries with locally configured addresses
- DNS-based parental controls (xdns): category-based content filtering via upstream filtering DNS
- Per-device DNS filtering policy: different filtering rules per LAN device MAC address
- DDNS: dynamic DNS record update on WAN IP change via standard DDNS provider APIs
- DNS-over-HTTPS (DoH) proxy support for privacy-preserving resolution

33.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
DNS								
DNS relay for LAN clients	dhcp-manager/PandM	—	OPT	OPT	OPT	OPT	—	—
DNS-based parental controls	xdns	—	OPT	OPT	OPT	OPT	—	—
DDNS	provisioning-and-management/Utopia	—	OPT	OPT	OPT	OPT	—	—

33.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- dhcp-manager
- provisioning-and-management
- xdns

- utopia

33.6 Testing Scope

- DNS relay: LAN client query forwarded to upstream and response returned correctly
- DNS negative caching: validate NXDOMAIN is cached and not re-queried for TTL period
- xdns: configure block list, attempt query for blocked domain, validate blocked response
- DDNS: trigger WAN IP change, validate DDNS provider receives update within SLA
- Upstream failover: block primary DNS, validate failover to secondary DNS upstream

34. Feature 23: WAN Connectivity

Sub-System: WAN **Primary Repository:** <https://github.com/rdkcentral/wan-manager>

34.1 Overview

WAN Connectivity is the broadest feature area in RDK-B, encompassing all access technologies, WAN protocol support, multi-WAN orchestration with failover, IPv6 transition mechanisms, and DHCP client-side address acquisition. The wan-manager component is the central orchestrator for all WAN connectivity, managing a configurable set of primary and secondary WAN profiles and handling failover/restore transitions transparently.

Access technology support covers the full range of broadband access types: Ethernet WAN (for fiber and Ethernet-delivered services), DOCSIS cable modem (cable-modem-agent), xDSL (xdsl-manager), GPON (gpon-manager), EPON (CcspEPONAgent, legacy), Qualcomm QMI cellular (cellular-manager), MBIM generic cellular (Cellular Modem Mgr), USB RNDIS cellular dongles, and Wi-Fi as a WAN backup link. Each access technology is managed by a dedicated agent that exposes WAN link state, signal quality, and statistics via RBUS.

IPv6 transition mechanisms address the challenge of providing IPv6 connectivity to LAN clients in networks where the ISP only provides IPv4 WAN addresses. MAP-T (Mapping of Address and Port using Translation) and MAP-E (encapsulation variant) translate IPv6 packets to IPv4+port for upstream transport. DS-Lite encapsulates IPv4 within IPv6. 464XLAT (CLAT on the gateway, PLAT in the network) enables IPv4 applications to work over an IPv6-only WAN.

34.2 Key Capabilities

- WAN profile orchestration: configurable primary and secondary WAN profiles selected at boot time
- WAN profile orchestration: configurable primary and secondary WAN profiles with automatic failover/restore
- WAN failover: primary failure detection within 30s, secondary activation within 60s
- Ethernet WAN: managed by Ethernet Agent for fiber/cable handoff scenarios
- DOCSIS: cable-modem-agent for cable modem status, events, and CM-status reporting
- xDSL: xdsl-manager for ADSL/VDSL modem status, line statistics, and diagnostics
- GPON: gpon-manager for GPON ONU registration, signal level, and link state
- EPON: CcspEPONAgent for EPON ONU management (legacy)
- Cellular QMI: cellular-manager for Qualcomm QMI modem management, data sessions, GPS
- Cellular MBIM: generic MBIM modem management for non-Qualcomm cellular modems
- USB RNDIS: cellular dongle connectivity via USB RNDIS network interface
- Wi-Fi WAN: OneWiFi STA mode as an upstream WAN link for backup connectivity
- DHCP client IPv4: WAN address acquisition, renewal, and release
- DHCP client IPv6/PD: upstream prefix delegation request
- PPPoE: PPP over Ethernet for DSL and some fiber operator deployments
- GRE tunnelling: WAN GRE tunnel for public hotspot encrypted WAN scenarios
- MAP-T / MAP-E: IPv4 traffic mapping over IPv6 WAN with shared address space
- DS-Lite: IPv4-in-IPv6 encapsulation for IPv6-only WAN access networks
- 464XLAT: CLAT function enabling IPv4 applications over IPv6-only WAN

34.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Profile based definition of primary and backup WAN links								
WAN profile orchestration	wan-manager	—	Required	Required	Required	Required	—	—
Primary WAN								
Primary WAN	wan-manager	—	Required	Required	Required	Required	—	—
Secondary WAN								
Backup WAN	wan-manager	—	OPT	OPT	OPT	OPT	—	—
WAN Failover and Restore								
Failover and restore use cases	wan-manager	—	OPT	OPT	OPT	OPT	—	—
DHCP Client (WAN-side IP acquisition)								
DHCP Client – WAN IPv4 acquisition	dhcp-manager	—	OPT	OPT	OPT	OPT	—	—
DHCP Client – WAN IPv6 / prefix delegation	dhcp-manager	—	OPT	OPT	OPT	OPT	—	—
IPv6 transition/translation mechanisms								
MAP-T	DHCP Manager	—	OPT	OPT	OPT	OPT	—	—
MAP-E	DHCP Manager	—	OPT	OPT	OPT	OPT	—	—
DSLITE	DHCP Manager	—	OPT	OPT	OPT	OPT	—	—
464XLAT	DHCP Manager	—	OPT	OPT	OPT	OPT	—	—
WAN Protocol – PPP								
PPP over Ethernet / ATM	ppp-manager	—	OPT	OPT	OPT	OPT	—	—
WAN Protocol – GRE								
WAN GRE tunnel	hotspot	—	OPT	OPT	OPT	OPT	—	—
Access Technology - Ethernet WAN								
Ethernet WAN interface mgmt	Ethernet Agent	—	Required	OPT	OPT	OPT	—	—
Access Technology – DOCSIS								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Cable modem status & events	cable-modem-agent	OPT	N/A	OPT	OPT	OPT	—	—
Access Technology – xDSL								
xDSL modem status & diag	xdsl-manager	OPT	N/A	OPT	OPT	OPT	—	—
Access Technology – EPON								
EPON ONU management ▸ Legacy	CcspEPONAgent	OPT	N/A	OPT	OPT	OPT	—	—
Access Technology – GPON								
GPON WAN interface mgmt	gpon-manager	OPT	N/A	OPT	OPT	OPT	—	—
Access Technology – Cellular								
Qualcomm QMI cellular	cellular-manager	OPT	N/A	OPT	OPT	OPT	—	—
GPS ▸ Location detection	cellular-manager	OPT	N/A	OPT	OPT	OPT	—	—
MBIM generic cellular	Cellular Modem Mgr	OPT	N/A	OPT	OPT	OPT	—	—
USB RNDIS	Cellular Modem Mgr	OPT	N/A	OPT	OPT	OPT	—	—
Access Technology - WiFi link to external WAN backup device								
WiFi used as a WAN link	OneWiFi WAN Manager	OPT	N/A	OPT	OPT	OPT	—	—

34.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- wan-manager
- dhcp-manager
- Ethernet Agent
- cable-modem-agent
- xdsl-manager
- gpon-manager
- cellular-manager
- cellular-modem-manager
- ppp-manager
- hotspot

- OneWiFi

34.5 Implementation Notes

EPON ONU management: Legacy

GPS: Location detection

34.6 Testing Scope

- Ethernet WAN: DHCP address acquisition and internet reachability
- WAN failover: primary WAN link-down event triggers secondary within SLA
- WAN restore: primary WAN recovers, validate failback and routing table correction
- DHCP client IPv4: discover/offer/request/ack cycle, lease renewal, expiry handling
- DHCP client IPv6/PD: upstream PD request and /64 sub-delegation to LAN
- MAP-T: configure MAP-T rule, validate IPv4 traffic is correctly mapped to IPv6 for WAN transit
- 464XLAT: validate IPv4-only application connectivity over IPv6-only WAN
- PPPoE: dial-up, authentication, and DNS acquisition via PPP
- Cellular: QMI data session establishment, IP address assignment, and internet reachability

35. Feature 24: Parental control

Sub-System: Parental Control **Primary Repository:** <https://github.com/rdkcentral/xdns>

35.1 Overview

Parental Controls provide operators and subscribers with tools to manage and restrict internet access on a per-device basis, protecting children from inappropriate content and enforcing household internet usage policies.

DNS-based filtering (xdns) receives DNS responses from the network based DNS with additional proprietary codes that indicate that domains are in operator-configured content categories. This approach is effective, lightweight, and cross-platform — it works for any device on the LAN as long as the device uses un-encrypted DNS, without requiring any client-side software installation. It does require a network based DNS service with these additional capabilities. This method does not support systems that use encrypted DNS traffic.

Device Pause allows subscribers to instantly cut internet access for a specific LAN device without disconnecting it from the network or changing any device settings. Managed sites, blocked keywords, and trusted device exemptions provide fine-grained control over which sites and devices are subject to filtering policies.

35.2 Key Capabilities

- DNS-based filtering (xdns): category-based content blocking for all LAN devices using non-encrypted DNS
- Device Pause: per-device internet access pause without network disconnection
- Managed sites: per-device allow/block site lists overriding default policy
- Blocked keywords: URL keyword-based blocking for domains matching configured strings for LAN devices using non-encrypted DNS.
- Trusted devices: exemption list for devices that bypass all parental control filtering
- Scheduling: time-of-day and day-of-week filtering schedules per device

35.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
DNS Filtering								
DNS-based parental control rules	xdns	—	OPT	OPT	OPT	OPT	—	—
Device Pause								
Device Pause	provisioning-and-management/Utopia	—	OPT	OPT	OPT	OPT	—	—
Managed sites								
Managed sites	provisioning-and-management/Utopia	—	OPT	OPT	OPT	OPT	—	—
Blocked keywords								

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Blocked keywords	provisioning-and-management/Utopia	—	OPT	OPT	OPT	OPT	—	—
Trusted devices								
Trusted devices	provisioning-and-management/Utopia	—	OPT	OPT	OPT	OPT	—	—

35.4 Component Dependencies

The following RDK-B components are required or closely related to this feature:

- xdns
- provisioning-and-management
- utopia

35.6 Testing Scope

- DNS filtering: configure category block, query blocked domain from LAN, validate blocked response
- Device Pause: pause specific device, validate internet blocked only for that MAC
- Trusted device: add device to trusted list, validate filtering bypassed for that MAC only
- Managed sites: add domain to per-device block list, validate blocked from that device only
- Keyword block: configure keyword, validate URL containing keyword is blocked

36. Feature 25: WiFi Hotspot

36.1 Overview

WiFi Hotspot provides public Wi-Fi SSID support and Passpoint / Hotspot 2.0 capability for operator deployments. This feature enables ISPs to offer public hotspot access over the same CPE used for residential broadband, supporting GRE tunnel-based traffic isolation to the operator backhaul. Encrypted WAN is required for security when deploying Public Hotspot functionality, because of this requirement, Public Hotspot functionality is not supported for routers using Ethernet WAN connectivity

36.2 Key Capabilities

Public Wi-Fi SSID (hotspot). Passpoint / Hotspot 2.0 via OneWiFi (GAS/ANQP query/response, Hotspot 2.0 indication element). WAN GRE tunnel for traffic isolation to operator backhaul (hotspot). Radius Failover and IPv6 support for public access.

36.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSync	GW EasyMesh	EXT OpenSync	EXT EasyMesh
Public Wi-Fi SSID	hotspot	n/a	n/a	OPT	OPT	OPT	OPT	OPT
Passpoint / Hotspot 2.0	OneWiFi	n/a	n/a	OPT	OPT	OPT	OPT	OPT

36.4 Component Dependencies

hotspot, OneWiFi, wan-manager (GRE tunnel support)

36.5 Implementation Notes

Encrypted WAN is required for security when Public Hotspot is deployed. Public SSID must be fully isolated from residential LAN traffic. Passpoint / Hotspot 2.0 requires GAS/ANQP support in OneWiFi and a valid Hotspot 2.0 configuration from the operator.

37. Feature 26: Code Security

37.1 Overview

Code Security encompasses the cryptographic, privilege, and runtime protection mechanisms that harden the RDK-B platform against exploitation. It includes certificate management for mTLS, secure system call wrappers, privilege-dropping utilities, and AppArmor profiles. libSyscallWrapper is a CORE component — a small, zero-external-dependency library used as basic infrastructure by most other components, required across all device profiles.

37.2 Key Capabilities

Secure system call wrapper (libSyscallWrapper) — CORE, required across all profiles. Certificate selector (rdk-cert-config) for mTLS configuration. Secure crypto API wrapper stubs for ISP-specific implementations (rdk-cert-config). Security Services Architecture (rdkssa) for MSO-specific crypto and secure mount. Privilege dropping for non-root process operation (rdk-libunpriv). AppArmor mandatory access control profiles (rdkb-apparmor-profiles).

37.3 Feature Detail and Profile Matrix

Feature Detail	Repository	Modem /ONU	EthWAN WiFi Router	GW	GW OpenSyn c	GW EasyMes h	EXT OpenSyn c	EXT EasyMes h
secure system call (libSyscallWrapper)	libSyscallWrapper	Required	Required	Required	Required	Required	Required	Required
Certificate selector (mTLS)	rdk-cert-config	OPT	OPT	OPT	OPT	n/a	n/a	n/a
Privilege drop (rdk-libunpriv)	rdk-libunpriv	OPT	OPT	OPT	OPT	n/a	n/a	n/a
AppArmor profiles	rdkb-apparmor-profiles	OPT	OPT	OPT	OPT	n/a	n/a	n/a

37.4 Component Dependencies

libSyscallWrapper (CORE), rdk-cert-config, rdkssa, rdk-libunpriv, rdkb-apparmor-profiles

37.5 Implementation Notes

libSyscallWrapper is a CORE dependency — a small, zero-external-dependency library forming basic infrastructure used by most RDK-B components. rdk-cert-config and rdkssa are optional and required only for mTLS deployments where ISP-specific crypto backends are needed. Privilege dropping via rdk-libunpriv should be applied to all new components running as non-root. AppArmor profiles provide mandatory access control and should be included in all production builds.

38. Security Framework

Security is a mandatory design requirement across the RDK-B platform. Every component is expected to meet defined security baselines. This section defines the security framework, requirements, and validation approach applicable to all RDK-B components and device profiles.

38.1 Security Principles

- **Secure by Default:** all interfaces, services, and credentials are locked down at factory default; no open ports, no default passwords, no insecure services enabled without explicit operator configuration
- **Defense in Depth:** security controls exist at multiple layers — hardware (secure boot), firmware (signed images), middleware (firewall, process isolation), management (TLS), and application (container isolation)
- **Least Privilege:** each process runs with the minimum required Linux capabilities and filesystem permissions; no unnecessary root processes
- **Patch Currency:** CVE tracking and patch application are mandatory for all components; patch SLA is 30 days for Critical, 90 days for High severity
- **Open Audit:** all security-sensitive code is open-source and subject to community review

38.2 Wi-Fi Security Requirements

Security Mode	Profile Requirement	Notes
WPA2-Personal	Required for GW, EthWAN WiFi Router, all EasyMesh/OpenSync profiles	Baseline minimum for all Wi-Fi-capable profiles
WPA2-Enterprise (802.1X)	OPT for all Wi-Fi profiles	RADIUS server integration required; not typically deployed on residential profiles
WPA3-Personal (SAE)	Required for GW and above	Mandatory for all new gateway deployments; replaces WPA2-Personal as default on new SSIDs
WPA3-Enterprise	OPT	For high-security business gateway deployments
WPA3-Transition	Required for GW and above	Required to support mixed WPA2/WPA3 client environments during transition
WPA3 Enhanced Open (OWE)	OPT	For public Wi-Fi SSID deployments; provides opportunistic encryption
WPA3 PCM	OPT	Post-Connection Management; additional security controls post-association
PMKSA Caching	OPT for all Wi-Fi profiles	Required if fast roaming is needed in EasyMesh environments

38.3 Management Channel Security

- All management protocols (WebPA, USP, TR-069, WebConfig) **MUST** use TLS 1.2 or higher — no plain-text management channels permitted
- Server certificate validation is mandatory for all TLS connections — self-signed certificates only permitted during manufacturing provisioning
- USP controller authentication uses mutual TLS (mTLS) — device certificate and controller certificate both validated

- Local WebUI: HTTPS only, with HSTS header; HTTP redirected to HTTPS; session tokens invalidated on logout
- RBUS: local socket access controlled by Linux file system permissions; remote RBUS bridge (inter-device-manager) requires authentication

38.4 Firmware Security

- All firmware images must be signed with operator-approved RSA-2048 or ECDSA-P256 key pair
- Signature verification is performed before any firmware image is written to flash storage
- Secure boot chain validates each boot stage (bootloader → kernel → rootfs) against stored signing keys
- OTA download must occur over HTTPS; HTTP downloads rejected even if signature verification passes
- A/B partition scheme ensures a verified fallback firmware is always available for automatic rollback

38.5 Container Security (App Framework)

- OCI container runtime (Dobby/crun) enforces Linux namespace isolation for all downloadable applications
- Containers run without host network access by default — network access explicitly granted per container by operator policy
- Root filesystem is mounted read-only for all containers; writable volumes are explicitly declared
- Linux capabilities are dropped to the minimum set required — CAP_NET_ADMIN and CAP_NET_RAW not granted by default
- Seccomp filters applied to all containers to restrict available system calls
- cgroup resource limits (CPU, memory, I/O) enforced per container to prevent denial-of-service from misbehaving apps

38.6 Security Testing Requirements

Security Test Area	Test Requirements
Firewall Validation	Validate all default-blocked ports are unreachable from WAN; validate stateful connection tracking allows return traffic; validate SYN flood defense activates under load
Wi-Fi Security Handshake	Full WPA3-SAE handshake protocol trace; validate PMK derivation and MIC verification; test with non-compliant supplicant versions
Management Channel TLS	Certificate validation enforcement; TLS version downgrade prevention; cipher suite conformance
Firmware Signature	Reject tampered image (bit-flip in payload); reject image signed with expired key; reject image with unknown signer
Container Isolation	Attempt container escape via /proc, /sys, /dev; validate namespace isolation prevents host process visibility; validate filesystem isolation
Credential Security	Validate no default/blank passwords exist at factory; validate SSID credentials are cryptographically random
CVE Scanning	Static analysis (cppcheck, clang-tidy) for components; OpenSSL, libcurl, kernel version currency checks against CVE database

39. Performance Requirements

Performance requirements define the minimum acceptable operational characteristics for each device profile. These requirements represent the floor — production implementations should exceed these values.

39.1 System Performance

Metric	Target	Measurement Method
Cold Boot Time	< 120 seconds from power-on to fully operational (WAN up, Wi-Fi beaconing, RBUS ready)	Automated HIL timer from power-on to first successful RBUS get
Warm Restart Time	< 45 seconds from reboot trigger to operational state	Automated HIL timer from reboot signal to RBUS ready
WAN Failover Time	< 60 seconds from primary WAN failure to secondary WAN traffic forwarding	Automated ping continuous stream; measure gap duration
WAN Restore Time	< 30 seconds from primary WAN recovery to primary WAN selected	Automated ping measurement from link-up event to restore complete
Memory Usage at Idle	< 60% of available RAM consumed by RDK-B middleware at steady state	top/free command via remote_debugger; 30-minute steady-state measurement
CPU Usage at Idle	< 15% average CPU utilization with no active traffic or diagnostics	mpstat 30-second average via remote_debugger
RBUS get/set Latency	< 100ms round-trip for local TR-181 parameter get/set under normal load. This is frequently < 1 mS.	Automated measurement across 1000 sequential get/set operations
Management Command SLA	Configuration change reflected in running config within 30 seconds of cloud delivery	USP command issued to ACS; timestamp when RBUS parameter reflects change

Notes:

- Cold Boot Time and Warm Restart Time do not include time for the WAN link to be re-established and connect. Some WAN types may take considerably longer than the times shown here.
- Many of these performance targets depend on hardware CPU performance, the total memory available and other factors that may vary depending on hardware support.

39.2 Wi-Fi Performance Requirements

Profile / Band	Minimum Throughput	Test Conditions
Wi-Fi 6 (802.11ax) — 5 GHz	600 Mbps aggregate (uplink + downlink)	Single client, 3m distance, anechoic chamber, iPerf3 TCP
Wi-Fi 6E (802.11ax) — 6 GHz	800 Mbps aggregate	Single client, 3m distance, anechoic chamber, iPerf3 TCP
Wi-Fi 7 (802.11be) — 5+6 GHz MLO	1.2 Gbps aggregate	Single client MLO capable, 3m, anechoic chamber, iPerf3 TCP
EasyMesh Backhaul (5 GHz)	400 Mbps	Extender to gateway backhaul link, iPerf3 TCP
EasyMesh End-to-End	300 Mbps	Client on extender through mesh to gateway WAN, iPerf3 TCP

Profile / Band	Minimum Throughput	Test Conditions
Client Association Time	< 3 seconds from scan to associated and DHCP complete	Automated client association measurement, 10-trial average
Roaming Time (802.11kv)	< 150ms BSS transition time	FTAA (Fast Transition Authenticated) measurement, 10-trial average

Note: Minimum Throughput may vary due to device hardware capabilities, WiFi congestion, WiFi SNR and other factors.

39.3 WAN Performance Requirements

Metric	Target	Notes
Routing Throughput	Line rate for the WAN interface (1 Gbps for GbE WAN, 2.5 Gbps for multi-gig)	IMIX traffic profile, hardware acceleration expected
NAT Sessions	Minimum 50,000 concurrent NAT sessions	Measured under sustained load with session table inspection
DHCP Lease Acquisition	< 10 seconds from DHCPDISCOVER to DHCPACK with IP assigned to LAN client	Measured on LAN client from DHCP request to IP reachable
IPv6 RA Response	< 2 seconds from client RS to receiving RA with prefix	Measured from RS send to prefix available on client interface
DNS Response Time	< 50ms for cached responses, < 200ms for forwarded queries	Measured from LAN client, against local resolver; 100-query average

Note: WAN performance may vary due to WAN link speed, congestion, device hardware acceleration capabilities, device CPU capabilities and other factors.

39.4 Reliability Requirements

Metric	Target
Uptime / MTBF	> 99.9% uptime over 30-day fleet sample (< 43 minutes downtime per device per month)
Crash Rate	< 0.1% unplanned restarts per device per day
OTA Success Rate	> 99% successful firmware update completions; 100% rollback on failure (no brick)
Log Upload Reliability	> 99.5% DCM log upload success rate over 30-day fleet sample
Factory Reset Reliability	100% clean post-reset boot across 50-cycle soak test
Memory Stability	No memory leak exceeding 1MB/hour over 7-day continuous operation soak

Note: Reliability metrics may vary due to device temperature, device manufacturing quality, WAN network issues and other factors.

40. RDKB Component Testing (draft)

Testing is a first-class citizen in the RDK-B platform. The RDKB Component Testing defines the mandatory validation framework for the RDK Platform layer — covering component behavior, data model correctness, integration, regression stability, and production readiness. Every component requires documented test coverage before release.

Testing Philosophy RDK-B platform testing is structured around "test the interface, not the implementation." Component tests validate TR-181 data model behavior and RBUS API correctness. Integration tests validate end-to-end feature correctness across the platform stack. This layering ensures failures are localized and attributable. HAL implementation testing is outside this scope and is governed by the HAL specification.

40.1 Test Framework Architecture

The RDK Platform Test Suite covers L2 and L3. L4 is operator-owned. L1 HAL validation is vendor-owned and governed separately.

Layer	Description	Owner
L4: End-to-End / Acceptance	Complete user scenarios from cloud management to device behavior. Operator ODQ campaigns.	Operator + SI
L3: Integration / Feature	Interactions between two or more RDK-B components across the platform layer.	RDK Community + OEM
L2: Component / Unit	Single component in isolation. TR-181 data model, RBUS API, state machine, sysevent behavior, component level HAL interaction	Component Owner
L1: Vendor Layer HAL Validation	Governed by HAL specification and Vendor Test Suite. Out of scope for this section.	SoC Vendor

40.2 RDK Platform Test Suite *(Mandatory for all Components — L2 + L3)*

The RDK Platform Test Suite defines the mandatory test coverage for every RDK-B component. A component cannot be declared tested and cannot be included in an RDKB release without evidence of coverage across all six categories below.

40.2.1 Mandatory Test Categories

Category 1 — Data Model

- All TR-181 parameters that are provided in the component are readable (with the rare exception of parameters that are defined as non-readable).
- Parameter count is stable across builds — any deviation is a blocking regression
- Run time Auto-discovery of the data model provided by the component matches the component documentation. This includes all objects, properties, eventing and methods provided by the component.
- The component registers the correct component name with the bus on start-up.
- TR-181 values are consistent with runtime state at all times

- Write operations reflect in runtime state within a defined time bound
- Write operators or methods that are expected to cause the component to interact with a HAL actually occur at the HAL API within a defined time bound and all in-range values written to the data model accurately occur at the HAL API.
- Dynamic multi-instance objects provided by the component can be added and removed successfully (This does not apply to static multi-instance objects.)
- Multi-instance objects that support index aliases work correctly.
- The count value of multi-index objects works correctly
- All sync and async methods provided by the component work correctly. The input and output values supported by all methods match the component documentation.
- All event subscriptions provided by the component work correctly.
- All properties provided by the component that support event “on-change” or event “on-interval” work correctly.
- Property values provided in event notifications are correct.
- RBUS CLI may be used to exercise the components data model including get, set, add instance, remove instance, subscribe, receive notification, invoke sync method, invoke async method.

Category 2 — State Machine

- Every state is reachable and externally observable via RBUS or sysevent
- Every transition fires on the correct trigger and completes within defined timing
- A stable state exists for every degraded or partial input condition — no thrash, no spin

Category 3 — RBUS and Events

- All get/set operations return correct values
- Events fire on state change within the defined timing window
- All registrations and subscriptions are restored correctly after component restart
- No orphaned subscriptions or resource leaks after restart or failure

Category 4 — E2E Scenarios

- Happy path — trigger to operational state
- Failover — primary fails, secondary activates within SLA
- Restore — primary recovers within SLA
- Restart recovery — component resumes correct state after restart
- Stress — 20 repeated cycles with no memory leak and no hung state

Category 5 — Negative and Boundary

- Component operates correctly when optional dependencies are absent
- Invalid inputs rejected and correct error returned
- Behavior correct at minimum and maximum values of all timeout and retry constants
- No resource leak after failure or teardown
- No new sysevent keys or usage is included in the component. Sysevent will eventually be deprecated. This test is designed to make sure that no new sysevent usage is added to existing components.

Category 6 — Post-Reboot

- PSM-persisted configuration restored correctly after reboot
- Component reaches operational state within platform boot SLA
- All owned RBUS parameters and sysevent keys repopulated with live values after reboot with the exception of parameters that are defined as non-persistent.

40.2.2 Timing SLA Gates

Every component must define timing SLAs from source and enforce them as blocking CI gates:

- State transition time
- Feature activation time
- Failover and restore time
- Restart recovery time
- RBUS get/set latency

SLA values must be documented per component. CI fails if any SLA regresses.

40.2.3 Test Evidence Required for Release

For every component the following must be attached to the release ticket:

- Test case list with pass/fail result per category
- Timing SLA values and CI gate results
- Stress cycle result (minimum 20 cycles)
- Post-reboot assertion result
- Known failures with root cause and disposition

No component is release-eligible without this evidence.

40.3 OneWiFi Test Suite (*OneWiFiTestSuite*)

40.3.1 OneWiFi Test Suite (*Non-EasyMesh tests*)

40.3.2 OneWiFi Test Suite (*EasyMesh Controller tests*)

40.3.3 OneWiFi Test Suite (*EasyMesh Agent tests*)

OneWiFiTestSuite (github.com/rdkcentral/OneWiFiTestSuite) is the dedicated platform test suite for the Wi-Fi LAN feature area, covering L2 and L3 for OneWiFi and EasyMesh components. It covers:

- Radio bring-up and configuration across all supported bands (2.4 / 5 / 5L / 5U / 6 GHz)
- Security: WPA2-Personal, WPA2-Enterprise, WPA3-Personal (SAE), WPA3-Enterprise, WPA3-Transition, WPA3 Enhanced Open (OWE), WPA3-PCM

- Multi-BSSID (VAP) creation, configuration, and isolation
- DFS radar detection and channel switch announcement (CSA) compliance
- 802.11kv neighbor reports and BSS transition management frames
- EasyMesh R6: Controller/Agent pairing, topology convergence, STA steering, channel utilization reporting
- Wi-Fi 7: 320 MHz channel, SLO/MLO link setup/teardown, data model parameter validation
- Passpoint / Hotspot 2.0: GAS/ANQP query/response, Hotspot 2.0 indication element validation
- Band steering: trigger conditions, steering request generation, and client response handling
- WPS PBC and PIN mode: session initiation, credential exchange, and AP configuration update
- Support for TR-181 “Data Elements” section that is required for the Easy Mesh Controller.

40.4 Regression Campaign Structure

Campaign	Duration	Trigger	Scope
Smoke Test	10–30 min	Per build (daily CI)	Critical path: boot, WAN connectivity, Wi-Fi association, RBUS get/set, basic routing
Sanity Test	1–2 hrs	Per branch merge	All features for target profile; config persistence, factory reset
Feature Regression	4–8 hrs	Weekly or per RC	Full feature matrix for target profile including edge cases and negative tests
Full Platform Regression	1–3 days	Per release	Complete matrix across all profiles and SoC platforms, including stress and soak
Operator ODQ Campaign	1–4 weeks	Per operator qualification milestone	Operator-specified security, performance, interoperability, and compliance

40.5 CI/CD Pipeline

40.5.1 Build CI

- Every component repository runs unit tests and static analysis (cppcheck, clang-tidy) on each push
- Yocto/BitBake builds triggered for each platform layer change, validating compile-time correctness
- Binary size and memory footprint metrics tracked over time to detect regressions
- AUTOREV injection patterns automatically pull the latest validated HEAD of each component repository
- TR-181 parameter count validated per component on every build — deviation triggers blocking warning
- Timing SLA gates enforced as blocking checks — build fails on SLA regression

40.5.2 Hardware-in-the-Loop (HIL) CI

- Automated HIL labs at RDK Central and major operator/OEM facilities run smoke and sanity tests on every nightly build
- RDK Platform Test Suite Categories 1–3 (Data Model, State Machine, RBUS) run on every nightly HIL build
- Results published to a central dashboard; failures automatically create bug tickets

- Test environments defined as code (Docker containers for test controller, network topology definitions) for reproducibility

40.5.3 TIP Wi-Fi Validation

- TIP validation campaigns validate Wi-Fi interoperability between RDK-B implementations and third-party clients/APs
- OneWiFi AP interoperability tested against OneWiFi client and AP modes

40.6 Lab Topology

- Device Under Test (DUT): the gateway running the RDK-B build under test
 - WAN Simulator: DOCSIS CMTS, Ethernet WAN switch, DSL DSLAM, or cellular access point emulator
 - LAN Test Clients: wired and wireless clients for traffic generation and connectivity validation
 - Test Controller: Linux server running Python-based RDK-B L2/L3 test harness, connected to DUT via SSH and RBUS
 - Traffic Generator: iperf3, Spirent, or IXIA for throughput and latency benchmarking
 - Packet Capture: Wireshark/tcpdump at WAN, LAN, and management interfaces for protocol-level validation
-

40.7 Acceptance Test Criteria (ATC)

ATC Item	Acceptance Criterion
Boot Time	Operational state within 120 seconds of cold-start power-on
WAN Failover	Secondary WAN active within 60 seconds of primary failure; traffic resumption confirmed
Wi-Fi Throughput (Wi-Fi 6)	Minimum 600 Mbps aggregate at 3m in anechoic chamber
Wi-Fi Throughput (Wi-Fi 7)	Minimum 1.2 Gbps aggregate MLO at 3m in anechoic chamber
Memory at Idle	< 60% of available RAM consumed by RDK-B middleware
RBUS Latency	< 100ms round-trip for TR-181 get/set under normal load
Management Command SLA	Configuration change reflected within 30 seconds of cloud delivery
Log Upload Reliability	> 99.5% DCM upload success rate over 30-day fleet sample
Crash Rate	< 0.1% unplanned restarts per device per day over deployed fleet
Factory Reset	Clean operational state within 180 seconds; 50-cycle soak without failure
OTA Success Rate	> 99% success rate; automatic rollback on failure — no bricked devices
Memory Stability	< 1MB/hour memory leak over 7-day continuous soak
Data Model Stability	TR-181 parameter count per component stable across three consecutive builds
State Machine Observability	All SM states observable via RBUS or sysevent within one poll interval
Restart Recovery	Component operational state restored within 30 seconds of restart
Stress Cycle Stability	20 failover/restore cycles with no memory growth and no hung state

41. Legacy Component Migration Plan

The RDK-B platform carries a set of legacy components that were written before the current architectural standards were established. These components use outdated IPC mechanisms, monolithic structure, and have excessive build-time dependencies. This section identifies each legacy component, its replacement path, and migration status.

Legacy Component	Issue	Migration Target	Status
Provisioning and Management (P&M)	Monolithic shell scripts; thousands of lines with undefined scope; contains networking, firewall, routing, and system init mixed together	Split into discrete components: network-init, firewall-manager, routing-manager; each with RBUS interface	Planned — RDK9 program
Utopia	sysEvent IPC and sysConfig storage; legacy IPC mechanism not based on RBUS; difficult to test	Replace sysEvent with RBUS event pub/sub; replace sysConfig with PSM	In Progress — RBUS migration ongoing Phase 1: Remove SysEvent keys that are never read. Phase 2: Remove SysEvent keys that are read but never written. Phase 3: Migrate sysEvent keys that already have equivalent data models supported by RBUS. Migrate sysConfig usage by these keys. Phase 4: Migrate SysEvent keys that require new data models. Migrate sysConif usage by these keys. Deprecate the sysEvent and sysConfig libraries.
CCSP Common Library (common-library)	Excessive memory allocation; legacy CCSP IPC shim; needed only for backward compatibility with old components	Phase1: Update this library to use the RBUS APIs rather than the older CCSP APIS. This will improve performance and reduce bus latency. Phase 2: Eliminate this library as components migrate to native RBUS APIs	Phase 1: Not started Phase 2: Gradual progress as selected components are migrated individually
CCSP WiFi Agent (CcspWiFiAgent)	Duplicate with OneWiFi; legacy CCSP-based Wi-Fi management; hard-coded dependencies	Fully replaced by OneWiFi	In Progress: — CcspWiFiAgent will be deprecated in RDK9
Harvester	Hard-coded WebPA telemetry pipeline; not profile-configurable; legacy approach	Replaced by T2 Telemetry 2.0 profile-driven framework	Legacy — retained for backward compatibility only; not for new dev

Legacy Component	Issue	Migration Target	Status
SNMP Protocol Agent	Legacy SNMP management; not recommended for new operator deployments	Replaced by the choice of WebPA, USP/TR-369 or TR-069 for operator management infrastructure	Complete: This has been marked as deprecated by the Remote Management working group. The code is still available but not actively supported
Cnotify-component	Hard-coded DM value change notification; not flexible	Replaced by RBUS event pub/sub subscriptions	Legacy — deprecation planned
CcspEPONAgent (EPON)	Legacy EPON ONU agent using CCSP IPC	Replace with RBUS-native EPON manager if required	Legacy — not for new dev; EPON largely obsolete
DMCLI (data-model-cli)	Legacy CCSP command-line data model access tool	Replace with RBUS CLI tool	Legacy — available for backward compat; RBUS CLI preferred Phase 1: Educate the community on how to use RBUSCLI for debugging. Phase 2: Migrate scripts that use DMCLI to use RBUS CLI.

41.1 Migration Principles

- No new components should use SysEvent, sysConfig, DMCLI, or CCSP Common Lib IPC — all new code must use RBUS and PSM.
- Existing components are migrated incrementally — priority given to components with the most downstream dependents. Steps are taken to ensure that no legacy usage is added with new software contributions. Component tests will add a new scan to ensure that no new usage is added in each software contribution.
- Each migrated component must pass the complete existing test suite plus new RBUS-based interface tests before the legacy IPC is removed
- Migration tracking: a published dependency matrix is maintained and updated with each release showing current IPC/storage mechanism per component
- No component is marked migrated unless it has zero remaining sysEvent/DMCLI/Common Lib usage verified by CI static analysis

42. Component Repository Reference

All repositories are hosted at <https://github.com/rdkcentral/> unless otherwise noted.

Component / Repository	GitHub URL
OneWiFi	https://github.com/rdkcentral/OneWiFi
unified-wifi-mesh	https://github.com/rdkcentral/unified-wifi-mesh
ieee1905-rs	https://github.com/rdkcentral/ieee1905-rs
mesh-agent	https://github.com/rdkcentral/mesh-agent
hotspot	https://github.com/rdkcentral/hotspot
OneWifiTestSuite	https://github.com/rdkcentral/OneWifiTestSuite
ethernet-agent	https://github.com/rdkcentral/ethernet-agent
moca-agent	https://github.com/rdkcentral/moca-agent
rdkb-halif-moca	https://github.com/rdkcentral/rdkb-halif-moca
rdkb-halif-test-moca	https://github.com/rdkcentral/rdkb-halif-test-moca
webui	https://github.com/rdkcentral/webui
webui-bwg	https://github.com/rdkcentral/webui-bwg
parodus / start-parodus / xmidt-org	https://github.com/xmidt-org/parodus2ccsp https://github.com/xmidt-org/parodus https://github.com/rdkcentral/start-parodus
usp-pa-vendor-rdk	https://github.com/rdkcentral/usp-pa-vendor-rdk
tr069-protocol-agent	https://github.com/rdkcentral/tr069-protocol-agent
snmp-protocol-agent	https://github.com/rdkcentral/snmp-protocol-agent
xconf-client / rfc	https://github.com/rdkcentral/xconf-client https://github.com/rdkcentral/rfc
WebConfig / WebconfigFramework	https://github.com/rdkcentral/webconfig https://github.com/rdkcentral/WebconfigFramework
open-virtual-switch-agent	https://github.com/rdkcentral/open-virtual-switch-agent
telemetry (T2)	https://github.com/rdkcentral/telemetry
crashupload	https://github.com/rdkcentral/crashupload
harvester	https://github.com/rdkcentral/harvester
dcm-agent	https://github.com/rdkcentral/dcm-agent
test-and-diagnostic	https://github.com/rdkcentral/test-and-diagnostic
rdk-speedtest-cli	https://github.com/rdkcentral/rdk-speedtest-cli
ipoe-health-check	https://github.com/rdkcentral/ipoe-health-check
remote_debugger / rdm-agent	https://github.com/rdkcentral/remote_debugger https://github.com/rdkcentral/rdm-agent
cpuprocanalyzer	https://github.com/rdkcentral/cpuprocanalyzer
notify-component	https://github.com/rdkcentral/notify-component
sys_mon_tools	https://github.com/rdkcentral/sys_mon_tools
breakpad_wrapper	https://github.com/rdkcentral/breakpad_wrapper
rdk_logger	https://github.com/rdkcentral/rdk_logger

syslog_helper	https://code.rdkcentral.com/r/plugins/gitiles/rdk/components/generic/syslog_helper/
rbus	https://github.com/rdkcentral/rbus
data-model-cli (DMCLI)	https://github.com/rdkcentral/data-model-cli
common-library	https://github.com/rdkcentral/common-library
inter-device-manager	https://github.com/rdkcentral/inter-device-manager
json-hal-library	https://github.com/rdkcentral/json-hal-library
BartonCore	https://github.com/rdkcentral/BartonCore
BartonCommon	https://github.com/rdkcentral/BartonCommon
home-security	https://github.com/rdkcentral/home-security
DSM	https://github.com/rdkcentral/DSM
Dobby	https://github.com/rdkcentral/Dobby
media-terminal-adapter-agent	https://github.com/rdkcentral/media-terminal-adapter-agent
rdkb-halif-mta	https://github.com/rdkcentral/rdkb-halif-mta
rdkb-halif-test-mta	https://github.com/rdkcentral/rdkb-halif-test-mta
telco-voice-manager	https://github.com/rdkcentral/telco-voice-manager
hal-voice-asterisk	https://github.com/rdkcentral/hal-voice-asterisk
power-manager	https://github.com/rdkcentral/power-manager
rdk-halif-power_manager	https://github.com/rdkcentral/rdk-halif-power_manager
rdk-halif-test-power_manager	https://github.com/rdkcentral/rdk-halif-test-power_manager
platform-manager	https://github.com/rdkcentral/platform-manager
rdkb-halif-fwupgrade	https://github.com/rdkcentral/rdkb-halif-fwupgrade
rdkb-halif-test-fwupgrade	https://github.com/rdkcentral/rdkb-halif-test-fwupgrade
lan-manager-lite	https://github.com/rdkcentral/lan-manager-lite
secure-upnp	https://github.com/rdkcentral/secure-upnp
provisioning-and-management	https://github.com/rdkcentral/provisioning-and-management
utopia	https://github.com/rdkcentral/utopia
component-registry	https://github.com/rdkcentral/component-registry
miscellaneous-broadband	https://github.com/rdkcentral/miscellaneous-broadband
sysint-broadband	https://github.com/rdkcentral/sysint-broadband
common_utilities	https://github.com/rdkcentral/common_utilities
led-manager	https://github.com/rdkcentral/led-manager
persistent-storage-manager	https://github.com/rdkcentral/persistent-storage-manager
core-net-library	https://github.com/rdkcentral/core-net-library
firewall-manager	https://github.com/rdkcentral/firewall-manager
wan-manager	https://github.com/rdkcentral/wan-manager
ppp-manager	https://github.com/rdkcentral/ppp-manager
cable-modem-agent	https://github.com/rdkcentral/cable-modem-agent
xdsl-manager	https://github.com/rdkcentral/xdsl-manager

gpon-manager	https://github.com/rdkcentral/gpon-manager
cellular-manager	https://github.com/rdkcentral/cellular-manager
cellular-modem-manager	https://github.com/rdkcentral/cellular-modem-manager
dhcp-manager	https://github.com/rdkcentral/dhcp-manager
xdns	https://github.com/rdkcentral/xdns
advanced-security	https://github.com/rdkcentral/advanced-security
libSyscallWrapper	https://github.com/rdkcentral/libSyscallWrapper
rdk-cert-config	https://github.com/rdkcentral/rdk-cert-config
rdkssa	https://github.com/rdkcentral/rdkssa
rdk-libunpriv	https://github.com/rdkcentral/rdk-libunpriv
rdkb-apparmor-profiles	https://github.com/rdkcentral/rdkb-apparmor-profiles

43. Revision History

Version	Date	Author	Description
1.0	2026-06	RDK Central	V1.0 update: 26 features across 7 device profiles. Aligned all profile matrix cells with RDK-B_Component_List_2026_V1. Added Feature 2 (WiFi Hotspot), Feature 25 (Parental Control / Browsing Security), Feature 26 (Code Security). Updated Features column definition. Corrected 40+ profile cell values across Wi-Fi, Diagnostics, DHCP, Routing, and IPC features. Updated Features column tags for miscellaneous-broadband, sysint-broadband, common_utilities.